Adaptively-Secure Three-Round Threshold Schnorr from DL

Guilhem Niot[1,2], Michael Reichle[3], Kaoru Takemure[1,4]

[1]PQShield
[2]Univ Rennes, CNRS, IRISA
[3]ETH Zurich
[4]AIST

Abstract

Threshold signatures are an important tool for trust distribution, and preserving the interface of standardized signatures, such as Schnorr signatures, is crucial for their adoption. In practice, latency dominates end-to-end signing time, so minimizing the number of interaction rounds is critical. Ideally, this is achieved under minimal assumptions and with adaptive security, where the adversary can corrupt signers on-the-fly during the protocol.

While Schnorr signatures are proven secure under the Discrete Logarithm (DL) assumption in the random oracle model, the most round-efficient adaptively-secure threshold Schnorr protocols rely on stronger assumptions such as Decisional Diffie-Hellman (DDH), the Algebraic One-More Discrete Logarithm (AOMDL) or even the Low-Dimensional Vector Representation (LDVR) assumptions. The only adaptively-secure threshold Schnorr signature from the DL assumption requires five rounds, leaving a significant gap in our understanding of this fundamental primitive. Achieving low-round protocols with adaptive security from the DL assumption has remained an open question.

We resolve this question by presenting the first adaptively-secure threshold Schnorr scheme in three rounds (two online, one offline) in the random oracle model under the DL assumption. Our result demonstrates that achieving both low round complexity and adaptive security is possible while preserving the (so far) minimal assumptions for Schnorr signatures. To achieve this, we introduce new techniques, including a novel use of an equivocal commitment scheme paired with a simulation-extractable NIZK, and a masking-based aggregated opening strategy for homomorphic commitments. Our work also makes several contributions that might be of independent interest, including a formalization of a strong adaptive security notion, a stronger commitment equivocation property, and an analysis of the simulation-extractability of the randomized Fischlin transformation.

This is the full version of a paper to appear in the proceedings of EUROCRYPT 2026. It additionally contains the formal definitions and detailed proofs deferred from the published version.

Contents

1 Introduction

A T-out-of-N threshold signature [Des90, DF90] enables to issue a signature σ on message M for a joint verification key vk in a distributed manner. The desired security notion for threshold signatures is unforgeability, *i.e.*, even in the presence of up to $T-1$ signers it is hard to forge a signature. It is desirable that the issued signature σ and the key vk follow the format of common signatures: this ensures the threshold signature scheme is compatible with existing systems. In this work, we focus on Schnorr signatures due to their widespread use and efficiency. We note that Schnorr is secure in the random oracle model (ROM) under the discrete logarithm assumption (DL).

Static vs Adaptive Security. In the static corruption model, the adversary declares up to $T-1$ signers as corrupted at the start of the game. This however does not reflect the real world in which signers might be corrupted late into the deployment process of the threshold signature, potentially after several signatures were issued. Adaptive security is a more realistic security notion in which the adversary can corrupt up to $T-1$ signers at an arbitrary point of the scheme's lifetime. Unfortunately, it has been notoriously harder to provably achieve this stronger security notion. For threshold Schnorr, this reflects in the need to make concessions in terms of assumptions (*e.g.*, DDH instead of DL), the number of rounds in the signing protocol, the use of secure erasures, or the reliance on additional idealized models, such as the algebraic group model (AGM).

Round Complexity. Other than the achieved level of security, threshold signatures should be efficient. Here, a key performance metric is round complexity. In distributed systems, network latency often dominates computation time, making each round of interaction a significant bottleneck. Under static corruptions, the state of the art for threshold Schnorr under minimal assumptions (*i.e.*, DL) is three rounds [CKM23]. The recent work by Bacho et al. [BDLR25a] also achieves three rounds under adaptive corruptions without relying on secure erasures, but relies on the stronger DDH assumption. Under DL, the state of the art with adaptive security suffers from *five* communication rounds [KRT24]. We note that this gap exists even if we consider idealized models like the algebraic group model.

1.1 Our Contributions

We show that static and adaptive threshold Schnorr signatures can enjoy *both* small round complexity and security from minimal assumptions. That is, we present the first three-round, adaptively-secure threshold Schnorr signature scheme, coined Rackle, from the DL assumption in the random oracle model. In summary, Rackle satisfies the following properties:

One offline and two online rounds. The signing protocol proceeds in three communication rounds. The first round can be preprocessed (*i.e.*, ran independent of the message and signer set).

Strong adaptive security from DL. Our scheme is secure under the DL assumption in the ROM under adaptive corruptions. We consider strong unforgeability [NT24], where the adversary $\mathcal{A}$ is tasked to output ℓ signatures $\sigma_1, \ldots, \sigma_\ell$ on some message m. To classify the forgeries as trivial, we adapt the (to the best of our knowledge) strongest notion in the literature [ABHR25] to adaptive corruptions. The forgeries are declared as trivial only if the adversary completed (at least) ℓ *consistent* signing sessions for the message m; a session only counts as *consistent* if the honest signers agree on the set of co-signers SS, the signed message M, and some postfix session-identifier ssid.

No secure and authenticated channels. In our model, the adversary $\mathcal{A}$ observes all sent honest protocol messages and can freely divert and alter sent messages.

No secure erasures. All random coins ρ_i sampled by signer i within the lifespan of the protocol are revealed to the adversary $\mathcal{A}$ on corruption. This strengthens the usual definition where the honest

Table 1: Comparison of adaptively secure threshold Schnorr signatures in the ROM. Communication complexity is per-party.

Scheme	Rounds (online+offline)	Online Comm.	Offline Comm.	Corruption	No Erasures	Assumption
ZeroS† [Mak22]	(3+0)	$T\lvert\mathbb{Z}_p\rvert + \lvert\mathbb{G}\rvert + 2\lambda$		T	✗	DL
HARTS [BLSW24]	$\mathcal{O}(1)$	$\lvert\mathbb{Z}_p\rvert$	$\mathcal{O}(1)(\lvert\mathbb{Z}_p\rvert + \lvert\mathbb{G}\rvert)$	$T/3$	✗	OMDL + AGM
Sparkle‡ [CKM23]	(2+1)	$3\lvert\mathbb{Z}_p\rvert + \lvert\mathbb{G}\rvert$	2λ	$T/2$	✓	AOMDL
Frost [KG20, CKK+25]	(1+1)	$\lvert\mathbb{Z}_p\rvert$	$2\lvert\mathbb{G}\rvert$	$T/2$	✓	AOMDL
				T	✓	AOMDL + LDVR + AGM
Frost-Mask [Che25]	(2+1)	$\lvert\mathbb{Z}_p\rvert + 2\lvert\mathbb{G}\rvert$	2λ	T	✓	AOMDL
Crackle [KRT24]	(4+1)	$3\lvert\mathbb{Z}_p\rvert + \lvert\mathbb{G}\rvert + 2\lambda$	2λ	T	✓	DL
Glacius [BDLR25b]	(5+0)	$5\lvert\mathbb{Z}_p\rvert + 4\lvert\mathbb{G}\rvert + 6\lambda$		T	✓	DDH
Gargos [BDLR25a]	(3+0)	$6\lvert\mathbb{Z}_p\rvert + 2\lvert\mathbb{G}\rvert + 3\lambda$		T	✓	DDH
Gerhart et al.* [GCRS25]	(2+0)	$\lvert\mathbb{Z}_p\rvert + \lvert\mathbb{G}\rvert + \lvert\pi\rvert$		T	✓	DDH
Rackle	(2+1)	$3\lvert\mathbb{Z}_p\rvert$	$\lvert\pi\rvert + 2\lvert\mathbb{G}\rvert$	T	✓	DL

Group elements and field elements are denoted by $\mathbb{G}$ and $\mathbb{Z}_p$, respectively. Above, $|\pi| = \Theta(\lambda)(\lambda + |\mathbb{G}| + |\mathbb{Z}_p|)$ denotes the size of an SLE proof π. For $2\lambda = \log p = 256$, the proof size $|\pi|$ is roughly 3 KB given our instantiation. †: This scheme assumes private channels. ‡: The authors of [BLT+24] identified a flaw in the proof of (selective and adaptive) Sparkle. While the authors of [CKM23] did address the issue, the recent work by [BLT+24] note that the adaptive proof of Sparkle must be flawed for the full adaptive setting. *: This scheme considers a relaxed model where parties perform an interactive refresh of their individual public keys every d signatures.

states are revealed as the states are a function of ρ_i and $\mathcal{A}$'s view. [1]

We present a comparison with other threshold Schnorr signatures with adaptive security in Table 1. Our proposal is inspired by the recent five-round threshold Schnorr signature from [KRT24], and we develop new techniques to reduce the round complexity from five to three. Roughly, we employ an equivocal commitment paired with a simulation-extractable proof system to remove explicit dependencies on individual nonces, together with a masking-based technique to open only the aggregated commitment, in such a manner that sessions are forced to be consistent. We refer to Section 2 for a more detailed overview. Along the way, we make three notable side contributions, which may be of independent interests.

Strong Adaptive Security Notion. We formalize the strong unforgeability notion by [ABHR25] in the adaptive setting. This requires care when evaluating whether the forgeries are trivial.

Randomized Fischlin is simulation-extractable. The randomized Fischlin transformation [Ks22] allows to compile a Σ-protocol into a non-interactive zero-knowledge proof system (NIZK), denoted Π, that is straight-line extractable (SLE). Here, SLE refers to the property that the extractor can extract from proofs on-the-fly (*i.e.*, without rewinding). In this work, we show that Π satisfies a stronger notion, called simulation extractability (SIM-EXT): even in the presence of simulated proofs it is possible to extract a witness from non-simulated proofs on-the-fly. In fact, we show that the (with minor modifications) compiled proof system satisfies a SIM-EXT notion strengthened in two aspects.

1. We introduce labels which allow for more fine-grained control over which proofs are extractable.

[1] We note that often, the state allows to recompute the signer's random coins. In our work, the state does not include all sampled random coins and we therefore define the strengthened notion explicitly.

2. We show that simulated proofs are *explainable* assuming a stronger zero-knowledge property of the Σ-protocol, *i.e.*, the random coins employed to generate a proof can be simulated given a witness [HK22].

The latter property is important to avoid erasures (as observed in [BLSW24]).

Product Equivocation for BCJ Commitments. The commitment scheme BCJ [BCJ08] is homomorphic and allows to equivocate for Schnorr nonces R, meaning commitments can be simulated and later, opened to a (real or simulated) Schnorr nonce given an appropriate trapdoor. In this work, we show a stronger property: the product of n commitments can be equivocated to a simulated nonce, while giving an adversary free choice on which commitments within the product to open (up to $n-1$ times).

Future work. Before we proceed, let us discuss two possible directions for future work. First, our scheme does not provide identifiable abort. [2] We believe it is plausible that we can augment our protocol with an identifiable abort protocol with the techniques from [PKN+25], where the protocol is ran interactively *only* if signing fails to identify a cheating party. As this is out of scope, we leave a formal analysis for future work.

Second, our communication complexity is considerably larger than in prior threshold Schnorr signatures due to our reliance on SLE proofs. Future improvements on SLE proof size could decrease the communication complexity considerably. However, we note that these proofs are part of the offline communication and our *online* communication is only $3\mathbb{Z}_p$ elements.

1.2 Related Work

Threshold Schnorr signatures. There are some early works that have proposed threshold Schnorr signatures [SS01, AF04, GJKR07], relying on verifiable secret sharing. In particular, Abe and Fehr [AF04] proposed adaptively secure *robust* threshold Schnorr based on the DDH assumption without relying on secure erasures but the signing protocol only supports $T < N/2$ and consists of more than three rounds.

Recent years have seen a new surge in proposals for threshold Schnorr schemes achieving selective security. FROST [KG20] is the first two-round threshold signatures, which is proven secure under the (algebraic) one-more discrete logarithm (AOM-DL) assumption by [BCK+22]. There have been several follow-up works [RRJ+22, CGRS23], considering additional properties such as robustness and distributed key generation. Lindell [Lin22] proposed a three-round scheme which is secure in the UC model. Boneh et al. [BHLS25] proposed a two-round threshold Schnorr scheme based on *exponent VRFs*, which is a newly introduced primitive, and constructed from the Paillier encryption scheme [Pai99] or the DDH assumption.

Let us turn towards adaptive security. Crites et al. [CKM23] proposed three-round adaptively-secure scheme Sparkle based on the AOMDL assumption but achieve *half-adaptive* security, *i.e.*, the adversary is allowed to corrupt at most $T/2$ signers. Makriyannis [Mak22] proposed an adaptively-secure three-round scheme under secure erasures which is proven in the UC model. HARTS [BLSW24] is a half-adaptively-secure and robust scheme from the AOM-DL assumption and the AGM. Katsumate et al. [KRT24] proposed an adaptively-secure five-round *stateless* scheme from the DL assumption, which can be modified into a *stateful* four-round scheme assuming unique session identifiers. Chen [Che25] constructed a three-round stateless and two-round stateful scheme applying the technique of [KRT24] to Frost. Bacho et al. [BDLR25b] proposed an adaptively-secure five-round scheme Glacius from the DDH assumption while achieving identifiable abort, which allows detecting the misbehaving signer when the signing protocol aborts, in contrast to [KRT24]. In [BDLR25a], they improved the round complexity to three rounds with a new scheme named Gargos. Following the recent work [CS25] which focuses on adaptive attacks on existing threshold signature schemes, Crites et al. [CKK+25] show that the two-round scheme Frost is adaptively secure under the AOM-DL assumption in the AGM, and the hardness of the low-dimensional vector representation (LDVR) problem, which is newly defined in that work. They also showed that the LDVR problem is unconditionally hard under specific parameters, yielding half-adaptive security. Gerhart et al. [GCRS25] proposed a two-round scheme from the

[2] This property allows to identify misbehaving parties during the signing protocol.

DDH assumption which allows only a fixed number of signing interactions, after which the individual partial keys need to be refreshed.

Equivocal commitments. The simplest equivocal commitment in pairing-free curves are Pedersen commitments which allow to commit to messages $m \in \mathbb{Z}_p$. For our purpose, it is important that we can commit to group elements $M \in \mathbb{G}$. While this can be done (*e.g.*, via ElGamal), there is no scheme that allows for *arbitrary* equivocation for group elements. In our case, similar to prior works [BCJ08, DEF+19, BD21, TSS+24, CATZ24, PW24, PW24, BW25], we need to equivocate to group elements of a specific form. Such an equivocation notion is often referred to as *special* or *restricted*. Prior *special* or *restricted* equivocation properties do not consider equivocation for aggregated commitments, as required in our work.

Fischlin transformation. The Fischlin transformation [Fis05] allows to compile a Σ-protocol into an NIZK with straight-line extraction. For the transformation to retain zero-knowledge, the Σ-protocol must satisfy a strong *quasi-unique responses* property. To relax this requirement, [Ks22] introduce the *randomized* Fischlin transformation which relies on the (weaker) strong special soundness notion. It was pointed out in [Sch22] that the zero-knowledge analysis is flawed. The work by [KRW24] provide a corrected analysis, relying on an exponential-sized challenge space. The recent work [HOS25] takes a different approach: it introduces stronger security properties for the Σ-protocol (namely, *witness collision resistance* and *efficient re-simulatability*) to correct the zero-knowledge analysis. In exchange for stronger properties, their transformation also works for polynomial-sized challenge space. To the best of our knowledge, no prior work analyzed simulation-extractability of Fischlin's transformation.

2 Technical Overview

Denote by $\mathbb{G}$ a group of prime-order p with generator G. We employ multiplicative notation for $\mathbb{G}$. Recall that our goal is a three-round threshold Schnorr signature from minimal assumptions (*i.e.*, DL in the ROM). The five-round threshold Schnorr signature Crackle [KRT24] is proven adaptively secure under DL. Therefore, it serves as our starting point. We give a brief recap of Crackle below.

The threshold Schnorr signature Crackle. The verification key vk is a random group element $X = G^x$ as in Schnorr. The secret key x is distributed to each signer using Shamir's secret sharing [Sha79]. That is, each signer $i \in [N]$ is given x_i such that for any T-subset $\mathsf{SS} \subseteq [N]$, we have $x = \sum_{i \in \mathsf{SS}} L_{\mathsf{SS},i} x_i$. Here, $L_{\mathsf{SS},i}$ denotes the ith Lagrange coefficients for set SS. In addition, each signer i obtains random seeds $\vec{\mathsf{seed}}_i = (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]}$, where signers i and j share seeds $(\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})$, and an individual key pair $(\mathsf{vk}_{\mathsf{S},i}, \mathsf{sk}_{\mathsf{S},i})$ for a separate standard signature scheme. In summary, the secret key share for signer i is $\mathsf{sk}_i = (x_i, \vec{\mathsf{seed}}_i, \mathsf{sk}_{\mathsf{S},i})$ and the other users' verification key $\mathsf{vk}_{\mathsf{S},i}$ for $i \in [N]$.

The scheme Crackle relies on masking based on zero shares. Let $\mathsf{H}_{\mathsf{mask}}$ be a random oracle. [3] On input in, the tuple $\vec{\mathsf{seed}}_i$ allows each signer to generate a zero share Δ_i (*i.e.*, Δ_i is uniform conditioned on $\sum_{i \in \mathsf{SS}} \Delta_i = 0$) via

$$\Delta_i = \sum_{j \in \mathsf{SS}} \left(\mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{i,j}, \mathsf{in}) - \mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{j,i}, \mathsf{in}) \right).$$

We write $\Delta_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{in})$ short for the above. Looking ahead, it is vital that input in is fresh when signer i masks values v_i via $\tilde{v}_i = v_i + \Delta_i$ to hide individual v_i's. Also, let $\mathsf{H}_{\mathsf{com}} : \{0,1\}^* \to \{0,1\}^{2\lambda}$ and $\mathsf{H}_c : \{0,1\}^* \to \mathbb{Z}_p$ denote random oracles. We are now ready to recall Crackle's signing protocol. To sign on a message M with a signer set SS, Crackle proceeds as follows.

Round 1. Signer i outputs a random string $\mathsf{str}_i \leftarrow \{0,1\}^{2\lambda}$.

[3] $\mathsf{H}_{\mathsf{mask}}$ either outputs group elements or $\mathbb{Z}_p$ elements. We leave the output space implicit in the overview, but the space can be inferred from the context in the following. If $\mathsf{H}_{\mathsf{mask}}$ is employed to sample $\mathbb{G}$ elements, the zero share Δ_i is multiplicative.

Round 2. Signer i obtains $\mathsf{sid} := (\mathsf{str}_i)_{i \in \mathsf{SS}}$ and samples a nonce $R_i = G^{r_i}$ for $r_i \leftarrow \mathbb{Z}_p$ as in Schnorr. Then, it computes a zero share $\widetilde{\Delta}_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{sid}) \in \mathbb{G}$ and a *masked* commitment $\widetilde{R}_i = R_i \cdot \widetilde{\mathbf{\Delta}}_i$. It outputs a hash commitment $\mathsf{cmt}_i = \mathsf{H}_{\mathsf{com}}(i, \widetilde{R}_i)$.

Round 3. Signer i obtains $\widetilde{\mathsf{ctnt}} = (\mathsf{cmt}_j)_{j \in \mathsf{SS}}$ (e.g., the set of all hash commitments) and outputs a signature $\sigma_{\mathsf{S},i} \xleftarrow{\$} \mathsf{Sign}(\mathsf{sk}_{\mathsf{S},i}, \mathsf{sid} \| \widetilde{\mathsf{ctnt}})$.

Round 4. Signer i checks that for all $j \in \mathsf{SS}$ the signature $\sigma_{\mathsf{S},j}$ verifies with respect to $\mathsf{sid} \| \widetilde{\mathsf{ctnt}}$ it signed in Round 3. If so, it outputs the opening $\widetilde{R}_i$.

Round 5. Signer i checks that for all $j \in \mathsf{SS}$, the hash commitment cmt_j are opened correctly by signer j, *i.e.*, $\mathsf{cmt}_j = \mathsf{H}_{\mathsf{com}}(j, \widetilde{R}_j)$. If the check passes, it computes the aggregate commitment $R = \prod_{j \in \mathsf{SS}} \widetilde{R}_j$ and sets $\mathsf{ctnt} = \mathsf{sid} \| (\mathsf{cmt}_j, \widetilde{R}_j)_{j \in \mathsf{SS}}$. It then computes the challenge $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$, a zero share $\Delta_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt}) \in \mathbb{Z}_p$, a response $z_i = c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i$ and outputs the masked response $\widetilde{z}_i = z_i + \Delta_i$.

To aggregate the signing transcript to a Schnorr signature, it suffices to set $\sigma = (R, c, z)$, where R and c are set as above and $z = \sum_{i \in \mathsf{SS}} \widetilde{z}_i$. Then, σ verifies since $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$ and $R \cdot X^c = G^z$. This holds as

$$\prod_{i \in \mathsf{SS}} \widetilde{R}_i = \prod_{i \in \mathsf{SS}} G^{r_i} \cdot \widetilde{\Delta}_i = G^{\sum_{i \in \mathsf{SS}} r_i}, \qquad \sum_{i \in \mathsf{SS}} \widetilde{z}_i = \sum_{i \in \mathsf{SS}} z_i + \Delta_i = c \cdot x + \sum_{i \in \mathsf{SS}} r_i.$$

Crackle's proof strategy. The unforgeability proof of Crackle simulates signing such that it is possible to return random secret keys x_i on corruption. This is enabled by the masking technique: the entire transcript only leaks the information on the full signature $\sigma = (R, c, z)$. [4] Since σ can be simulated via honest-verifier zero-knowledge (HVZK), it is possible to simulate signing without knowing x. Rewinding then allows to solve DL as in the original security proof by Pointcheval and Stern for Schnorr signatures [PS00]. We provide a simplified overview of Crackle's simulation strategy. It consists of three major steps.

1. First, the game enforces that all honest signers employ a fresh sid in Round 2 to compute the zero share $\tilde{\Delta}_i$.

   This allows to argue that masked nonce $\widetilde{R}_i = R_i + \tilde{\Delta}_i$, except for the last one, is uniform random. Therefore, it holds that the individual R_i remain hidden until signer i's corruption, which we refer to by property hidden-nonces. [5]

2. Next, the game enforces that all honest signers with sid in Round 3, eventually finishing Round 4, employ fresh input ctnt for ZeroShare and that they all use the same unique challenge c. This allows to argue that every masked response $\widetilde{z}_i$ except for the last one is uniform random and that the aggregated response z can be expressed using the (full) secret key x, as opposed to using the secret key shares x_i.

   Again, freshness of ctnt ensures that z_i remains hidden until signer i's corruption, which we refer to by property hidden-resps. To enforce that all signers agree on challenge c, it suffices to enforce that all agree on the aggregate nonce $R = \prod_{i \in \mathsf{SS}} \widetilde{R}_i$. We refer to this by property unique-aggr-nonce.

3. Lastly, Katsumata et al. [KRT24] employ properties hidden-nonces, hidden-resps, and unique-aggr-nonce together to show that signing can be simulated by HVZK. We note that here, property unique-aggr-nonce is vital as it determines the query $(\mathsf{vk}, \mathsf{M}, R)$ for which to embed the simulated c. At the same time, property hidden-nonces and hidden-resps ensure that nothing more than the simulated signature (R, c, z) is leaked, making it possible to handle corruption queries.

[4] We note that R may be omitted from the signature σ, however, we consider as part of the signature for exposition.

[5] We note that after $T - 1$ corruptions, the honest R_i may be leaked. This does not conflict with the proof strategy because the adversary cannot make further corruption queries at this point.

The need for five rounds. Let us give some intuition why Crackle's five rounds seem necessary. The second, fourth and fifth round correspond roughly to the classical commit-and-open protocol for interactive Schnorr signatures and forms the foundation for the issuance protocol.

Let us see why we cannot avoid masking R_i (*i.e.*, removing property hidden-nonces) to omit Round 1. Observe that if adversary $\mathcal{A}$ corrupts signer i, it obtains (among others) the key share x_i and seeds $\vec{\mathsf{seed}}_i$. $\mathcal{A}$ can then compute Δ_i to reveal z_i. Since $z_i = c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i$ and $R_i = G^{r_i}$ determine x_i, a randomly sampled secret key x_i' will likely not satisfy these relations.

On the other hand, Round 3 ensures property unique-aggr-nonce, however, unique-aggr-nonce is needed for the simulation strategy. Let us elaborate. Notice that the commitment cmt_i determines $\widetilde{R}_i$. Therefore, $\widetilde{\mathsf{ctnt}} = (\mathsf{cmt}_i)_{i \in \mathsf{SS}}$ determines the aggregated nonce R. The signature check in Round 4 ensures therefore that all honest signers agree upon R. In the adaptive setting, the signing of individual commitment cmt_i is not sufficient as diverging views $\widetilde{\mathsf{ctnt}}$ might become consistent after corruptions. [6] The same issue appears if the nonce $\widetilde{R}_i$ is sent together with the signature $\sigma_{\mathsf{S},i}$. The view $\widetilde{\mathsf{ctnt}}$ can become consistent amongst honest signers through corruptions, however, the reduction is stuck with its choice $\widetilde{R}_i$ while the final aggregated nonce R is not yet determined. Therefore, any naive modification to reduce rounds means that the reduction cannot reliably embed the simulated challenge c into H_c at the correct location $(\mathsf{vk}, \mathsf{M}, R)$.

Moving forward, our goal will be nonetheless to remove the need for Round 1 and Round 3. As eluded to above, we will do so by using an appropriate commitment—paired with an SLE NIZK—and an opening strategy that enforces both property hidden-nonces and unique-aggr-nonce at the same time.

Removing the first round. Our approach to avoid the first round is to commit to an unmasked R_i, which eliminates the need for establishing sid in Round 1. Naively, this would clash with property hidden-nonces as when signer i opens the hash commitment cmt_i to R_i, clearly R_i is revealed before corruption. To address this, we instead employ a homomorphic commitment scheme CM over $\mathbb{G}$. That is, signer i commits to R_i directly in the Round 1 via $\mathbf{C}_i = \mathsf{CM.Commit}(\mathsf{ck}, R_i; s_i)$ for commitment key ck random opening s_i. Note that the commitment key ck is sampled during setup. We then design the signing protocol such that only

$$\mathbf{C} = \prod_{i \in \mathsf{SS}} \mathbf{C}_i = \mathsf{CM.Commit}\left(\mathsf{ck}, \prod_{i \in \mathsf{SS}} R_i; \sum_{i \in \mathsf{SS}} s_i\right)$$

is opened. Note that we assume that the commitment is also homomorphic over its randomness space, using additive notation. This ensures property hidden-nonces is preserved without an additional round. Before we discuss our opening procedure in more detail, let us discuss some properties that we expect from CM.

The common choice for the commitment within the commit-and-open protocol is a random oracle. This is not merely for efficiency, but several important properties are required:

- **Equivocation.** For some fixed commitment cmt_i, the reduction can delay programming the random oracle $\mathsf{H}_{\mathsf{com}}$ and open $\mathsf{cmt}_i = \mathsf{H}_{\mathsf{com}}(m)$ to any value m, assuming m has high min-entropy.
- **Extraction.** Given a commitment cmt_i generated by adversary $\mathcal{A}$, it is possible to extract an opening m from the query table of $\mathsf{H}_{\mathsf{com}}$. If no such m exists, then $\mathcal{A}$ cannot find an opening later on.

These properties are important as the reduction must be able to embed a simulated Schnorr nonce into the honest commitments after the malicious nonces R_i are extracted from the commitments. Therefore, both properties are required *simultaneously*, however, this is a property that seems reliant on the fact that cmt_i is sampled via a random oracle. For structured commitments the properties of extraction and equivocation are at odds: we can either setup the commitment key ck to be extractable or equivocable, but not both at the same time.

[6] With *consistent*, we refer to the fact that all honest signers share the same view $\widetilde{\mathsf{ctnt}}$ within the signing session. Observe that for instance, if signer 1 and signer 2 are honest with views $\widetilde{\mathsf{ctnt}} = (\mathsf{cmt}_1, \mathsf{cmt}_2)$ and $\widetilde{\mathsf{ctnt}}' = (\mathsf{cmt}_1', \mathsf{cmt}_2)$, then these views determine (potentially) distinct R and R', respectively. If signer 1 is corrupted, then the view of the remaining honest signer 2 is consistent, therefore the session will aggregate to R', and vice versa, if signer 2 is corrupted, the session will aggregate to R. Therefore, consistency must be ensured before cmt_i is opened.

Our commitment scheme. To resolve the tension between equivocation and extraction, we use the commitment scheme BCJ from [BCJ08] paired with an NIZK Π that provides simulation-extractability (SIM-EXT). Roughly, BCJ allows to commit to group elements $R \in \mathbb{G}$, and [BCJ08] shows that BCJ is binding under DL and satisfies a restricted equivocation property. That is, commitments are equivocal for messages of the form $R = G^{\alpha} X^{\beta}$, which correspond to HVZK-simulated nonces for Schnorr. For our purpose, we require a stronger notion [7], namely that in addition to equivocation for individual commitments $\mathbf{C}_i$, it is possible to equivocate the product of commitments $\mathbf{C} = \prod_{i \in \mathsf{SS}} \mathbf{C}_i$ *without* revealing any individual opening for $\mathbf{C}_i$. We name this notion restricted *product* equivocation, and we show that BCJ satisfies it, making it an excellent choice for our needs.

Furthermore, recall that we need the commitments to be extractable *at the same time*. Thus, we employ a simulation-extractable proof system Π in addition to BCJ, *i.e.*, the proofs π are straight-line extractable, even in the presence of simulated proofs. Now, signer i outputs $(\mathbf{C}_i, \pi_i)$ in Round 1, where $\mathbf{C}_i = \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; s_i)$ as before and π_i proves knowledge of $\mathbf{C}_i$'s openings (R_i, s_i) via Π. In the security proof, we can extract from malicious $\mathbf{C}_i$ while simulating proofs π_i for honest signers. Notice that if signer i is corrupted, we must provide the adversary $\mathcal{A}$ with all random coins sampled during signing, including the randomness used to generate π_i. We therefore require Π to have *explainable* simulated proofs, *i.e.*, given a witness, the random coins for π_i can be simulated. For technical reasons, we also augment the proofs with more fine-grained extraction control. We show that the randomized Fischlin transformation [Ks22] satisfies these properties. We refer to Section 5 for details.

Removing the third round. Let us describe how to open the commitments in a secure manner. Note that simply revealing the individual openings s_i reveals R_i which conflicts with property $\mathsf{hidden\text{-}nonces}$. Instead, our goal is to perform this opening step preserving this property. Looking ahead, our technique also enforces property $\mathsf{unique\text{-}aggr\text{-}nonce}$ for *free*, thereby making it possible to omit Round 3. In short, our idea is to mask the openings s_i with input $\widetilde{\mathsf{ctnt}} = (\mathbf{C}_i)_{i \in \mathsf{SS}}$, *i.e.*, signer i reveals

$$\tilde{s}_i = s_i + \tilde{\Delta}_i$$

in Round 2, where $\tilde{\Delta}_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$. As before, the masking ensures that only the aggregated opening $s = \sum_{i \in \mathsf{SS}} s_i$ is revealed within the signing session. This ensures that the openings for individual R_i remain hidden, thereby enforcing property $\mathsf{hidden\text{-}nonces}$. Note that given aggregated s and $\mathbf{C}$, the signers can recompute the committed value R in the final Round 3.

It is much more technical to argue property $\mathsf{unique\text{-}aggr\text{-}nonce}$. Roughly, we manage to argue via binding of BCJ that $\mathbf{C}$ cannot be opened unless all honest signers agreed on $\widetilde{\mathsf{ctnt}}$ and revealed their masked opening $\tilde{s}_i$ in Round 2. Therefore, the $\mathbf{C}$ verification in Round 3 also serves as an *implicit* consistency check. Looking ahead, we must also enforce that $\widetilde{\mathsf{ctnt}}$ determines a unique nonce R for Round 3, as this is needed for HVZK-based simulation. Note that for $\mathsf{H}_{\mathsf{com}}$-based commitments, this is immediate. In contrast, the commitment BCJ only achieves computational binding, therefore the adversary could in principle open the aggregated $\mathbf{C}$ to any value. To argue this, we employ the SIM-EXT simulator of Π to extract the committed R_i from π_i for malicious commitments $\mathbf{C}_i$. When the last honest signer opens $\tilde{s}_i$, the game precomputes $R = \sum_{i \in \mathsf{SS}} R_i$. We show that binding enforces that the adversary must open $\mathbf{C}$ to this R which establishes property $\mathsf{unique\text{-}aggr\text{-}nonce}$. Note that at this point, the game still signs via the full secret key x and therefore, we do not need equivocation yet.

Putting everything together. In the final step of the proof, we can switch the commitment key ck for BCJ into equivocal mode which allows to embed a simulated Schnorr signature into the signing session. Then, rewinding allows to break DL if $\mathcal{A}$ is successful. In summary, our threshold signature Rackle is as follows. The setup works as in $\mathsf{Crackle}$, except that the signing keys are omitted. In exchange, a commitment key ck is sampled and shared with all signers. In total, the verification key is $X = G^x$ and signer i holds $\mathsf{sk}_i = (x_i, \vec{\mathsf{seed}}_i)$, where x_i is a secret share of x and $\vec{\mathsf{seed}}_i$ are pairwise-shared seeds.

[7] The sketched notion is simplified for exposition.

Round 1. Signer i samples a nonce $R_i = G^{r_i}$ for $r_i \leftarrow \mathbb{Z}_p$. Then, it commits to R_i via $\mathbf{C}_i = \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; s_i)$ for random s_i and computes a proof π which certifies that it knows an opening for $\mathbf{C}_i$ via Π. It outputs $(\mathbf{C}_i, \pi_i)$.

Round 2. Upon receiving $(\mathbf{C}_j, \pi_j)_{j \in \mathsf{SS}}$, signer i verifies the proofs π_j for $j \in \mathsf{SS}$. If verification passes, it sets $\widetilde{\mathsf{ctnt}} = (\mathbf{C}_j)_{j \in \mathsf{SS}}$ and computes a zero share $\tilde{\Delta}_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$. It outputs the masked opening $\tilde{s}_i = s_i + \tilde{\Delta}_i$.

Round 3. Upon receiving $\tilde{s}_i$, signer i computes $s = \sum_{j \in \mathsf{SS}} \tilde{s}_i$ and $\mathbf{C} = \prod_{j \in \mathsf{SS}} \mathbf{C}_j$. Then, it recovers R from $(\mathbf{C}, s)$ and sets $\mathsf{ctnt} = (\mathbf{C}_j)_{j \in \mathsf{SS}} \| R$ and computes the challenge $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$. Finally, it computes a zero share $\Delta_i = \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt})$ and a response $z_i = c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i$, and outputs the masked response $\tilde{z}_i = z_i + \Delta_i$.

On proving strong(er) unforgeability. To establish the stronger unforgeability notion from [ABHR25], we observe that the proof strategy in [KRT24] also applies here. That is, we guess an H_c query $(\mathsf{vk}, \mathsf{M}, R)$ such that adversary $\mathcal{A}$ provides a forgery with this R *and* no session that aggregates to this R is completed (in a consistent manner). Then, the sessions are simulated in such a manner that H_c is never programmed on this input, meaning we can extract a DL solution for X upon rewinding.

3 Preliminary

This section provides necessary backgrounds and notations for this work.

Notations. Throughout, λ denotes the security parameter. For some random variable V with (finite) image $\mathcal{Y}$, we denote the *min-entropy* by $\mathsf{H}_{\mathsf{min}}(V) := -\log \max \Pr_{y \in \mathcal{Y}}[V = y]$. We denote the string of b zeros by 0^b. For a randomized algorithm $\mathcal{A}$, we write $y = \mathcal{A}(x; r)$ short for running $\mathcal{A}$ on input x with explicit randomness r. In protocol/algorithm descriptions, the protocol/algorithm outputs $\perp$ if a **require** statement fails; otherwise, it continues. With a slight abuse of notation, in game descriptions, **require** means restricting the class of valid adversaries to those that do not violate the specified condition.

Groups. Let $\mathsf{GenG}(1^\lambda)$ be an algorithm that outputs a description of a cyclic group $\mathbb{G}$ of prime order p with a generator G. We denote this by $(\mathbb{G}, G, p) \leftarrow \mathsf{GenG}(1^\lambda)$. We use multiplicative notation for $\mathbb{G}$. We assume all group operations are efficient. Our security games and algorithms implicitly take as input the group description $(\mathbb{G}, G, p)$.

3.1 Linear Shamir Secret Sharing

We recall the *linear Shamir secret sharing* scheme [Sha79]. Let $N < p$ be an integer such that for distinct $i, j \in [N]$, $(i - j)$ is invertible over $\mathbb{Z}_p$. Let $S \subseteq [N]$ be a set of cardinality at least T. Then, given $i \in S$, we define the Lagrange coefficient $L_{S,i}$ as

$$L_{S,i} := \prod_{j \in S \setminus \{i\}} \frac{-j}{(i - j)}.$$

The share generation algorithm $\mathsf{LSecShr}$ takes as input a secret $x \in \mathbb{Z}_p$ to be shared, a total number of parties N, and the reconstruction threshold $T \leqslant N$ and produce secret shares as follows: it first uniformly samples a degree $T - 1$ polynomial $P \in \mathbb{Z}_p[X]$ such that $P(0) = s$. Then, it generates each party i's share $x_i := P(i)$ and outputs $(x_i)_{i \in [N]}$. Given any set of shares $E = \{(i, x_i)\}_{i \in S}$, we can reconstruct the secret x by $x = \sum_{i \in S} L_{S,i} \cdot x_i$.

3.2 Non-interactive Zero Shares

We rely on a helper algorithm $\mathsf{ZeroShare}$ [DKM+24, KRT24], used to generate the *masks*. We denote a tuple of seeds $(\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in \mathsf{SS}}$ by $\vec{\mathsf{seed}}_i[\mathsf{SS}]$ for any set $\mathsf{SS} \subseteq [N]$. Let $\mathsf{H}_{\mathsf{mask}}$ be a random oracle which has variable range. The helper algorithm $\mathsf{ZeroShare}$ defined with respect to $\mathsf{H}_{\mathsf{mask}}$ as follows: Given $\vec{\mathsf{seed}}_i[\mathsf{SS}]$ and a string $x \in \{0,1\}^*$ as input, it outputs a zero share Δ_i (a.k.a. mask) such that

$$\Delta_i := \sum_{j \in \mathsf{SS} \setminus \{i\}} (\mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{i,j}, x) - \mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{j,i}, x))$$

where $\mathsf{H}_{\mathsf{mask}}$ outputs elements over $\mathbb{Z}_p^2$ and $\mathbb{Z}_p$ when the first bit of x is 0 and 1, respectively. It can be easily checked that $\sum_{i \in \mathsf{SS}} \Delta_i = 0$.

3.3 Threshold Signatures

We define R round threshold signatures. Let N be the number of total signers and T be a reconstruction threshold s.t. $T \leqslant N$. Also, let SS be a signer set such that $\mathsf{SS} \subseteq [N]$ with size T. Each signer $i \in [N]$ maintains a state st_i to retain short-lived session specific information. We also define correctness and adaptive security.

$\mathsf{Setup}(1^\lambda, N, T) \to \mathsf{tspar}$**:** The setup algorithm takes as input a security parameter 1^λ, the number N of total signers, and a reconstruction threshold $T \leqslant N$ and outputs a public parameter tspar. We assume tspar includes N and T.

$\mathsf{KeyGen}(\mathsf{tspar}) \to (\mathsf{vk}, (\mathsf{sk}_i)_{i \in [N]})$**:** The key generation algorithm takes as input a public parameter tspar and outputs a verification key vk, and secret key shares $(\mathsf{sk}_i)_{i \in [N]}$. It implicitly sets up an empty state $\mathsf{st}_i := \varnothing$ for all N signers. We assume vk includes tspar.

$\mathsf{Sign} = (\mathsf{Sign}_1, \cdots, \mathsf{Sign}_R, \mathsf{Agg})$**:** The signing algorithms for the signing protocol of R round threshold signatures consist the following $(R+1)$ algorithms.

- $\mathsf{Sign}_r(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{r-1,j})_{j \in \mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i) \to (\mathsf{pm}_{r,i}, \mathsf{st}_i)$**:** The signing algorithm for the rth round for $r \in [R]$ takes as input a verification key vk, a signer set SS, a message M, an index i of a signer, a tuple of protocol messages of the $(r-1)$th round $(\mathsf{pm}_{r-1,j})_{j \in \mathsf{SS}}$, a secret key share sk_i, and a state st_i of the signer i and outputs a protocol message $\mathsf{pm}_{r,i}$ for the second round and an updated state st_i. For $r = R$, the state st_i is set to ssid.

  Note that $(\mathsf{pm}_{0,j})_{j \in \mathsf{SS}}$ and st_i is not required as input to the first round $r = 1$, therefore we omit them as input to Sign_1. Likewise, the first round $r = 1$ can be executed before deciding SS and M in this work, and we also omit these inputs to Sign_1.
- $\mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_{r,i})_{r \in [R], i \in \mathsf{SS}}) \to \mathsf{sig}$**:** The aggregation algorithm takes as input a verification key vk, a signer set SS, a message M, and a tuple of protocol messages $(\mathsf{pm}_{r,i})_{r \in [R], i \in \mathsf{SS}}$ and outputs a signature sig.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig}) \to 1$ or 0**:** The verification algorithm takes as input a verification key vk, a message M, and a signature sig, and outputs 1 if sig is valid and 0 otherwise.

3.3.1 Correctness

Below, we define the correctness of a R round threshold signature scheme.

Definition 3.1 (Correctness). *We say that a R round threshold signature scheme TS satisfies correctness if, for all $\lambda \in \mathbb{N}$, $N, T \in \mathsf{poly}(\lambda)$ s.t. $T \leqslant N$, message M, and $\mathsf{SS} \subseteq [N]$ s.t. $|\mathsf{SS}| = T$, the following respectively hold:*

$$\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, \mathsf{M}, \mathsf{SS}) = 1\right] \geqslant 1 - \mathsf{negl}(\lambda),$$

where $\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}_3}$ are shown in Fig. 1.

$\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, \mathsf{M}, \mathsf{SS})$

1 : **for** $i \in \mathsf{SS}$ **do** $\mathsf{st}_i := \varnothing$
2 : $\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T)$
3 : $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$
4 : **for** $i \in \mathsf{SS}$ **do** $\mathsf{pm}_{0,i} := \bot$
5 : **for** $r \in [R]$ **do**
6 : **for** $i \in \mathsf{SS}$ **do**
7 : $(\mathsf{pm}_{r,i}, \mathsf{st}_i) \xleftarrow{\$} \mathsf{Sign}_r(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{r-1,j})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$
8 : $\mathsf{sig} \xleftarrow{\$} \mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_{r,i})_{r\in[R],i\in\mathsf{SS}})$
9 : **return** $\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig})$

Figure 1: Correctness game for a R round threshold signature scheme.

3.3.2 Adaptive Security.

We define the adaptive security of threshold signature schemes. In the adaptive setting, an adversary is allowed to corrupt a signer at any time via a corruption oracle $\mathcal{O}_{\mathsf{Corrupt}}$. The oracle $\mathcal{O}_{\mathsf{Corrupt}}$ receives an index i of a honest signer and returns its secret key share sk_i and the state st_i of the ith signer. In this work, we strengthen the definition along several axes:

- The corruption oracle $\mathcal{O}_{\mathsf{Corrupt}}$ outputs all random coins ρ sampled during signing instead of returning the state st_i. In prior works, the state st_i often allows to recompute the random coins ρ, however, this is not true in our scheme. We therefore capture this property explicitly. Note that the state can be recomputed given all ρ.

- Our definition of adaptive security is based on the game-based definition for selective security (*i.e.*, corruptions occur at the start of the game) provided by [ABHR25] which considers strong security guarantees based on [LNÖ25, NT24]. This notion is a much stronger security notion than previous game-based definitions of adaptive security of threshold signatures [CKM23, BLT+24, BDLR25b, KRT24, BDLR25a].

  Roughly, the adversary provides ℓ forgeries sig^*_j for $j \in [\ell]$ for message M^*. It wins if these are (i) *non-trivial*, (ii) pairwise-distinct, and (iii) verify under vk for M^*. Let us explain condition (i) in more detail. In the Rth round, the signers output a postfix session identifier ssid. The forgeries are considered trivial if there are ℓ *completed* signing sessions, where all honest users agreed on ssid and message M^*. Note that due to adaptive corruptions, we must carefully evaluate condition (i) at the end of the game.

  Finally, the definition in [ABHR25] also considers some outer context ctx. We do not consider this for simplicity, but it is straightforward to adapt the definition and our TS construction accordingly. [8]

Definition 3.2 (TS-SUF-4 Adaptive Security). *For an R round threshold signature scheme TS, the advantage of an adversary $\mathcal{A}$ (with oracle access to a random oracle H) against the adaptive security of TS is defined as*

$$\mathsf{Adv}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T) = \Pr[\mathsf{Game}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T) = 1],$$

[8] Roughly, the signing protocol also receives ctx and SSID also is indexed by ctx, *i.e.*, all honest co-signers must also agree on ctx. Looking ahead, we can modify our construction to satisfy this definition by including ctx in ssid. We omit details.

$\mathsf{Game}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T)$

$\mathsf{HS} := [N], \mathsf{CS} := \varnothing$
$\mathsf{SSet}[\cdot] := \bot; \mathsf{Msg}[\cdot] := \bot; \mathsf{C}[\cdot] := \bot$
$\mathsf{R}[\cdot] := 0; \mathsf{RM}[\cdot] := \bot; \mathsf{St}[\cdot] := \bot$
$\mathsf{ctr}_{\mathsf{sid}} = 0; \mathsf{SSID}[\cdot] = \bot$
$\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T)$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$
$((\mathsf{sig}^*_j)_{j\in[\ell]}, \mathsf{M}^*) \xleftarrow{\$} \mathcal{A}^{(\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}, \mathcal{O}_{\mathsf{Corrupt}}, \mathcal{O}_{\mathsf{NextSes}}, \mathsf{H}}(\mathsf{vk})$
return 1 **if** $\ell > \left|\mathsf{FinishedSSIDs}(\mathsf{M}^*)\right|$
$\quad \wedge \forall j \in [\ell], \mathsf{Verify}(\mathsf{vk}, \mathsf{M}^*, \mathsf{sig}^*_j) = 1$
$\quad \wedge (\mathsf{sig}^*_j)_{j\in[\ell]}$ pairwise distinct
return 0

$\mathsf{FinishedSSIDs}(\mathsf{M}^*)$

$\mathcal{S}^* := \varnothing$
for $(\mathsf{ssid}, \mathsf{SS})$ s.t. $\exists i \in \mathsf{HS}, \mathsf{SSID}[\mathsf{ssid}, \mathsf{SS}, \mathsf{M}^*, i] \neq \bot$ **do**
  $\mathsf{sHS} := \mathsf{SS} \cap \mathsf{HS}$
  if $\forall j \in \mathsf{sHS}, \mathsf{SSID}[\mathsf{ssid}, \mathsf{SS}, \mathsf{M}^*, i] = \top$ **then**
    $\mathcal{S}^* \leftarrow \mathcal{S}^* \cup \{(\mathsf{ssid}, \mathsf{SS})\}$
return $\mathcal{S}^*$

$\mathcal{O}_{\mathsf{NextSes}}()$

$\mathsf{ctr}_{\mathsf{sid}} \leftarrow \mathsf{ctr}_{\mathsf{sid}} + 1$
return $\mathsf{sid} := \mathsf{ctr}_{\mathsf{sid}}$

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

require $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T - 1]\!]$
$\mathsf{HS} := \mathsf{HS}\backslash\{i\}$; $\mathsf{CS} := \mathsf{CS} \cup \{i\}$
return $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$ // secret key and random coins for signer i

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$ // round $r = 1$

require $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$
$\rho_{1,i} \xleftarrow{\$} \mathcal{R}_{\mathsf{TS},1}$; $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow \rho_{1,i}$
$(\mathsf{pm}_{1,i}, \mathsf{st}_{1,i}) \xleftarrow{\$} \mathsf{Sign}_1(\mathsf{vk}, i, \mathsf{sk}_i; \rho_{1,i})$
$\mathsf{RM}[\mathsf{sid}, 1, i] \leftarrow \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] \leftarrow \mathsf{st}_{1,i}, \mathsf{R}[\mathsf{sid}, i] \leftarrow 1$
return $\mathsf{pm}_{1,i}$

$\mathcal{O}_{\mathsf{Sign}_r}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{r-1,j})_{j\in\mathsf{SS}})$ // round $r > 1$

require $[\![\mathsf{SS} \subseteq [N]]\!] \wedge [\![i \in \mathsf{HS} \cap \mathsf{SS}]\!]$
require $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = r - 1$
require $\mathsf{RM}[\mathsf{sid}, r-1, i] = \mathsf{pm}_{r-1,i}$
if $r = 2$ **then** $\mathsf{SSet}[\mathsf{sid}, i] \leftarrow \mathsf{SS}$; $\mathsf{Msg}[\mathsf{sid}, i] \leftarrow \mathsf{M}$
if $r > 2$ **then require** $\mathsf{SSet}[\mathsf{sid}, i] = \mathsf{SS} \wedge \mathsf{Msg}[\mathsf{sid}, i] = \mathsf{M}$
$\mathsf{st}_{r-1,i} \leftarrow \mathsf{St}[\mathsf{sid}, i]$
$\rho_{r,i} \xleftarrow{\$} \mathcal{R}_{\mathsf{TS},r}$; $\mathsf{C}[\mathsf{sid}, r, i] \leftarrow \rho_{r,i}$
$(\mathsf{pm}_{r,i}, \mathsf{st}_{r,i}) \xleftarrow{\$} \mathsf{Sign}_r(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{r-1,j})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_{r-1,i}; \rho_{r,i})$
$\mathsf{RM}[\mathsf{sid}, r, i] \leftarrow \mathsf{pm}_{r,i}; \mathsf{St}[\mathsf{sid}, i] \leftarrow \mathsf{st}_{r,i}; \mathsf{R}[\mathsf{sid}, i] \leftarrow r$
if $r = R$ **then** // final round
  $\mathsf{ssid} := \mathsf{st}_{r,i}$
  // signer i signed M with ssid and co-signers SS
  $\mathsf{SSID}[\mathsf{ssid}, \mathsf{SS}, \mathsf{M}, i] \leftarrow \top$
return $\mathsf{pm}_{r,i}$

Figure 2: Adaptive security game for a R round threshold signature scheme, where H denotes the random oracle. In the above, the oracles return $\bot$ to $\mathcal{A}$ when Sign_r outputs $\bot$ for $r \in [R]$ (i.e., fail to output a protocol message or a partial signature). We denote the randomness space for Sign_r, *i.e.*, for the r-th round of the signing protocol, by $\mathcal{R}_{\mathsf{TS},r}$.

where $\mathsf{Game}^{\mathsf{ts\text{-}adp\text{-}uf}}$ *is described in Fig. 2. We say that* TS *is adaptive secure in the random oracle model if, for all* $\lambda \in \mathbb{N}$, $N, T \in \mathsf{poly}(\lambda)$ *s.t.* $T \leqslant N$ *and PPT adversary* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T) \leqslant \mathsf{negl}(\lambda)$ *holds.*

In the following, we refer to threshold signature security to TS-SUF-4 unless specified otherwise.

3.4 Hardness Assumptions

We recall the definitions of the discrete logarithm (DL) problem and the self-target DL (SelfTargetDL) problem [KMP16, BD21, KRT24]. The SelfTargetDL is an interactive variant of the DL problem, introduced by [KMP16] and renamed by [KRT24]. Bellare and Dai showed that this problem is hard under the DL assumption [BD21].

Definition 3.3 (The DL Problem). *Let* $(\mathbb{G}, p, G) \leftarrow \mathsf{GenG}(1^\lambda)$. *The advantage of an adversary* $\mathcal{A}$ *against*

the DL *problem, is defined as:*

$$\mathsf{Adv}^{\mathsf{DL}}_{\mathcal{A}}(\lambda) = \Pr\left[x \xleftarrow{\$} \mathbb{Z}_p, X := G^x, x' \xleftarrow{\$} \mathcal{A}(X) : X = G^{x'}\right].$$

The DL *assumption states that any efficient adversary* $\mathcal{A}$ *has no more than negligible advantage. Also, we say that the* DL *problem for* GenG *is* $(\varepsilon_{dl}, T_{dl})$*-hard if there is no PPT* $\mathcal{A}$ *such that* $\mathsf{Adv}^{\mathsf{DL}}_{\mathcal{A}}(\lambda) \leqslant \varepsilon_{dl}$ *and the running time is at most* T_{dl}*.*

Definition 3.4 (The SelfTargetDL problem). *Let* $(\mathbb{G}, p, G) \leftarrow \mathsf{GenG}(1^\lambda)$*. Let* $\mathsf{H} : \mathbb{G}^2 \times \{0,1\}^* \to \mathbb{Z}_p$ *be a cryptographic function modeled as a random oracle. The advantage of an adversary* $\mathcal{A}$ *against the* Self Target DL *problem, noted* SelfTargetDL*, is defined as:*

$$\mathsf{Adv}^{\mathsf{SelfTargetDL}}_{\mathcal{A}}(\lambda) = \Pr\left[\begin{array}{c} x \xleftarrow{\$} \mathbb{Z}_p \\ X := G^x, \end{array} \begin{array}{c} (\mathsf{M}, c, z) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(X) : \\ \end{array} \begin{array}{c} (c, z) \in \mathbb{Z}_p^2 \\ \wedge\ \mathsf{H}(X, G^z \cdot X^{-c}, \mathsf{M}) = c \end{array}\right].$$

The SelfTargetDL *assumption states that any efficient adversary* $\mathcal{A}$ *has no more than negligible advantage.*

Lemma 3.5 (Hardness of SelfTargetDL [BD21]). *Let* $(\mathbb{G}, p, G) \leftarrow \mathsf{GenG}(1^\lambda)$*. Let* $\mathsf{H} : \mathbb{G}^2 \times \{0,1\}^* \to \mathbb{Z}_p$ *be a cryptographic function modeled as a random oracle. For any adversary* $\mathcal{A}$ *against the* SelfTargetDL *problem making at most* Q_{H} *queries to* H*, there exists an adversary* $\mathcal{B}$ *against the* DL *problem such that*

$$\mathsf{Adv}^{\mathsf{SelfTargetDL}}_{\mathcal{A}}(\lambda) \leqslant \sqrt{Q_{\mathsf{H}} \cdot \mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}}(\lambda)} + \frac{Q_{\mathsf{H}}}{p}$$

where $\mathsf{Time}(\mathcal{B}) \approx 2 \cdot \mathsf{Time}(\mathcal{A})$*.*

3.5 Commitments

A commitment scheme C with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ are PPT algorithms (C.CKSetup, C.Commit) such that

$\mathsf{C.CKSetup}(1^\lambda) \to \mathsf{ck}$**:** Given security parameter 1^λ, outputs commitment key ck.

$\mathsf{C.Commit}(\mathsf{ck}, m; r) \to c$**:** Given the commitment key ck, message $m \in \mathcal{M}$ and explicit randomness $r \in \mathcal{R}$, it outputs a commitment c to m.

We sometimes omit ck as parameter if it is clear by context. For simplicity, we do not define an explicit verification algorithms; instead we verify the whether a commitment c opens to message m with randomness r via $c = \mathsf{C.Commit}(\mathsf{ck}, m; r)$. Below, we recall statistical hiding and computational binding notions of commitment schemes.

Definition 3.6 (Perfectly Hiding). *A commitment scheme* C *is perfectly hiding if for any* $\mathsf{ck} \leftarrow \mathsf{C.CKSetup}(1^\lambda)$ *and messages* $m_0, m_1 \in \mathcal{M}$*, the distributions of* c_0 *and* c_1 *are identical, where* $c_b = \mathsf{C.Commit}(\mathsf{ck}, m_b; r_b)$ *for* $r_b \leftarrow \mathcal{R}$*.*

Definition 3.7 (Binding). *A commitment scheme* C *is* binding *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathcal{A},\mathsf{C}}(\lambda) := \Pr\left[\begin{array}{c} \mathsf{ck} \leftarrow \mathsf{CKSetup}(1^\lambda), \\ (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\mathsf{ck}) \end{array} : \ m_0 \neq m_1 \wedge c_0 = c_1 \right] = \mathsf{negl}(\lambda),$$

where $c_i = \mathsf{C.Commit}(\mathsf{ck}, m_i; r_i)$ *for* $i \in \{0,1\}$*.*

$\mathsf{BCJ.CKSetup}(1^\lambda)$	$\mathsf{BCJ.Commit}(\mathsf{ck}, M)$
1 : $x_1, x_2 \leftarrow \mathbb{Z}_p$ s.t. $x_1 \neq x_2$	**require** $M \in \mathbb{G}$
2 : $H := G^{x_1}$; $Y \leftarrow \mathbb{G}\backslash\{0\}$; $Z := Y^{x_2}$	$(s,t) \leftarrow \mathbb{Z}_p^2$
// specifies $\mathcal{M} = \mathbb{G}, \mathcal{R} = \mathbb{Z}_p^2$	$\mathbf{C} = (C_0, C_1) := (G^s H^t, Y^s Z^t M)$
3 : **return** $\mathsf{ck} := (G, H, Y, Z)$	**return** $\mathbf{C}$

Figure 3: The commitment scheme BCJ from [BCJ08] for group elements. Above, we assume that Commit parses $(G, H, Y, Z) \leftarrow \mathsf{ck}$.

3.5.1 Commitments with Restricted Equivocability.

We recall the commitment scheme BCJ by [BCJ08] in Fig. 3. BCJ satisfies binding, perfect hiding and is homomorphic.

Lemma 3.8 (Binding [BCJ08]). *The scheme* BCJ *satisfies binding under the DL assumption. Specifically, if there exists an adversary $\mathcal{A}$ breaking the binding property, there is an adversary $\mathcal{B}$ solving the DL problem such that* $\mathsf{Adv}^{\mathsf{bind}}_{\mathcal{A},\mathsf{BCJ}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}}(\lambda)$*, where* $\mathsf{Time}(\mathcal{B}) \approx \mathsf{Time}(\mathcal{A})$.

Remark 3.9 (Homomorphism). The scheme BCJ is (linearly) homomorphic, *i.e.*, if for all $\mathsf{ck} \leftarrow \mathsf{BCJ.CKSetup}(1^\lambda)$, $M_0, M_1 \in \mathbb{G}$, $\mathbf{s}_0, \mathbf{s}_1 \in \mathbb{Z}_p^2$, we have for $\mathbf{C}_b = \mathsf{BCJ.Commit}(\mathsf{ck}, M_b; \mathbf{s}_b)$ that $\mathbf{C}_0 \cdot \mathbf{C}_1 = \mathsf{BCJ.Commit}(\mathsf{ck}, M_0 \cdot M_1; \mathbf{s}_0 + \mathbf{s}_1)$.

It also satisfies two additional properties, restricted equivocation and message recoverability, recalled below. Note that we introduce the restricted equivocability notion for context, but we will employ a stronger notion (cf. Section 4).

Lemma 3.10 (Restricted Equivocation [BCJ08]). *There exists PPT algorithms* EqvSetup, SimCom *and* EqvCom *such that for every $X \in \mathbb{G}\backslash\{1\}$ the following properties hold.*

1. *The statistical distance between the distributions of* ck *and* $\tilde{\mathsf{ck}}$ *is at most* $2/p$*, where* $\mathsf{ck} \leftarrow \mathsf{BCJ.CKSetup}(1^\lambda)$ *and* $(\tilde{\mathsf{ck}}, \mathsf{td}) \leftarrow \mathsf{EqvSetup}(1^\lambda, X)$.
2. *For every* $(\tilde{\mathsf{ck}}, \mathsf{td}) \leftarrow \mathsf{EqvSetup}(1^\lambda, X)$ *and* $\beta \in \mathbb{Z}_p$*, the distributions* $(\mathbf{C}, (s,t), \alpha)$ *and* $(\tilde{\mathbf{C}}, (\tilde{s}, \tilde{t}), \tilde{\alpha})$ *are distributed identically, where* $\alpha \leftarrow \mathbb{Z}_p, (s,t) \leftarrow \mathbb{Z}_p^2, M = G^\alpha X^\beta$ *and* $\mathbf{C} = \mathsf{BCJ.Commit}(\tilde{\mathsf{ck}}, M; (s,t))$*, and* $(\tilde{\mathbf{C}}, \mathsf{st}) \leftarrow \mathsf{SimCom}(\tilde{\mathsf{ck}}, \mathsf{td})$ *and* $(\tilde{s}, \tilde{t}, \tilde{\alpha}) \leftarrow \mathsf{EqvCom}(\tilde{\mathsf{ck}}, \mathsf{st}, \beta)$.

Lemma 3.11 (Message Recoverability). *There exists a function* RcvMsg *s.t. for any* $\mathsf{ck} \leftarrow \mathsf{BCJ.CKSetup}(1^\lambda)$ *and* $(M, (s,t)) \in \mathcal{M} \times \mathcal{R}$ *and* $\mathbf{C} = \mathsf{BCJ.Commit}(\mathsf{ck}, M; (s,t))$*, we have that* $M = M'$ *for* $M' = \mathsf{RcvMsg}(\mathsf{ck}, \mathbf{C}, (s,t))$.

Proof. Let RcvMsg output $M = C_1 \cdot Y^{-s} Z^{-t}$ on input $\mathsf{ck} = (G, H, Y, Z), \mathbf{C} = (C_0, C_1)$ and (s,t). It is easy to see that RcvMsg satisfies the above property. □

3.6 Σ-protocols

Let $\mathcal{R}$ be an NP-relation with statements $\mathbb{x}$ and witnesses $\mathbb{w}$. A Σ-protocol for an NP-relation $\mathcal{R}$ with challenge space $\mathcal{C}$ is a tuple of PPT algorithms $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ such that

$\mathsf{Init}(\mathbb{x}, \mathbb{w}) \to (a, \mathsf{st})$**:** Given a statement $\mathbb{x}$ and a witness $\mathbb{w}$ with $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, outputs a first flow message (*i.e.*, commitment) a and a state st,

$\mathsf{Resp}(\mathsf{st}, c) \to z$**:** Given a state st and a challenge $c \in \mathcal{C}$, outputs a third flow message (*i.e.*, response) z,

$\mathsf{Verify}(\mathbb{x}, a, c, z) \to 1$ or 0**:** given a statement $\mathbb{x} \in \mathcal{L}_\mathcal{R}$, a commitment a, a challenge $c \in \mathcal{C}$, and a response z, outputs a bit $b \in \{0,1\}$.

We call the tuple (a, c, z) the *transcript* and say that they are *valid for* $\mathbb{x}$ if $\mathsf{Verify}(\mathbb{x}, a, c, z)$ outputs 1. We may omit $\mathbb{x}$ if clear by context. Below, we recall standard properties for Σ-protocols (taken verbatim from [KRW24]).

Definition 3.12 (Correctness). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be an Σ-protocol for $\mathcal{R}$. We say Π_Σ is* correct*, if for all $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, $(a, \mathsf{st}) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w})$, $c \in \mathcal{C}$, and $z \leftarrow \mathsf{Resp}(\mathsf{st}, c)$, it holds that $\mathsf{Verify}(\mathbb{x}, a, c, z) = 1$.*

Definition 3.13 (Special HVZK). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$. We say that Π_Σ is* special honest-verifier zero-knowledge *(HVZK), if there exists a PPT zero-knowledge simulator ZKSim such that for any (potentially unbounded) adversary Adv, it holds that for any $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and $c \in \mathcal{C}$ that $\mathsf{D}_{\mathsf{real}} = \mathsf{D}_{\mathsf{sim}}$ for*

$$\mathsf{D}_{\mathsf{real}} := \{(a, c, z) \mid (a, \mathsf{st}) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w}), z \leftarrow \mathsf{Resp}(\mathsf{st}, c)\},$$
$$\mathsf{D}_{\mathsf{sim}} := \{(a, c, z) \mid (a, z) \leftarrow \mathsf{ZKSim}(\mathbb{x}, c)\}.$$

Definition 3.14 (Special Soundness). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$. We say that Π_Σ is* (2-)special sound*, if there exists a* deterministic *PT extractor Ext such that given two valid transcripts $\{(a, c_b, z_b)\}_{b\in[2]}$ for statement $\mathbb{x}$ with $c_0 \neq c_1$, along with x, outputs a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$.*

Definition 3.15 (High Min-Entropy). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$. We say that Π_Σ has* high min-entropy *if for all $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, it holds for $(a, \mathsf{st}) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w})$ that*

$$2^{-\mathsf{H}_{\mathsf{min}}(a)} = \mathsf{negl}(\lambda).$$

We denote $\mathsf{H}_{\mathsf{min}}(\Pi_\Sigma) := \min_{\mathbb{x}\in\mathcal{L}_\mathcal{R}} \mathsf{H}_{\mathsf{min}}(a)$.

Definition 3.16 ((Relaxed) Strong (2-)Special Soundness). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$. We say that Π_Σ is* strong (2-)special sound *for NP-relation $\tilde{R} \supseteq \mathcal{R}$ if there exists a deterministic PT machine extractor ZKExt such that given as input two accepting transcripts $\mathsf{trans} = (a, c, z)$ and $\mathsf{trans}' = (a, c', z')$ for statement $\mathbb{x}$ with $\mathsf{trans} \neq \mathsf{trans}'$, the extractor $\mathbb{w} \leftarrow \mathsf{ZKExt}(\mathsf{trans}, \mathsf{trans}')$ outputs a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \tilde{R}$.*

4 Stronger Equivocation for BCJ Commitments

We denote by BCJ the commitment scheme defined in Section 3.5.1. Looking ahead, in our signature scheme, we will need to simulate commitments of the signing parties on $R_j \leftarrow G^{\alpha_j}$ to later equivocate their product to be consistent with a Fiat-Shamir response $z := \beta \cdot x + \sum_{\mathbb{t}\in I} \alpha_\mathbb{t}$, with only the knowledge of $X = g^x$. We leverage the same equivocation setup for BCJ as in [BCJ08] (recalled in Fig. 4), which builds on introducing a trapdoor in Z so that it is a power of X and allows to compensate for $\beta \cdot x$ when equivocating, without actually knowing x. In addition, as parties can get corrupted, we will need to equivocate individual commitments and equivocate a consistent opening $(\alpha_j, \mathbf{s}_j)$. The core observation allowing to equivocate both the product of commitments and individual commitments is that for each product of commitments to equivocate, there will always be at least one individual commitment that is not opened individually, thus ensuring that the opening of the sum is randomized and independent of other commitments.

We introduce our stronger equivocation property below, named *restricted product equivocation*, capturing the possibility to equivocate individual and product of commitments, and show that BCJ satisfies this property.

Theorem 4.1 (Restricted Product Equivocation for BCJ). *Denote by $\mathsf{EqvSetup}$ the algorithm from Theorem 3.10. There exists a stateful [9] PPT simulator CSim such that for any (unbounded) adversary $\mathcal{A}$, we have*

$$\mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{A},\mathsf{CSim},\mathcal{T}}(1^\lambda) := \left| \Pr[\mathsf{Game}^{\mathsf{rst\text{-}eqv\text{-}real}}_{\mathcal{A},\mathcal{T}}(1^\lambda) = 1] - \Pr[\mathsf{Game}^{\mathsf{rst\text{-}eqv\text{-}ideal}}_{\mathcal{A},\mathsf{CSim},\mathcal{T}}(1^\lambda) = 1] \right| = 0,$$

[9] We leave the state implicit in the description of the games.

$\mathsf{BCJ.EqvSetup}(1^\lambda, X)$

$u_1 \leftarrow \mathbb{Z}_p; \quad u_2, u_3 \leftarrow \mathbb{Z}_p^*$

$H := G^{u_1}; \quad Y := G^{u_2}; \quad Z := X^{u_3}$

// specifies $\mathcal{M} = \mathbb{G}, \mathcal{R} = \mathbb{Z}_p^2$

return $\mathsf{ck} := (G, H, Y, Z), \mathsf{td} := (u_1, u_2, u_3)$

Figure 4: The equivocation setup for BCJ from [BCJ08].

$\mathsf{Game}^{\text{rst-eqv-real}}_{\mathcal{A},\mathcal{T}}(1^\lambda)$, $\mathsf{Game}^{\text{rst-eqv-ideal}}_{\mathcal{A},\mathsf{CSim},\mathcal{T}}(1^\lambda)$

$x \xleftarrow{\$} \mathbb{Z}_p^*; \; X := G^x$

$(\mathsf{ck}, \mathsf{td}) \leftarrow \mathsf{EqvSetup}(1^\lambda, X)$

$(\mathsf{ctr}_\mathbb{t})_{\mathbb{t} \in \mathcal{T}} := 1; \; \mathsf{ctr}_{\mathsf{prod}} := 1$

$\mathcal{T}_{\mathsf{com}}[\cdot,\cdot] := \bot, \mathcal{T}_{\Pi\text{-eqv}}[\cdot] := \bot,$

$b \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{com}}, \mathcal{O}_{\Pi\text{-eqv}}, \mathcal{O}_{\mathsf{eqv}}}(\mathsf{ck}, \mathcal{T}, x)$

return b

$\mathcal{O}_{\mathsf{com}}(\mathbb{t})$

require $\mathbb{t} \in \mathcal{T}$

$\mathbf{s}_\mathbb{t} \leftarrow \mathbb{Z}_p^2; \quad \alpha_\mathbb{t} \leftarrow \mathbb{Z}_p$

$\mathbf{C}_\mathbb{t} := \mathsf{BCJ.Commit}(\mathsf{ck}, G^{\alpha_\mathbb{t}}; \mathbf{s}_\mathbb{t})$

$\mathcal{T}_{\mathsf{com}}[\mathbb{t}, \mathsf{ctr}_\mathbb{t}] := (\mathbf{s}_\mathbb{t}, \alpha_\mathbb{t})$

$\mathbf{C}_\mathbb{t} \leftarrow \mathsf{CSim}(\mathsf{cmt}, \mathsf{td}, \mathbb{t}, \mathsf{ctr}_\mathbb{t})$

$\mathcal{T}_{\mathsf{com}}[\mathbb{t}, \mathsf{ctr}_\mathbb{t}] := \top$

$\mathsf{ctr}_\mathbb{t} \leftarrow \mathsf{ctr}_\mathbb{t} + 1$

return $\mathbf{C}_\mathbb{t}$

$\mathcal{O}_{\mathsf{eqv}}(\mathbb{t}, i)$

require $\mathcal{T}_{\mathsf{com}}[\mathbb{t}, i] \neq \bot$

if $\exists j \in [\mathsf{ctr}_{\mathsf{prod}}], \; \mathcal{T}_{\Pi\text{-eqv}}[j] = \{(\mathbb{t}, i)\}$ **then**

// Last commitment cannot be opened individually

return 0

parse $(\mathbf{s}_\mathbb{t}, \alpha_\mathbb{t}) \leftarrow \mathcal{T}_{\mathsf{com}}[\mathbb{t}, i]$

$(\mathbf{s}_\mathbb{t}, \alpha_\mathbb{t}) \leftarrow \mathsf{CSim}(\mathsf{eqv}, \mathsf{td}, \mathbb{t}, i)$

$\mathcal{T}_{\mathsf{com}}[\mathbb{t}, i] \leftarrow \bot$ // Cannot equivocate twice

if $\exists j \in [\mathsf{ctr}_{\mathsf{prod}}], \; (\mathbb{t}, i) \in \mathcal{T}_{\Pi\text{-eqv}}[j]$ **then**

$\mathcal{T}_{\Pi\text{-eqv}}[j] \leftarrow \mathcal{T}_{\Pi\text{-eqv}}[j] \backslash \{(\mathbb{t}, i)\}$

return $(\mathbf{s}_i, \alpha_i)$

$\mathcal{O}_{\mathsf{prod\text{-}eqv}}(\beta, I, (\mathbb{t}, i_\mathbb{t})_{\mathbb{t} \in I})$

require $(I \subseteq \mathcal{T}) \wedge (\beta \in \mathbb{Z}_p)$

// $\mathbf{C}_\mathbb{t}$ generated via $\mathcal{O}_{\mathsf{com}}$ and not equivocated

require $\mathcal{T}_{\mathsf{com}}[\mathbb{t}, i_\mathbb{t}] \neq \bot$

// $\mathbf{C}_\mathbb{t}$ has never been equivocated within prod of commitments

require $\forall (\mathbb{t}, j) \in I \times [\mathsf{ctr}_{\mathsf{prod}}], \; (\mathbb{t}, i_\mathbb{t}) \notin \mathcal{T}_{\Pi\text{-eqv}}[j]$

$\mathcal{T}_{\Pi\text{-eqv}}[\mathsf{ctr}_{\mathsf{prod}}] \leftarrow \{\mathbb{t}, i_\mathbb{t}\}_{\mathbb{t} \in I}$

for $\mathbb{t} \in I$ **do parse** $(\mathbf{s}_\mathbb{t}, \alpha_\mathbb{t}) \leftarrow \mathcal{T}_{\mathsf{com}}[\mathbb{t}, i_\mathbb{t}]$

$\mathbf{s} := \sum_{\mathbb{t} \in I} \mathbf{s}_\mathbb{t}; \; z := \beta \cdot x + \sum_{\mathbb{t} \in I} \alpha_\mathbb{t}$

$(\mathbf{s}, z) \leftarrow \mathsf{CSim}(\mathsf{prod\text{-}eqv}, \mathsf{td}, \beta, I, (\mathbb{t}, i_\mathbb{t})_{\mathbb{t} \in I})$

$\mathsf{ctr}_{\mathsf{prod}} \leftarrow \mathsf{ctr}_{\mathsf{prod}} + 1$

return $(\mathbf{s}, z)$

Figure 5: The real and ideal games for our restricted product equivocation notion. The real game is obtained by ignoring gray-highlighted code and the ideal game is obtained by ignoring boxed code. Note that equivocation does not return the committed message as it can be recomputed due to Theorem 3.11.

where $\mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{A},\mathcal{T}}$ *and* $\mathsf{Game}^{\text{rst-eqv-ideal}}_{\mathcal{A},\mathsf{CSim},\mathcal{T}}$ *are defined in Fig. 5.*

We provide a proof sketch below; the full proof can be found in Section A.

Proof sketch. The proof proceeds by showing that the adversary's view in the real and ideal games is identically distributed, which implies that no adversary can distinguish them. This is done via an inductive

argument on the number of oracle queries made by the adversary. The core of the proof lies in the construction of the simulator, CSim, which must be able to perfectly replicate the outputs of all oracles.

For the commitment oracle $\mathcal{O}_{\mathsf{com}}$, the simulator generates a commitment with uniform exponents in G and X, omitting the committed message. In the real game, a commitment to G^{α} is of the form $(G^{s+tu_1}, G^{su_2+\alpha}X^{tu_3})$ for uniform (s,t,α). In the ideal game, the simulator creates a commitment as (G^r, G^aX^b) for uniform (r,a,b). We show that a bijective linear map exists between the exponents (s,t,α) and (r,a,b). Since a bijection over a finite field preserves uniformity, the distributions of the commitments are identical in both games.

It then remains to simulate the equivocation oracles consistently and without knowledge of x. To do so, the simulator leverages the trapdoor $\mathsf{td} = (u_1,u_2,u_3)$ to compute valid openings for any commitment it has generated, inverting the relationship between commitments and equivocation outputs. When an equivocation is requested, CSim sets up and solves a system of linear equations so as to match the exponents in G and X relations with the real world, which crucially avoids the appearance of the x variable. Specifically, for individual equivocations, we define the simulator to recompute s,t,α so that $\mathbf{C} := (G^r, G^aX^b) = (G^{s+tu_1}, G^{su_2+\alpha}X^{tu_3})$. For product equivocation, we enforce $\prod_{\mathbb{t}\in I}(G^{r_{\mathbb{t}}}, G^{a_{\mathbb{t}}}X^{b_{\mathbb{t}}}) = (G^{s+tu_1}, G^{su_2+z}X^{tu_3-\beta})$. The trapdoor ensures this system always has a unique solution, allowing the simulator to produce a valid opening.

A crucial detail arises in product equivocations to prove that distributions are identical. The game rules guarantee that at least one commitment in any product equivocation is never opened individually. This is key as the randomness from this unopened commitment ensures that the aggregated value z remains uniformly distributed from the adversary's perspective, making the simulation perfect. For a product equivocation with T commitments, we formalize this argument by defining a bijection between their internal randomness in the real and ideal games with at most $T-1$ equations covering the individual equivocations output, and one equivocation for the product equivocation output, defining a full-rank linear system. We introduce an induction step showing that for any query, the conditional distribution of the output is identical in both games, thus the adversary's entire view remains identically distributed. □

5 Simulation-Extractability of Fischlin's Transformation

In this section, we show that the randomized Fischlin transformation [Fis05, Ks22, KRW24] is simulation-extractable [BFW15]. That is, it is possible to *simultaneously* simulate proofs and extract a valid witness from non-simulated proofs. In fact, we show a stronger notion where simulated proofs π are *explainable* [HK22] (*i.e.*, well-distributed randomness ρ such that $\pi = \mathsf{Prove}^{\mathsf{H}}(\mathbb{x},\mathbb{w};\rho)$ can be simulated given a witness for the relation). We also introduce labels that allow more fine-grained control over which proofs are extractable. Looking ahead, these properties are important in our threshold signature (cf. Section 6).

5.1 Labeled Non-Interactive Zero-Knowledge Proofs

Let $\mathcal{R}$ be an NP-relation and $\mathcal{L}_{\mathcal{R}} = \{\mathbb{x} : (\mathbb{x},\mathbb{w}) \in \mathcal{R}\}$ be the language induced by $\mathcal{R}$. We recall the notion of NIZKs in the random oracle model below, except that both prove and verification algorithms also obtain a label $\mathbb{t}$ as additional input. This allows for more fine-grained extraction (*i.e.*, even simulated proofs can be extracted if the label is fresh).

Definition 5.1 (Labeled NIZK). *A labeled non-interactive zero-knowledge proof (NIZK) in the random oracle model for relation $\mathcal{R}$ with label space $\mathcal{T}$ is a tuple $\Pi = (\mathsf{Prove},\mathsf{Verify})$ with access to oracle H such that*

$\mathsf{Prove}^{\mathsf{H}}(\mathbb{x},\mathbb{w},\mathbb{t})$**:** *Given statement $\mathbb{x}$, witness $\mathbb{w}$ and label $\mathbb{t} \in \mathcal{T}$ such that $(\mathbb{x},\mathbb{w}) \in \mathcal{R}$, outputs a proof π.*

$\mathsf{Verify}^{\mathsf{H}}(\mathbb{x},\pi,\mathbb{t})$**:** *Given statement $\mathbb{x}$, proof π and label $\mathbb{t} \in \mathcal{T}$, outputs a bit b.*

We denote the randomness space of Prove *by $\mathcal{R}_{\Pi,\$}$.*

Definition 5.2 (Correctness). *A labeled NIZK* $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ *for a relation* $\mathcal{R}$ *with label space* $\mathcal{T}$ *has* correctness error κ *if for all* $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ *and labels* $\mathbb{t} \in \mathcal{T}$*, it holds that*

$$\Pr\left[\ \mathsf{Verify}^{\mathsf{H}}(\mathbb{x}, \pi, \mathbb{t}) = 1 \quad : \quad \pi \leftarrow \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}, \mathbb{t})\ \right] \geqslant 1 - \kappa(\lambda),$$

where the probability is over the choice of H *and the randomness of* Prove *and* Verify*. We call* Π correct *if* $\kappa(\lambda) = \mathsf{negl}(\lambda)$*.*

We define simulation-extractability for labeled NIZKs below. That is, the adversary $\mathcal{A}$ gets access to an $\mathcal{O}_{\mathsf{prove}}$ oracle which given $(\mathbb{x}, \mathbb{w}, \mathbb{t})$ outputs an honestly-generated proof π in the real experiment. In the ideal experiment, $\mathcal{O}_{\mathsf{prove}}$ outputs a simulated proof π. Also, $\mathcal{A}$ obtains access to an $\mathcal{O}_{\mathsf{ext}}$ oracle which, in the real world, outputs a bit b indicating whether the provided proof π for statement $\mathbb{x}$ verifies with respect to label $\mathbb{t}$ or not. In the ideal experiment, the simulator is tasked to extract a witness given an accepting proof. If extraction fails, then the $\mathcal{O}_{\mathsf{ext}}$ oracle outputs 0. Observe that in the real experiment, the $\mathcal{O}_{\mathsf{ext}}$ oracle always outputs the bit b, therefore if extraction fails but the proof verifies (*i.e.*, $b = 1$), the ideal and real experiment are trivial to distinguish as 0 is output only in the ideal experiment. Finally, our definition differs from prior definitions as $\mathcal{A}$ also obtains access to an $\mathcal{O}_{\mathsf{rvl}}$ oracle. This oracle outputs the randomness ρ which was employed to generate proof π in the real experiment. In the ideal experiment, the simulator is tasked to *explain* a simulated proof π, *i.e.*, given a valid witness $\mathbb{w}$, the simulator must provide well-distributed random coins ρ such that $\pi = \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}; \rho)$.

Definition 5.3 (SIM-EXT). *A labeled NIZK* $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ *is simulation-extractable for knowledge relation* $\tilde{R} \supseteq \mathcal{R}$ *if there exists a stateful* [10] *PPT simulator* ZKSim *such that for every PPT adversary* $\mathcal{A}$ *we have*

$$\mathsf{Adv}^{\mathsf{SIM\text{-}EXT}}_{\Pi,\mathcal{A},\mathsf{ZKSim}}(1^\lambda) := \left| \Pr[\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}real}}_{\Pi,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}ideal}}_{\Pi,\mathcal{A},\mathsf{ZKSim}}(1^\lambda) = 1] \right| = \mathsf{negl}(\lambda),$$

where $\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}real}}_{\Pi,\mathcal{A}}$ *and* $\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}ideal}}_{\Pi,\mathcal{A},\mathsf{ZKSim}}$ *are defined in Fig. 6.*

5.2 Labeled Fischlin Transformation

We show that a labeled variant of the randomized Fischlin transform [Ks22] satisfies our strong SIM-EXT notion. The transformation takes as input a Σ-protocol and compiles it into an NIZK via a ROM-based proof of work. As we demand that proofs are explainable, we rely on a stronger HVZK notion for Σ-protocols. Roughly, we demand that there exists a simulator $\Pi_\Sigma.\mathsf{ZKExp}$ which allows to explain simulated transcripts, *i.e.*, given a witness, $\Pi_\Sigma.\mathsf{ZKExp}$ simulates the random coins employed to generate a simulated transcript. We stress that without the $\mathcal{O}_{\mathsf{rvl}}$ oracle, this stronger property is *not* required.

5.2.1 Construction

Let $\mathcal{R}$ be an NP-relation. Let $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$ with challenge space $\mathcal{C}$. Let $\mathsf{H}\colon \{0,1\}^* \to \{0,1\}^b$ be a random oracle for $b \in \mathbb{N}$. We recall the randomized Fischlin transformation from [Ks22] in its simplified form [KRW24]. However, we modify it such that the associated label $\mathbb{t}$ is also hashed within random oracle H. Looking ahead, this simple but crucial modification allows us to even extract from *simulated* proofs if the label differs.

We consider the following parameters. Let $k := \log(|\mathcal{C}|)$ denote the bit-size of the challenge space, let $b \in \mathbb{N}$ denote the bit-size of H's output space, let $r \in \mathbb{N}$ denote the number of parallel repetitions, and t an upper bound for number of loops. As in [KRW24], we set the parameters such that $r \geqslant \lceil \lambda/b \rceil$ for λ bits of security, and $t \in \Theta(\log(\lambda) b \log(r))$ and $k = \omega(b)$ for negligible correctness error and zero-knowledge. We set $2^t = \mathsf{poly}(\lambda)$ to ensure the prover runs in polynomial time. The transformation is given in Fig. 7. We write $\mathsf{RF}[\Pi_\Sigma] = (\mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$ for the obtained labeled NIZK.

[10] We leave the state implicit.

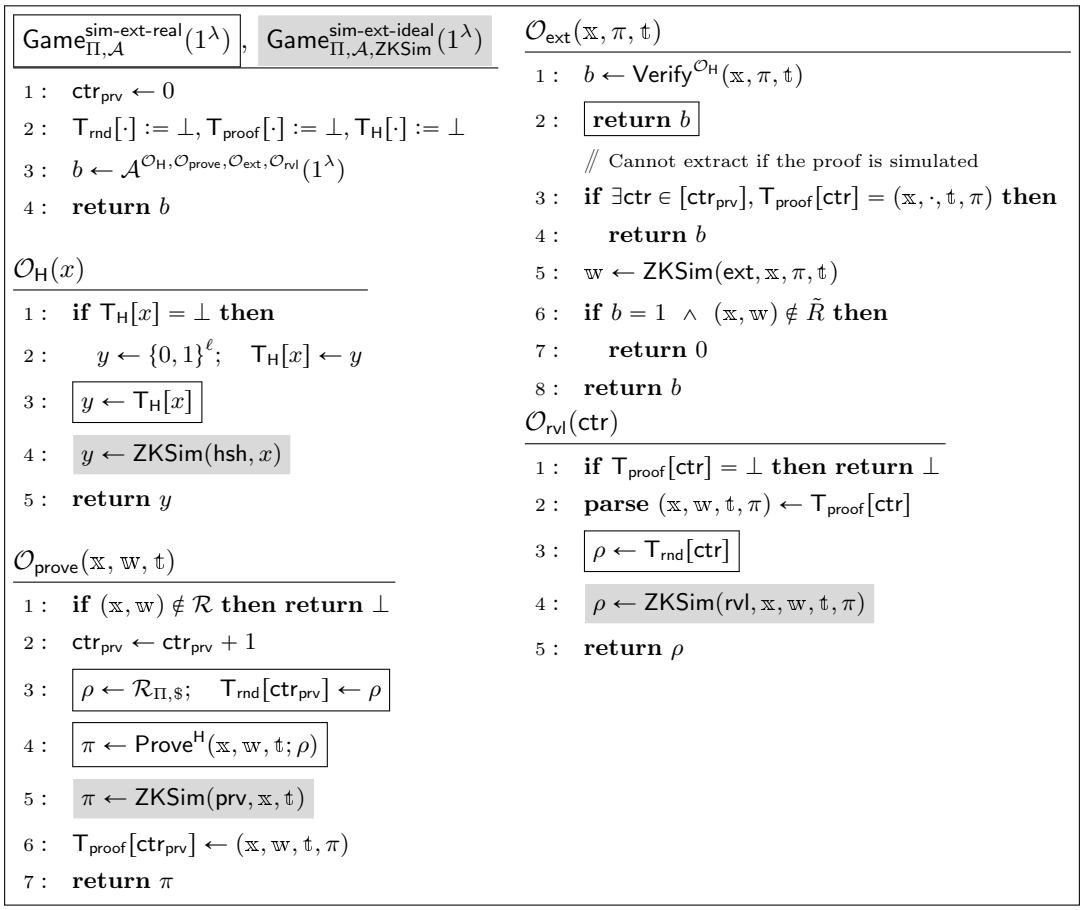

Figure 6: The real and ideal games of our *strengthened* simulation-extractability notion.

5.2.2 Security Analysis

Let us show that the randomized Fischlin transformation is correct and simulation-extractable. Correctness follows as in [Ks22]. Let us sketch simulation extractability. For this, it is helpful to briefly recap the security analysis in [Ks22, KRW24]. Below, we already adapt the argument to the label-based variant from this work.

Zero-knowledge (ZK). The simulator proceeds as follows to simulate a proof π for label $\mathbb{t}$. First, for all $i \in [r]$, it samples a random challenge $c_i \leftarrow \mathcal{C}$ and runs the special HVZK simulator $(a_i, z_i) \leftarrow \Pi_\Sigma.\mathsf{ZKSim}(\mathbb{x}, c_i)$. It sets $\mathbf{a} = (a_1, \ldots, a_r)$ and for all $i \in [r]$, it programs $\mathsf{H}(\mathbb{x}, \mathbb{t}, \mathbf{a}, i, c_i, z_i) := 0^b$. Finally, it outputs $\pi = (a_i, c_i, z_i)_{i\in[r]}$.

Observe that by HVZK, real and simulated transcripts (a_i, c_i, z_i) are indistinguishable. To argue zero-knowledge, it remains to analyze the distribution of H which has been changed due to programming. By high min-entropy, the oracle has not yet been queried on input $(\mathbb{x}, \mathbb{t}, \mathbf{a}, i, c_i, z_i)$ except with probability $Q \cdot 2^{-\mathsf{H}_{\mathsf{min}}(\Pi_\Sigma)}$. However, the output distribution of H is skewed as the number of 0^b outputs is higher than in the real game. Klooß et al. [KRW24] show that the statistical difference between a random oracle $\mathsf{H}' \colon \mathcal{C} \to \{0,1\}^b$ and its programmed variant $\mathsf{H}'[c \mapsto 0^b]$ for a random $c \leftarrow \mathcal{C}$ is at most $4 \cdot 2^{-(k-b)/2}$. Here, $\mathsf{H}'[c \mapsto 0^b]$ denotes the random oracle obtained by reprogramming random oracle H' at input c with output 0^b. This suffices to show zero-knowledge with the above considerations (employing a union bound over all iterations).

Straightline extraction (SLE). Suppose an adversary $\mathcal{A}$ succeeds to generate an accepting proof $\pi =$

$\mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}, \mathbb{t})$

1 : **for** $i \in [r]$ **do**
2 : $\quad (a_i, \mathsf{st}_i) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w})$
3 : $\mathbf{a} := (a_1, \ldots, a_r)$; $\quad \mathsf{ctr} := 0$
4 : **for** $i \in [r]$ **do**
5 : $\quad \mathcal{C}_i \leftarrow \varnothing \quad \mathsf{ctr} \leftarrow \mathsf{ctr} + 1$
6 : $\quad$ **if** $\mathsf{ctr} > 2^t$ **then return** $\perp$
$\quad$ // sample *without replacement* each iteration
7 : $\quad c_i \leftarrow \mathcal{C} \backslash \mathcal{C}_i$; $\quad \mathcal{C}_i \leftarrow \mathcal{C}_i \cup \{c_i\}$
8 : $\quad z_i \leftarrow \mathsf{Resp}(\mathsf{st}_i, c_i)$
9 : $\quad h_i := \mathsf{H}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$
10 : $\quad$ **if** $h_i \neq 0^b$ **then**
11 : $\quad\quad$ Go back to line 5 and try again
12 : **return** $\pi := (a_i, c_i, z_i)_{i \in [r]}$

$\mathsf{Verify}^{\mathsf{H}}(\mathbb{x}, \pi, \mathbb{t})$

1 : **parse** $(a_i, c_i, z_i)_{i \in [r]} \leftarrow \pi$
2 : **for** $i \in [r]$ **do**
3 : $\quad h_i := \mathsf{H}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$
4 : $\quad$ **if** $h_i \neq 0^b \vee \mathsf{Verify}(\mathbb{x}, a_i, c_i, z_i) = 0$ **then**
5 : $\quad\quad$ **return** 0
6 : **return** 1

Figure 7: The labeled NIZK $\mathsf{RF}[\Pi_\Sigma] = (\mathsf{Prove}^{\mathsf{H}}, \mathsf{Verify}^{\mathsf{H}})$ obtained by applying the randomized Fischlin transformation to Π_Σ.

$(a_i, c_i, z_i)_{i \in [r]}$ for label $\mathbb{t}$. Denote $\mathcal{A}$'s queries to H with $\mathcal{Q}$. The extractor searches for a *distinct* valid transcript (a_i, c_i', z_i') for $\mathbb{x}$ completing a_i for some $i \in [r]$ amongst $\mathcal{Q}$. If $(c_i', z_i') \neq (c_i, z_i)$, then the extractor succeeds to extract a valid witness from π by strong 2-special soundness.

Hence, the extractor fails if H was queried as $\mathsf{H}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$ with accepting transcript $(\mathbb{x}, a_i c_i, z_i)$ only *once* for every i. However, the proof is only accepting if $\mathsf{H}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i) = 0^b$ which happens with probability 2^{-b}. Therefore, the extractor fails for proof π with probability at most $(2^{-b})^r = 2^{-rb}$. Denote by Q the number of H queries. Due to a union bound over all possible $\mathbf{a}$ which $\mathcal{A}$ may have tried, it follows that extraction fails except with probability $Q \cdot 2^{-rb}$.

Let us turn towards simulation extractability of $\mathsf{RF}[\Pi_\Sigma]$. We must ensure that extraction is possible *while* simulating proofs. We simulate proofs by programming the random oracle H as described above. We can argue that this change is unnoticeable with a similar analysis.

For extraction, assume that proof $\pi = (a_i, c_i, z_i)_{i \in [r]}$ for statement $\mathbb{x}$ with label $\mathbb{t}$ is passed to the extraction oracle. Also, assume that π is verified. With the above in mind, it is easy to see that the sketched SLE extractor suffices if either $\mathbf{a}$ is fresh (*i.e.*, $\mathbf{a}$ is not from a simulated proof) or π was simulated but for a different label $\mathbb{t}'$ or statement $\mathbb{x}'$. This holds because for such proofs, the random oracle was not reprogrammed on *any* input of the form $(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$ yet, and the same probabilistic argument as above allows to conclude that the extractor finds a witness.

It remains to argue that we can also extract if $\mathbf{a}$ is reused from a simulated proof $\pi' = (a_i, c_i', z_i')_{i \in [r]}$ for same statement $\mathbb{x}$ and label $\mathbb{t}$, if $\pi \neq \pi'$. In this case, we can turn towards strong 2-special soundness immediately. As $\pi' \neq \pi$, there must be a transcript $(a_i, c_i', z_i') \neq (a_i, c_i, z_i)$. Since both are accepting, the Σ-protocol extractor yields a witness.

Finally, it remains to argue that we can explain simulated proofs π, *i.e.*, simulate well-distributed randomness ρ such that $\pi = \mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}, \mathbb{t}; \rho)$ given a witness $\mathbb{w}$. For this, we strengthen the special HVZK notion such that it is possible to explain simulated Σ-protocol transcripts.

Definition 5.4 (Explainable Special HVZK). *Let $\mathcal{R}$ be an NP-relation and $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ be a Σ-protocol for $\mathcal{R}$. Denote by $\mathcal{R}_{\Sigma,\$}$ the randomness space of Init and assume that Resp is deterministic* [11].

[11] This assumption is without loss of generality since Init can forward the random coins for Resp in the state st.

We say that Π_Σ *is* explainable *special* HVZK*, if there exists a PPT zero-knowledge simulators* $\Pi_\Sigma.\mathsf{ZKSim}$ *and* $\Pi_\Sigma.\mathsf{ZKExp}$ *such that for any (potentially unbounded) adversary* Adv*, it holds that for any* $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ *and* $c \in \mathcal{C}$ *that* $\mathsf{D}_{\mathsf{real}} = \mathsf{D}_{\mathsf{sim}}$ *for*

$$\mathsf{D}_{\mathsf{real}} := \{(a, c, z, \rho) \mid \rho \leftarrow \mathcal{R}_{\Sigma,\$}, (a, \mathsf{st}) := \mathsf{Init}(\mathbb{x}, \mathbb{w}; \rho), z := \mathsf{Resp}(\mathsf{st}, c)\},$$
$$\mathsf{D}_{\mathsf{sim}} := \{(a, c, z, \rho) \mid (a, z, \mathsf{st}_{\mathsf{sim}}) \leftarrow \Pi_\Sigma.\mathsf{ZKSim}(\mathbb{x}, c), \rho \leftarrow \Pi_\Sigma.\mathsf{ZKExp}(\mathsf{st}_{\mathsf{sim}}, \mathbb{w})\}.$$

Before we provide a formal proof, we comment on a subtle issue when formalizing the above intuition. When the randomness is revealed, the simulator must also provide the randomness for *failed* runs (*i.e.*, when $h_i \neq 0^b$ within the loop). The simulator sketched above does not suffice for this purpose since the proofs are generated within a single run; this is unlikely in the real game. Therefore, we change the simulation strategy as follows.

- The simulator aborts if H queries are made that led to failed runs during simulation. Here, the crucial observation is that challenges c_i have high min-entropy. Therefore, such queries will only be made when the sampled c_i is revealed to $\mathcal{A}$ through a call to $\mathcal{O}_{\mathsf{rvl}}$.
- The simulator programs $\mathcal{O}_{\mathsf{H}}$ consistently in $\mathcal{O}_{\mathsf{rvl}}$ via the provided witness $\mathbb{w}$, *i.e.*, it employs $\Pi_\Sigma.\mathsf{ZKExp}$ to simulate the randomness ρ_i which was used to compute a_i. Then, the simulator recomputes the Σ-protocol state st_i via ρ_i, and given st_i, it recomputes responses z_i for failed challenges c_i and embeds a well-distributed output h_i into H.

We are now ready to establish the properties satisfied by the labeled Fischlin transformation. Correctness is immediate.

Theorem 5.5. *Let* $\mathcal{R}$ *be an NP-relation and* $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ *be a correct* Σ*-protocol for* $\mathcal{R}$*. Then, the labeled NIZK* $\mathsf{RF}[\Pi_\Sigma]$ *is correct with correctness error* $\kappa \leqslant r \cdot e^{-2^{t-b}}$*. Moreover, an honest prover runs in at most* $\mathsf{poly}(2^t)$ *steps.*

Proof. This follows as in [KRW24, Lemma 3] and we omit details. □

Security follows from the above. We give a formal proof in Section B.

Theorem 5.6. *Let* $\mathcal{R}$ *be an NP-relation and* $\Pi_\Sigma = (\mathsf{Init}, \mathsf{Resp}, \mathsf{Verify})$ *be a* Σ*-protocol for* $\mathcal{R}$*. Let* $\tilde{R} \supseteq \mathcal{R}$*. Assume that* Π_Σ *is correct, explainable special* HVZK*, strong* 2*-special sound for* $\tilde{R}$ *and has high min-entropy. Denote by* $\Pi := \mathsf{RF}[\Pi_\Sigma]$*. Then, there exists a stateful PPT simulator* ZKSim *such that for every PPT adversary* $\mathcal{A}$ *we have*

$$\mathsf{Adv}^{\mathsf{SIM\text{-}EXT}}_{\Pi,\mathcal{A},\mathsf{ZKSim}}(1^\lambda) \leqslant \frac{Q_{\mathsf{H}} \cdot Q_{\mathsf{prv}} \cdot 2^t}{|\mathcal{C}|} + Q_{\mathsf{H}} \cdot 2^{-rb}.$$

6 Our Threshold Signature

We now present our three-round adaptively secure threshold signature scheme Rackle. The construction is depicted in Fig. 8. It relies on a product-equivocable commitment scheme BCJ, as defined in Section 3.5.1, to commit to the ephemeral nonces in the first round. Also, let $\Pi_{\mathsf{aux}} = (\Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}, \Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}})$ be an NIZK with random oracle $\mathsf{H}_{\mathsf{aux}}$ for the relation

$$\mathcal{R}_{\mathsf{aux}} := \{(\mathbb{x}, \mathbb{w}) \mid \mathbf{C} = (G^s H^t, Y^s Z^t M) \ \wedge \ M = G^r\}, \tag{1}$$

where $\mathbb{x} = (G, Y, H, Z, \mathbf{C}) \in \mathbb{G}^6$ and $\mathbb{w} = (r, s, t) \in \mathbb{Z}_p^3$. Also, assume that Π_{aux} is simulation-extractable for the relation

$$\tilde{\mathcal{R}}_{\mathsf{aux}} := \{(\mathbb{x}, \mathbb{w}) \mid ((G, Y, Z), \mathbb{w}) \in \mathcal{R}_{\mathsf{dl}} \ \vee \ (\mathbb{x}, \mathbb{w}) \in \mathcal{R}_{\mathsf{aux}}\}, \tag{2}$$

where $\mathcal{R}_{\mathsf{dl}}$ is the relation containing *non-trivial* discrete logarithms, formally defined in Section C (cf. Eq. (7)). We give an instantiation for Π_{aux} in Section C. Looking ahead, Π_{aux} will allow us to prove knowledge of the committed nonces. The signing protocol proceeds in three rounds, followed by an aggregation step.

$\mathsf{Setup}(1^\lambda, N, T)$

$(\mathbb{G}, G, p) \leftarrow \mathsf{GenG}(1^\lambda)$
$\mathsf{ck} = (G, H, Y, Z) \xleftarrow{\$} \mathsf{CKSetup}(1^\lambda)$
$\mathsf{tspar} := (\mathsf{ck}, N, T)$
return tspar

$\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig})$

parse $(c, z) \leftarrow \mathsf{sig}$
parse $(\mathsf{tspar}, X) \leftarrow \mathsf{vk}$
$c' := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, G^z \cdot X^{-c})$
return $(c = c')$

$\mathsf{KeyGen}(\mathsf{tspar})$

$x \xleftarrow{\$} \mathbb{Z}_p$
$X \leftarrow G^x$
for $i \in [N]$ **do**
  for $j \in [N]$ **do**
    $\mathsf{rand}_{i,j} \xleftarrow{\$} \{0,1\}^\lambda$
    $\mathsf{seed}_{i,j} := i \| j \| \mathsf{rand}_{i,j}$
$(\vec{\mathsf{seed}}_i)_{i\in[N]} := \left((\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j\in[N]}\right)_{i\in[N]}$
$(x_i)_{i\in[N]} := \mathsf{LSecShr}(x, N, T)$
$\mathsf{vk} := (\mathsf{tspar}, X)$
$(\mathsf{sk}_i)_{i\in[N]} := \left(x_i, \vec{\mathsf{seed}}_i\right)_{i\in[N]}$
return $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]})$

$\mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_{b,j})_{(b,j)\in[3]\times\mathsf{SS}})$

parse $(\mathbf{C}_j, \pi_j, \widetilde{\mathbf{s}}_j, \widetilde{z}_j)_{j\in\mathsf{SS}}$
  $\leftarrow (\mathsf{pm}_{1,j}, \mathsf{pm}_{2,j}, \mathsf{pm}_{3,j})_{j\in\mathsf{SS}}$
$\mathbf{s} := \sum_{j\in\mathsf{SS}} \widetilde{\mathbf{s}}_j$; $\mathbf{C} := \prod_{j\in\mathsf{SS}} \mathbf{C}_j$
$R := \mathsf{RcvMsg}(\mathsf{ck}, \mathbf{C}, \mathbf{s})$
$c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$
$z := \sum_{j\in\mathsf{SS}} \widetilde{z}_j$
return (c, z)

$\mathsf{Sign}_1(\mathsf{vk}, i, \mathsf{sk}_i)$

parse $\left(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i\right) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$
$r_i \leftarrow \mathbb{Z}_p$; $R_i \leftarrow G^{r_i}$
$\mathbf{s}_i \leftarrow \mathbb{Z}_p^2$; $\mathbf{C}_i := \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; \mathbf{s}_i)$
$\mathbb{x}_i := (\mathsf{ck}, \mathbf{C}_i)$; $\mathbb{w}_i := (r_i, \mathbf{s}_i)$; $\mathbb{t}_i := i$
$\rho_i \leftarrow \mathcal{R}_{\mathsf{nizk}}$
$\pi_i \leftarrow \Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_i, \mathbb{w}_i, \mathbb{t}_i)$
$\mathsf{st}_i \leftarrow (\mathbf{C}_i, r_i, \mathbf{s}_i)$
return $(\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i), \mathsf{st}_i)$

$\mathsf{Sign}_2(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $(\mathbf{C}_j, \pi_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}$
parse $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$
for $j \in \mathsf{SS}$ **do**
  $\mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j)$; $\mathbb{t}_j := j$
  require $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$
$\widetilde{\mathsf{ctnt}} := 0 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}}$
$\widetilde{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
$\widetilde{\mathbf{s}}_i := \mathbf{s}_i + \widetilde{\Delta}_i$
$\mathsf{st}_i \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i)$
return $(\mathsf{pm}_{2,i} := \widetilde{\mathbf{s}}_i, \mathsf{st}_i)$

$\mathsf{Sign}_3(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $(\widetilde{\mathbf{s}}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}}$
parse $(\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i) \leftarrow \mathsf{st}_i$
$\mathbf{s} := \sum_{j\in\mathsf{SS}} \widetilde{\mathbf{s}}_j$; $\mathbf{C} := \prod_{j\in\mathsf{SS}} \mathbf{C}_j$
// Recover committed aggregate nonce via opening
$R := \mathsf{RcvMsg}(\mathsf{ck}, \mathbf{C}, \mathbf{s})$
require $\mathbf{C} = \mathsf{BCJ.Commit}(\mathsf{ck}, R; \mathbf{s})$
$c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$
$\mathsf{ctnt} := 1 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}} \| R$
$\Delta_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt})$
$\widetilde{z}_i := c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i$
$\mathsf{st}_i \leftarrow \widetilde{\mathsf{ctnt}}$ // $\widetilde{\mathsf{ctnt}} = 0 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}}$
return $(\mathsf{pm}_{3,i} := \widetilde{z}_i, \mathsf{st}_i)$

Figure 8: Setup, KeyGen, Verify, and the signing protocol of our three round threshold signature Rackle.

6.1 Correctness

The following establishes the correctness of our protocol.

Lemma 6.1. *The three-round threshold signature* Rackle *in Fig. 8 is correct.*

Proof. We first check that honestly generated protocol messages pass the requirements in the signing protocol. Specifically, we have two checks: the verification of π_i for all signers in the second round, and the validity check of the aggregated commitment and opening $(\mathbf{C}, \mathbf{s})$ in the final round. Due to Theorem 5.5, π_i produced by $\Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_i, \mathbb{w}_i, \mathbb{t}_i)$ in Sign_1 satisfies $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$ except with negligible probability. Also, following the fact that $\sum_{j\in\mathsf{SS}} \widetilde{\Delta}_j = 0$, the homomorphism of BCJ and message recoverablility (c.f., Theorem 3.11), the following equations, which is the requirement in Sign_3, always hold:

$$\prod_{j\in\mathsf{SS}} \mathbf{C}_i = \mathsf{BCJ.Commit}\left(\mathsf{ck}, R; \sum_{j\in\mathsf{SS}} \widetilde{\mathbf{s}}_i\right), \text{ and } R = \mathsf{RcvMsg}\left(\mathsf{ck}, \prod_{j\in\mathsf{SS}} \mathbf{C}_i, \sum_{j\in\mathsf{SS}} \widetilde{\mathbf{s}}_i\right) = \prod_{j\in\mathsf{SS}} G^{r_i}.$$

Thus an honest execution correctly terminates except with negligible probability.

Finally, we see that (c, z) produced by Agg is a valid signature. Because RcvMsg is deterministic, R in Agg is the same as R used in the final round and satisfies $R = \prod_{j\in\mathsf{SS}} G^{r_i}$. Also, because of $\sum_{j\in\mathsf{SS}} \Delta_j = 0$, we have $z = \sum_{j\in\mathsf{SS}}(c \cdot L_{\mathsf{SS},j} \cdot x_j + r_j + \Delta_j) = c \cdot x + \sum_{j\in\mathsf{SS}} r_j$. Then we obtain $G^z \cdot X^{-c} = \prod_{j\in\mathsf{SS}} G^{r_j} = R$. Therefore (c, z) produced by Agg passes the verification. □

6.2 Adaptive Security

The following theorem establishes the adaptive security of Rackle.

Theorem 6.2. *The 3-round threshold signature* Rackle *in Fig. 8 is adaptive secure under the* DL *assumption.*

Formally, for any N and T with $T \leqslant N$ and an adversary $\mathcal{A}$ against the adaptive security game making at most Q_{H_c}, $Q_{\mathsf{H}_{\mathsf{mask}}}$, $Q_{\mathsf{H}_{\mathsf{aux}}}$ and Q_{S} queries to the random oracles H_c, $\mathsf{H}_{\mathsf{mask}}$, and $\mathsf{H}_{\mathsf{aux}}$ and the signing oracle, respectively, if the DL *problem for* GenG *is $(\varepsilon_{dl}, T_{dl})$-hard, we have*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{Rackle},\mathcal{A}}(1^\lambda, N, T) \leqslant (Q_{\mathsf{H}_c} + Q_{\mathsf{S}}) \cdot \sqrt{(Q_{\mathsf{H}_c} + 1) \cdot \varepsilon_{dl}} + \frac{3p-1}{p-1} \cdot \varepsilon_{dl} + \frac{N \cdot Q_{\mathsf{S}}^2}{p^2} \\ + \frac{Q_{\mathsf{S}}(Q_{\mathsf{H}_c} + Q_{\mathsf{S}} + Q_{\mathsf{H}_{\mathsf{aux}}} \cdot 2^t) + Q_{\mathsf{H}_c} + 10}{p} + \frac{Q_{\mathsf{H}_{\mathsf{mask}}} + Q_{\mathsf{H}_{\mathsf{aux}}}}{2^\lambda}\end{aligned}$$

where t is a parameter of Π_{aux} (see Section 5.2) and $2 \cdot \mathsf{Time}(\mathcal{A}) \approx T_{dl}$.

6.2.1 Proof Overview

We use a sequence of hybrids to transition from the real security game to a final game where the challenger can simulate the game without the secret key x. This enables a reduction from the DL problem. The proof is structured in three main phases.

First, we change the game to allow the challenger to extract the nonces from corrupted signers' NIZK proofs, while simulating the proofs for honest signers to remove the dependency on individual nonces r_i and commitment openings $\mathbf{s}_i$. At the end of this phase, the challenger has access to the aggregated nonce R for a signing session, which is needed to program the random oracle H_c later on.

Second, we alter how the masked openings $\widetilde{\mathbf{s}}_i$ and masked responses $\widetilde{z}_i$ are generated for honest signers. Instead of computing them from secret values, they are chosen randomly for all but the last honest signer in a session. The last signer's values are then computed to ensure consistency, without using individual honest secret nonces. This is possible thanks to the zero-sharing mechanism and the equivocability of the commitment scheme. This phase culminates in programming the oracle H_c and simulating commitments.

Finally, we remove the dependency on the secret key x and individual secret shares x_i for honest signers. The public key is chosen randomly, and a forgery from the adversary can be used to solve a SelfTargetDL instance.

The hybrid argument.

We give a rough sketch of the hybrid argument below, starting with the real adaptive security game Hyb_1.

First phase (Hyb_2 to Hyb_5): This phase focuses on simulating honest signers' proofs and extracting secret nonces from proofs of corrupted or impersonated signers. This allows programming the challenge oracle H_c with a consistent aggregated nonce R.

Hyb_2: The challenger aborts if an honest signer generates the same commitment $\mathbf{C}_i$ twice. This ensures in the rest of the proof that the zero-share mechanism always gets a fresh input since it contains the set of commitments of the session.

Hyb_3: We introduce internal lists for the challenger to track signers' states and simulated values. This does not affect the adversary's view.

Hyb_4: We use the NIZK simulator for honest signers' proofs and the extractor to obtain a witness for $\tilde{\mathcal{R}}_{\mathsf{aux}}$ (see Section 5) for proofs from corrupted or impersonated signers (*i.e.*, honest signers whose commitment has been replaced with malicious commitments in the second round). This change is indistinguishable due to the simulation-extractability of the NIZK. Here, it is important that each signer i uses its index i as label for the proof π_i associated to $\mathbf{C}_i$. Therefore, even if an honest commitment $\mathbf{C}_i$ is used for some malicious signer $j \in \mathsf{CS}$, the proof π_j associated to $\mathbf{C}_j = \mathbf{C}_i$ ensures that the nonce is extractable, even if, later on, the nonce in $\mathbf{C}_i$ is not yet determined.

Hyb_5: The game aborts if an extracted witness satisfies $\mathcal{R}_{\mathsf{dl}}$, instead of being a commitment opening. This happens with negligible probability, as we show that we can extract a DL solution when this abort occurs. At the end of this hybrid, the challenger has access to all nonces r_i and openings $\mathbf{s}_i$.

Second phase (Hyb_6 to Hyb_{12}): In this phase, we change the generation of masked openings $\widetilde{\mathbf{s}}_i$ and responses $\widetilde{z}_i$ to rely on simulation and random oracle programming, to move to a simulation that only depends on their final aggregated values. This then also allows us to simulate commitments and equivocate them without the secret x.

Hyb_6: The game aborts if the adversary queries the masking function's seed for two honest parties. This is a negligible probability event.

Hyb_7: We change how masked openings $\widetilde{\mathbf{s}}_i$ for honest signers are generated. Instead of computing them from secret shares, they are sampled randomly, except for the last honest signer in a session, whose $\widetilde{\mathbf{s}}_i$ is computed to ensure the sum is correct. This is indistinguishable thanks to the zero-sharing mechanism.

Hyb_8: The game aborts if the aggregated nonce R computed from extracted/simulated nonces does not match the one revealed in the protocol. This is ruled out by the binding property of the commitment scheme.

Hyb_9: The challenger now pre-computes the aggregated nonce R and programs the random oracle H_c to output a chosen challenge c on input $(\mathsf{vk}, \mathsf{M}, R)$. This is possible because we can extract all constituent nonces and the commitment scheme is perfectly hiding.

Hyb_{10}: We introduce more internal lists for the challenger to prepare for the simulation of the final responses $\widetilde{z}_i$. This is another conceptual change.

Hyb_{11}: Similar to Hyb_7, we change how the responses $\widetilde{z}_i$ for honest signers are generated. They are sampled randomly, except for the last honest signer, whose response is computed for consistency from the aggregated response $z = c \cdot x + \sum_{i \in \mathsf{SS} \cap \mathsf{HS}} r_i$, simulated responses $\widetilde{z}_i$ and corrupted values.

Hyb_{12}: We use the equivocability of the commitment scheme to simulate the commitments $\mathbf{C}_i$ for honest signers. At this point, the simulation only needs to consistently equivocate the opening of the product of honest commitments according to an aggregated response z, as well as all but one individual honest commitment in a session to answer corruptions. This change is made possible by the restricted *product* equivocability of BCJ.

Final phase (Hyb_{13} and Hyb_{14}): In this phase, we remove all dependencies on secret keys for honest signers and finalize the game to enable the reduction.

Hyb_{13}**:** The public key X is now chosen as a random group element, and honest signers' secret shares are no longer used (we sample random shares on corruption instead). The adversary's view is unchanged because all honest party computations are now simulated without the secret.

Hyb_{14}**:** The challenger guesses which of the adversary's queries to H_c will be used in the final forgery. If the guess is correct, the adversary's successful forgery will directly provide a solution to the SelfTargetDL instance.

Reduction from SelfTargetDL**.** The reduction works by embedding the SelfTargetDL challenge X as the public key in the security game. A successful forgery by the adversary on a guessed query yields a valid signature (c, z), which is a solution to the SelfTargetDL problem.

6.2.2 Formal Proof

Below we provide the formal security proof of Theorem 6.2, establishing adaptive security of Rackle.

Proof of Theorem 6.2. Let $\mathcal{A}$ be an adversary against the adaptive security game for Rackle. We consider a sequence of games where the first hybrid is the original game and the last is a game that can be reduced to the SelfTargetDL problem. We denote the winning probability of $\mathcal{A}$ in Hyb_i by ϵ_i. We denote a set of honest signers (resp. corrupted signers) in SS, i.e., $\mathsf{SS} \cap \mathsf{HS}$ (resp. $\mathsf{SS} \cap \mathsf{CS}$), by sHS (resp. sCS).

Hyb_1**.** This is the real unforgeability game. This is formalized in in Fig. 9. By definition, we have

$$\epsilon_1 = \mathsf{Adv}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{Rackle},\mathcal{A}}(1^\lambda, N, T).$$

Hyb_2**.** In this hybrid, the challenger aborts the game if an honest signer uses the same commitment twice. This is formalized in Fig. 10.

Concretely, this hybrid maintains a set Cmt_i for each $i \in [N]$, initialized to $\perp$. We modify $\mathcal{O}_{\mathsf{Sign}_1}$, so that, after the generation of $\mathbf{C}_i$, it checks whether $\mathbf{C}_i$ is already defined in Cmt_i. If so, the game aborts (i.e. the game directly returns 0). Otherwise, it adds $\mathbf{C}_i$ to Cmt_i and continues the execution.

Let us bound the probability of aborting the game. $\mathbf{C}_i$ is uniformly distributed over $\mathbb{G}^2$ as r_i and $\mathbf{s}_i$ are uniformly chosen from $\mathbb{Z}_p$ and $\mathbb{Z}_p^2$. Since $\mathcal{A}$ makes at most Q_S signing queries for each honest signer, we have

$$|\epsilon_1 - \epsilon_2| \leqslant \frac{N \cdot Q_\mathsf{S}^2}{p^2}.$$

Hyb_3**.** In this hybrid, we introduce several bookkeeping lists HonestMsg, CtntSid, InitializeOpen, UnOpenedHS, ImpersonatedHS, and MaskedRnd that will be leveraged in the rest of this proof. This is formalized in Fig. 11. We provide a description and the purpose of each list below.

- $(\pi_i, r_i, \mathbf{s}_i) := \mathsf{HonestMsg}[i, \mathbf{C}_i]$. This list associates the proofs π_i, signature randomness r_i, and commitment randomness $\mathbf{s}_i$ to a user id i and commitment $\mathbf{C}_i$ for honest users. This will be leveraged for the simulation in later hybrids.

- $\mathsf{sid} := \mathsf{CtntSid}[i, \mathbf{C}_i]$. This list associates the signing session identifier sid to a user id i and commitment $\mathbf{C}_i$ for honest users. Looking ahead, this will be useful after we simulate $\mathbf{C}_i$ without generating r_i and $\mathbf{s}_i$ in $\mathcal{O}_{\mathsf{Sign}_1}$. These randomnesses will however still need to be simulated in case of corruption, and for consistency we will then need to update $\mathsf{C}[\mathsf{sid}, 1, i]$. As $\mathcal{O}_{\mathsf{Corrupt}}$ does not have access to sid we will use CtntSid to recover it from i and $\mathbf{C}_i$.

$\mathsf{Game}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T)$

// Identical to lines 1-4 in Fig. 2.

5 : $(\mathbb{G}, G, p) \leftarrow \mathsf{GenG}(1^\lambda)$

6 : $\mathsf{ck} = (G, H, Y, Z) \xleftarrow{\$} \mathsf{CKSetup}(1^\lambda)$

7 : $\mathsf{tspar} := (\mathsf{ck}, N, T)$

8 : $x \xleftarrow{\$} \mathbb{Z}_p;\ X \leftarrow G^x$

9 : **for** $i \in [N]$ **do**

10 : **for** $j \in [N]$ **do**

11 : $\mathsf{rand}_{i,j} \xleftarrow{\$} \{0,1\}^\lambda$

12 : $\mathsf{seed}_{i,j} := i \| j \| \mathsf{rand}_{i,j}$

13 : $(\vec{\mathsf{seed}}_i)_{i\in[N]} := \left((\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j\in[N]}\right)_{i\in[N]}$

14 : $(x_i)_{i\in[N]} := \mathsf{LSecShr}(x, N, T)$

15 : $\mathsf{vk} := (\mathsf{tspar}, X)$

16 : $(\mathsf{sk}_i)_{i\in[N]} := (x_i, \vec{\mathsf{seed}}_i)_{i\in[N]}$

17 : $((\mathsf{sig}^*_j)_{j\in[\ell]}, \mathsf{M}^*) \xleftarrow{\$} \mathcal{A}^{(\mathcal{O}_{\mathsf{Sign}_r})_{r\in[3]}, \mathcal{O}_{\mathsf{Corrupt}}, \mathcal{O}_{\mathsf{NextSes}}, \mathsf{H}}(\mathsf{vk})$

18 : **return** 1 **if** $\ell > \left|\mathsf{FinishedSSIDs}(\mathsf{M}^*)\right|$
$\wedge\ \forall j \in [\ell], \mathsf{Verify}(\mathsf{vk}, \mathsf{M}^*, \mathsf{sig}^*_j) = 1$
$\wedge\ (\mathsf{sig}^*_j)_{j\in[\ell]}$ pairwise distinct

19 : **return** 0

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$

1 : **require** $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$

2 : **parse** $(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$

3 : $r_i \leftarrow \mathbb{Z}_p;\ R_i \leftarrow G^{r_i}$

4 : $\mathbf{s}_i \leftarrow \mathbb{Z}_p^2;\ \mathbf{C}_i := \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; \mathbf{s}_i)$

5 : $\mathbb{x}_i := (\mathsf{ck}, \mathbf{C}_i);\ \mathbb{w}_i := (r_i, \mathbf{s}_i);\ \mathbb{t}_i := i$

6 : $\rho_i \leftarrow \mathcal{R}_{\mathsf{nizk}}$

7 : $\pi_i \leftarrow \Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_i, \mathbb{w}_i, \mathbb{t}_i; \rho_i)$

8 : $\mathsf{st}_i \leftarrow (\mathbf{C}_i, r_i, \mathbf{s}_i)$

9 : $\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i)$

10 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$

11 : $\mathsf{RM}[\mathsf{sid}, 1, i] := \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] := \mathsf{st}_i, \mathsf{R}[\mathsf{sid}, i] = 1$

12 : **return** $\mathsf{pm}_{1,i}$

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

7 : **parse** $(\mathbf{C}_j, \pi_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}$

8 : **parse** $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$

9 : **for** $j \in \mathsf{SS}$ **do**

10 : $\mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j);\quad \mathbb{t}_j := j$

11 : **require** $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$

12 : $\widetilde{\mathsf{ctnt}} := 0 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}}$

13 : $\widetilde{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

14 : $\widetilde{\mathbf{s}}_i := \mathbf{s}_i + \widetilde{\Delta}_i$

15 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i)$

16 : $\mathsf{pm}_{2,i} := \widetilde{\mathbf{s}}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Sign}_3}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

7 : **parse** $(\widetilde{\mathbf{s}}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}}$

8 : **parse** $(\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i) \leftarrow \mathsf{st}_i$

9 : $\mathbf{s} := \sum_{j\in\mathsf{SS}} \widetilde{\mathbf{s}}_j;\ \mathbf{C} := \prod_{j\in\mathsf{SS}} \mathbf{C}_j$

10 : $R := \mathsf{RcvMsg}(\mathsf{ck}, \mathbf{C}, \mathbf{s})$

11 : **require** $\mathbf{C} = \mathsf{BCJ.Commit}(\mathsf{ck}, R; \mathbf{s})$

12 : $c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$

13 : $\mathsf{ctnt} := 1 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}} \| R$

14 : $\Delta_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt})$

15 : $\widetilde{z}_i := c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i$

16 : $\mathsf{st}_i \leftarrow \widetilde{\mathsf{ctnt}}$ // $\widetilde{\mathsf{ctnt}} = 0 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j\in\mathsf{SS}}$

17 : $\mathsf{pm}_{3,i} := \widetilde{z}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

1 : **require** $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T - 1]\!]$

2 : $\mathsf{HS} := \mathsf{HS} \backslash \{i\};\ |\mathsf{CS} := \mathsf{CS} \cup \{i\}$

3 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

Figure 9: The first game, which is identical to the security game. Oracle $\mathcal{O}_{\mathsf{NextSes}}$ is identical to one in Fig. 2.

- $\mathsf{SS} := \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$. This list stores the signer set SS included in the signing context $\widetilde{\mathsf{ctnt}}$. Note that we only set $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \mathsf{SS}$ when a honest party executes the second round with $\widetilde{\mathsf{ctnt}}$, such that if $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \neq \bot$, there is at least one honest who has completed the second round under $\widetilde{\mathsf{ctnt}}$. When $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \bot$, no honest signer executed the second round with $\widetilde{\mathsf{ctnt}}$.
- $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \subseteq \mathsf{sHS}$. This list maintains the set of *honest* signers that did not execute the second

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$

1 : **require** $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$
2 : **parse** $(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$
3 : $r_i \leftarrow \mathbb{Z}_p;\ R_i \leftarrow G^{r_i}$
4 : $\mathbf{s}_i \leftarrow \mathbb{Z}_p^2;\ \mathbf{C}_i := \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; \mathbf{s}_i)$
5 : **abort if** $\mathbf{C}_i \in \mathsf{Cmt}_i$
6 : $\mathsf{Cmt}_i \leftarrow \mathsf{Cmt}_i \cup \{\mathbf{C}_i\}$
7 : $\mathbb{x}_i := (\mathsf{ck}, \mathbf{C}_i);\ \mathbb{w}_i := (r_i, \mathbf{s}_i);\ \mathbb{t}_i := i$
8 : $\rho_i \leftarrow \mathcal{R}_{\mathsf{nizk}}$
9 : $\pi_i \leftarrow \Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_i, \mathbb{w}_i, \mathbb{t}_i; \rho_i)$
10 : $\mathsf{st}_i \leftarrow (\mathbf{C}_i, r_i, \mathbf{s}_i)$
11 : $\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i)$
12 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$
13 : $\mathsf{RM}[\mathsf{sid}, 1, i] := \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] := \mathsf{st}_i, \mathsf{R}[\mathsf{sid}, i] = 1$
14 : **return** $\mathsf{pm}_{1,i}$

Figure 10: The second hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes empty sets $\mathsf{Cmt}_i := \varnothing$ for $i \in [N]$ at the beginning of the game.

of signing with $\widetilde{\mathsf{ctnt}}$ yet. This will be used to determine the last honest signer in the second round, who will be in charge of simulating the final protocol message of the second round.

- $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \subseteq \mathsf{sHS}$. This list stores the set of *honest* signers whose $\mathbf{C}_j$ in the list $\widetilde{\mathsf{ctnt}}$ was not actually generated in $\mathcal{O}_{\mathsf{Sign}_1}$ and is replaced by a different one chosen by the adversary $\mathcal{A}$. Looking ahead, in order to simulate the signing oracles, the challenger will need to extract committed messages and randomness from the proofs π_j generated by the adversary $\mathcal{A}$ thanks to the extractability of NIZK. To tackle the potential impersonation raised above, we will run the extractor on the proofs of both corrupted parties, and honest parties in $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$.
- $\widetilde{\mathbf{s}}_i := \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i]$. This list stores the masked opening $\widetilde{\mathbf{s}}_i$ of signer i under $\widetilde{\mathsf{ctnt}}$ used in its second protocol message. This will be used to remove the dependency of $\widetilde{\mathbf{s}}_i$ on the opening $\mathbf{s}_i$ and keep track of its replacement – a fresh uniform sample, until the last honest signer executes the second round.

Concretely, in $\mathcal{O}_{\mathsf{Sign}_1}$, after generating $\mathbf{C}_i$ and π_i, the game stores $(\pi_i, r_i, \mathbf{s}_i)$ in $\mathsf{HonestMsg}[i, \mathbf{C}_i]$ and sid in $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$. Due to the uniqueness of honest users' own commitment enforced in the previous game, once an index is set for these lists, it will never be overwritten. In $\mathcal{O}_{\mathsf{Sign}_2}$, if $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \bot$ (i.e. if signer i is the first honest signer executing the second round with $\widetilde{\mathsf{ctnt}}$), the game sets $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$ and $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$. Moreover, the game adds honest signers' indices whose $(\mathbf{C}_j, \pi_j)$ does not satisfy $\mathsf{HonestMsg}[j, \mathbf{C}_j] = (\pi_j, \cdot, \cdot)$ to $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$, which is equivalent to checking that $(\mathbf{C}_j, \pi_j)$ was not honestly generated by $\mathcal{O}_{\mathsf{Sign}_1}$. After the generation of $\widetilde{\mathbf{s}}_i$ in $\mathcal{O}_{\mathsf{Sign}_2}$, the game stores it in $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i]$ and removes i from $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]$. Finally, in $\mathcal{O}_{\mathsf{Corrupt}}$, the game removes i from $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]$ and $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ for all $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$.

Notice that $\widetilde{\mathsf{ctnt}}$ may include $\mathbf{C}_i$ generated by $\mathcal{A}$ in place of honest signers' ones, and then signing session queries under such $\widetilde{\mathsf{ctnt}}$ are inconsistent among honest signers. At first glance, it may seem that the challenger does not need to care for such queries as the signature will be hidden by our masking technique as soon as honest signers have an inconsistent view. However, in fact, $\mathcal{A}$ may later corrupt users for which it replaced a commitment and recompute values as if the views were consistent from the start, requiring careful bookkeeping of these impersonations.

All the modifications made in this hybrid are conceptual and the view of $\mathcal{A}$ does not change. Therefore, we have

$$\epsilon_2 = \epsilon_3.$$

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$

1 : **require** $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$
2 : **parse** $(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$
3 : $r_i \leftarrow \mathbb{Z}_p;\ R_i \leftarrow G^{r_i}$
4 : $\mathbf{s}_i \leftarrow \mathbb{Z}_p^2;\ \mathbf{C}_i := \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; \mathbf{s}_i)$
5 : **require** $\mathbf{C}_i \notin \mathsf{Cmt}_i$
6 : $\mathsf{Cmt}_i \leftarrow \mathsf{Cmt}_i \cup \{\mathbf{C}_i\}$
7 : $\mathbb{x}_i := (\mathsf{ck}, \mathbf{C}_i);\ \mathbb{w}_i := (r_i, \mathbf{s}_i);\ \mathbb{t}_i := i$
8 : $\rho_i \leftarrow \mathcal{R}_{\mathsf{nizk}}$
9 : $\pi_i \leftarrow \Pi_{\mathsf{aux}}.\mathsf{Prove}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_i, \mathbb{w}_i, \mathbb{t}_i; \rho_i)$
10 : $\mathsf{HonestMsg}[i, \mathbf{C}_i] \leftarrow (\pi_i, r_i, \mathbf{s}_i)$
11 : $\mathsf{CtntSid}[i, \mathbf{C}_i] \leftarrow \mathsf{sid}$
12 : $\mathsf{st}_i \leftarrow (\mathbf{C}_i, r_i, \mathbf{s}_i)$
13 : $\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i)$
14 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$
15 : $\mathsf{RM}[\mathsf{sid}, 1, i] := \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] := \mathsf{st}_i, \mathsf{R}[\mathsf{sid}, i] = 1$
16 : **return** $\mathsf{pm}_{1,i}$

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

1 : **require** $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T - 1]\!]$
2 : **for** $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$ **do**
3 : $\quad \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
4 : $\quad \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
5 : $\mathsf{HS} := \mathsf{HS} \backslash \{i\};\ \mathsf{CS} := \mathsf{CS} \cup \{i\}$
6 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j \in \mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.
7 : **parse** $(\mathbf{C}_j, \pi_j)_{j \in \mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j \in \mathsf{SS}}$
8 : **parse** $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$
9 : **for** $j \in \mathsf{SS}$ **do**
10 : $\quad \mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j); \quad \mathbb{t}_j := j$
11 : $\quad$ **require** $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$
12 : $\widetilde{\mathsf{ctnt}} := 0 \| \mathsf{SS} \| \mathsf{M} \| (\mathbf{C}_j)_{j \in \mathsf{SS}}$
13 : **if** $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \perp$ **then**
14 : $\quad \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$
15 : $\quad \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$
16 : $\quad \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$
$\quad\quad \leftarrow \{j \in \mathsf{sHS} | \mathsf{HonestMsg}[j, \mathbf{C}_j] \neq (\pi_j, \cdot, \cdot)\}$
17 : $\widetilde{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
18 : $\widetilde{\mathbf{s}}_i := \mathbf{s}_i + \widetilde{\Delta}_i$
19 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\mathbf{s}}_i$
20 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
21 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j \in \mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i)$
22 : $\mathsf{pm}_{2,i} := \widetilde{\mathbf{s}}_i$
// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

Figure 11: The third hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes empty lists $\mathsf{HonestMsg}[\cdot]$, $\mathsf{CtntSid}[\cdot]$, $\mathsf{InitializeOpen}[\cdot]$, $\mathsf{UnOpenedHS}[\cdot]$, $\mathsf{ImpersonatedHS}[\cdot]$, $\mathsf{MaskedRnd}[\cdot] := \perp$ at the beginning of the game.

Hyb_4. In this hybrid, the challenger simulates the proofs of honest signers in the first round and extracts witnesses from the proofs provided by the adversary $\mathcal{A}$. This is formalized in Fig. 12.

More concretely, the challenger will maintain a list Opened storing the $(r_j, \mathbf{s}_j)$ pairs for some signers in the session for $\widetilde{\mathsf{ctnt}}$. In $\mathcal{O}_{\mathsf{Sign}_1}$, the challenger replaces the honest sampling of π_i with $\mathsf{ZKSim}(\mathsf{prv}, (\mathsf{ck}, \mathbf{C}_i), i)$. In $\mathcal{O}_{\mathsf{Sign}_2}$, when $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \perp$, it extracts the witness of π_j for indices j of corrupted and impersonated honest signers. We need to be careful when performing this step as corrupted signers may use proofs that were simulated while the signer was still honest. In this case – detected by checking if $\mathsf{HonestMsg}[j, \mathbf{C}_j] = (\pi_j, \cdot, \cdot)$ holds, we cannot use the extractor to obtain the witness from the proof and we instead recover r_j and $\mathbf{s}_j$ from

$\mathsf{HonestMsg}[j, \mathbf{C}_j]$ and add them to $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$. In the general case when the proof is generated by $\mathcal{A}$, the game extracts $\mathbb{w}_i$ using $\mathsf{ZKSim}(\mathsf{ext}, (\mathsf{ck}, \mathbf{C}_j), \pi_j, j)$ and if the extraction fails, i.e., $\mathbb{w} \notin \tilde{\mathcal{R}}_{\mathsf{aux}}$ (c.f. Section C), we abort the game.

In $\mathcal{O}_{\mathsf{Corrupt}}$, when party i gets corrupted, we need to explain the simulated proofs. For $\mathbf{C}_i$ such that $(\pi_i, r_i, \mathbf{s}_i) := \mathsf{HonestMsg}[i, \mathbf{C}_i] \neq \bot$, we simulate the proof randomness using $\rho_i := \mathsf{ZKSim}(\mathsf{rvl}, (\mathsf{ck}, \mathbf{C}_i), (r_i, \mathbf{s}_i), i, \pi_i)$, and then we store the final coins in $\mathsf{C}[\mathsf{sid}, 1, i]$ – where sid is stored in $\mathsf{CtntSid}[i, \mathbf{C}_i]$. Moreover, before updating $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]$ and $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$, if i is not in $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$, we add $(r_i, \mathbf{s}_i)$ to $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$. We also manage the random oracle $\mathsf{H}_{\mathsf{aux}}$ via ZKSim, i.e. when inp is queried, it returns the output of $\mathsf{ZKSim}(\mathsf{hsh}, \mathsf{inp})$.

Below, we show that there is an adversary $\mathcal{B}_1$ breaking the simulation-extractability for labeled NIZKs if $\mathcal{A}$ distinguishes Hyb_3 and Hyb_4. $\mathcal{B}_1$ replicates the behavior of the challenger in Hyb_3 and Hyb_4 with the simulation-extractability oracles (c.f. Fig. 6). Specifically, it first initializes a counter $\mathsf{ctr}_{\mathsf{prv}}$ to 0 at the beginning of the game. In $\mathcal{O}_{\mathsf{Sign}_1}$, it queries $((\mathsf{ck}, \mathbf{C}_i), (r_i, \mathbf{s}_i), i)$ to $\mathcal{O}_{\mathsf{prove}}$ and uses the received π as π_i. It stores $\mathsf{ctr}_{\mathsf{prv}}$ with i and $\mathbf{C}_i$ and increases $\mathsf{ctr}_{\mathsf{prv}} \leftarrow \mathsf{ctr}_{\mathsf{prv}} + 1$. In $\mathcal{O}_{\mathsf{Sign}_2}$, when it extracts a witness, it queries $((\mathsf{ck}, \mathbf{C}_j), \pi_j, j)$ to $\mathcal{O}_{\mathsf{ext}}$. If it receives 0, it terminates and outputs 0. Otherwise, it continues the game. In $\mathcal{O}_{\mathsf{Corrupt}}$, it queries to $\mathcal{O}_{\mathsf{rvl}}$ a randomness ρ_i to explain the witness $\mathbf{C}_i$. Moreover, it passes queries for $\mathsf{H}_{\mathsf{aux}}$ though $\mathcal{O}_{\mathsf{H}}$ and returns the received value y. Finally, it outputs 1 if $\mathcal{A}$ wins the game. Otherwise, it outputs 0.

We now argue that $\mathcal{B}_1$ perfectly simulates games Hyb_3 and Hyb_4 for the adversary $\mathcal{A}$.

First, observe that the adversary's view is identical. When $\mathcal{B}_1$ is in the real world, its interaction with $\mathcal{A}$ is distributed identically to Hyb_3. Likewise, when $\mathcal{B}_1$ is in the ideal world, its interaction with $\mathcal{A}$ is distributed identically to Hyb_4. The generation of values such as π, ρ, and y by $\mathcal{B}_1$ follows the same distribution as in the original challenger code for Hyb_3 and Hyb_4. Furthermore, the step where $\mathcal{B}_1$ stores $(r_i, \mathbf{s}_i)$ in the $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ list is a conceptual change for the reduction's bookkeeping and does not affect the adversary's view.

The main subtlety lies in the handling of the $\mathcal{O}_{\mathsf{Sign}_2}$ oracle. $\mathcal{B}_1$ queries its external oracle $\mathcal{O}_{\mathsf{ext}}$ only when $(\mathbf{C}_j, \pi_j)$ pairs are valid for all signers.

- In the real world, $\mathcal{O}_{\mathsf{ext}}$ always returns 1. Thus, $\mathcal{B}_1$ never aborts within this oracle, perfectly matching the behavior of Hyb_3.
- In the ideal world, $\mathcal{O}_{\mathsf{ext}}$ returns 0 if and only if the extracted witness $\mathbb{w}$ is not in the relation $\tilde{\mathcal{R}}_{\mathsf{aux}}$. Importantly, when $\mathcal{A}$ produces a simulated π_i with $\mathbf{C}_i$ in signing queries such that $\mathsf{HonestMsg}[i, \mathbf{C}_i] = (\pi_i, \cdot, \cdot)$, $\mathcal{B}_1$ does not make a query π_i to $\mathcal{O}_{\mathsf{ext}}$ along with $\mathbf{C}_i$ and $\mathbb{t} = i$. We note that $\mathcal{A}$ may reuse a simulated π_i with some $\mathbf{C}_j$ and j such that $\mathsf{HonestMsg}[j, \mathbf{C}_j] \neq (\pi_i, \cdot, \cdot)$ but a query $((\mathsf{ck}, \mathbf{C}_j), \pi_i, j)$ to $\mathcal{O}_{\mathsf{ext}}$ satisfies $\mathsf{T}_{\mathsf{proof}}[\mathsf{ctr}] \neq (\mathbb{x}, \cdot, \mathbb{t}, \pi)$ for all $\mathsf{ctr} \in [\mathsf{ctr}_{\mathsf{prv}}]$ in $\mathcal{O}_{\mathsf{ext}}$. Therefore, the probability of receiving 0 from $\mathcal{O}_{\mathsf{ext}}$ is identical to the probability that the game aborts due to the new abort condition introduced in Hyb_4.

Since the simulation is perfect in both cases, we can conclude that the probability of $\mathcal{A}$ winning is the same. Therefore, we have

$$|\epsilon_3 - \epsilon_4| \leqslant \mathsf{Adv}^{\mathsf{SIM\text{-}EXT}}_{\Pi_{\mathsf{aux}}, \mathcal{B}_1, \mathsf{ZKSim}}(1^\lambda).$$

Hyb_5. In this hybrid, we change the abort condition in $\mathcal{O}_{\mathsf{Sign}_2}$ added in the previous hybrid. This is formalized in Fig. 13. Specifically, after extracting $\mathbb{w}$, the game checks whether $((\mathsf{ck}, \mathbf{C}_j), \mathbb{w}) \in \mathcal{R}_{\mathsf{aux}}$ (c.f. Section C). If so, it parses $(r_i, \mathbf{s}_i)$ from $\mathbb{w}$ and stores it in $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$. Otherwise, the game aborts.

Let us analyze the difference in the adversary's advantage between Hyb_4 and Hyb_5. The only change in Hyb_5 is that the game now aborts if the extracted witness $\mathbb{w}$ for a statement $\mathbb{x}$ satisfies the $\mathcal{R}_{\mathsf{dl}}$ relation. Recall that the full relation for the NIZK is $\tilde{\mathcal{R}}_{\mathsf{aux}} = \{(\mathbb{x}, \mathbb{w}) \mid ((G, Y, Z), \mathbb{w}) \in \mathcal{R}_{\mathsf{dl}} \vee (\mathbb{x}, \mathbb{w}) \in \mathcal{R}_{\mathsf{aux}}\}$, where $\mathbb{x} = (G, Y, H, Z, \mathbf{C})$ and

$$\mathcal{R}_{\mathsf{dl}} := \{((G_i)_{i\in[3]}, (x_i)_{i\in[3]}) \mid 1 = \textstyle\prod_{i\in[3]} G_i^{x_i} \wedge \exists i \in [3], x_i \neq 0\}.$$

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$

1 : **require** $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$
2 : **parse** $(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$
3 : $r_i \leftarrow \mathbb{Z}_p;\ R_i \leftarrow G^{r_i}$
4 : $\mathbf{s}_i \leftarrow \mathbb{Z}_p^2;\ \mathbf{C}_i := \mathsf{BCJ.Commit}(\mathsf{ck}, R_i; \mathbf{s}_i)$
5 : **require** $\mathbf{C}_i \notin \mathsf{Cmt}_i$
6 : $\mathsf{Cmt}_i \leftarrow \mathsf{Cmt}_i \cup \{\mathbf{C}_i\}$
7 : $\pi_i \leftarrow \mathsf{ZKSim}(\mathsf{prv}, (\mathsf{ck}, \mathbf{C}_i), i)$
8 : $\mathsf{HonestMsg}[i, \mathbf{C}_i] \leftarrow (\pi_i, r_i, \mathbf{s}_i)$
9 : $\mathsf{CtntSid}[i, \mathbf{C}_i] \leftarrow \mathsf{sid}$
10 : $\mathsf{st}_i \leftarrow (\mathbf{C}_i, r_i, \mathbf{s}_i)$
11 : $\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i)$
12 : $\mathsf{RM}[\mathsf{sid}, 1, i] := \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] := \mathsf{st}_i, \mathsf{R}[\mathsf{sid}, i] = 1$
13 : **return** $\mathsf{pm}_{1,i}$

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

1 : **require** $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T - 1]\!]$
2 : **for** $\mathbf{C}_i$ s.t. $\mathsf{HonestMsg}[i, \mathbf{C}_i] \neq \bot$ **do**
3 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
4 : **parse** $\mathsf{sid} \leftarrow \mathsf{CtntSid}[i, \mathbf{C}_i]$
5 : $\rho_i \leftarrow \mathsf{ZKSim}(\mathsf{rvl}, (\mathsf{ck}, \mathbf{C}_i), (r_i, \mathbf{s}_i), i, \pi_i)$
6 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$
7 : **for** $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$ **do**
8 : **parse** $0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}} \leftarrow \widetilde{\mathsf{ctnt}}$
9 : **if** $i \notin \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **then**
10 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
11 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$
12 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
13 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
14 : $\mathsf{HS} := \mathsf{HS} \backslash \{i\}$
15 : $\mathsf{CS} := \mathsf{CS} \cup \{i\}$
16 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.
7 : **parse** $(\mathbf{C}_j, \pi_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}$
8 : **parse** $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$
9 : **for** $j \in \mathsf{SS}$ **do**
10 : $\mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j); \quad \mathbb{t}_j := j$
11 : **require** $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$
12 : $\widetilde{\mathsf{ctnt}} := 0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}$
13 : **if** $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \bot$ **then**
14 : $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$
15 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$
16 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$
$\leftarrow \{j \in \mathsf{sHS} | \mathsf{HonestMsg}[j, \mathbf{C}_j] \neq (\pi_j, \cdot, \cdot)\}$
17 : **for** $j \in \mathsf{sCS} \cup \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **do**
18 : **if** $\mathsf{HonestMsg}[j, \mathbf{C}_j] = (\pi_j, \cdot, \cdot)$ **do**
19 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$
20 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$
21 : **else**
22 : $\mathbb{w} \leftarrow \mathsf{ZKSim}(\mathsf{ext}, (\mathsf{ck}, \mathbf{C}_j), \pi_j, j)$
23 : **abort if** $((\mathsf{ck}, \mathbf{C}_j), \mathbb{w}) \notin \tilde{\mathcal{R}}_{\mathsf{aux}}$
24 : $\tilde{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
25 : $\tilde{\mathbf{s}}_i := \mathbf{s}_i + \tilde{\Delta}_i$
26 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\mathbf{s}}_i$
27 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$
28 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \tilde{\mathbf{s}}_i)$
29 : $\mathsf{pm}_{2,i} := \tilde{\mathbf{s}}_i$
// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{H}_{\mathsf{aux}}}(\mathsf{inp})$

1 : $y \leftarrow \mathsf{ZKSim}(\mathsf{hsh}, \mathsf{inp})$
2 : **return** y

Figure 12: The fourth hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes an empty list $\mathsf{Opened}[\cdot] := \bot$ at the beginning of the game.

Let E_{dl} be the event that the reduction extracts a witness $\mathbb{w}$ such that $((G, Y, Z), \mathbb{w}) \in \mathcal{R}_{\mathsf{dl}}$. The adversary's view in Hyb_5 is identical to its view in Hyb_4 unless the event E_{dl} occurs, which causes Hyb_5 to abort. Therefore, the difference in advantage is bounded by the probability of this event. We have:

$$|\epsilon_4 - \epsilon_5| \leqslant \Pr[E_{\mathsf{dl}}].$$

To bound the probability of E_{dl}, we construct a reduction $\mathcal{B}_2$ that solves the Discrete Logarithm (DL) problem.

First, we introduce an intermediate game, Hyb_5', which is identical to Hyb_5 except for how the commitment key $\mathsf{ck} = (H, Y, Z)$ is generated within $\mathsf{BCJ.CKSetup}$. In Hyb_5', Z is set to G^{x_2} for a uniformly random $x_2 \in \mathbb{Z}_p^*$. In Hyb_5, Z is sampled from $\mathbb{G}$ subject to $\log_G H \neq \log_Y Z$. The statistical distance between the distributions of ck in Hyb_5 and Hyb_5' is negligible, at most $2/p$. Let $E_{\mathsf{dl}}{}'$ be the corresponding event in Hyb_5'. It follows that $\Pr[E_{\mathsf{dl}}] \leqslant \Pr[E_{\mathsf{dl}}{}'] + 2/p$.

We now construct the DL reduction $\mathcal{B}_2$. Given a DL instance $P \in \mathbb{G}$ (where $P = G^x$ for $x \leftarrow \mathbb{Z}_p$), $\mathcal{B}_2$ simulates Hyb_5' for $\mathcal{A}$. If $P = 1$, which happens with probability $1/p$, $\mathcal{B}_2$ aborts. Otherwise, it proceeds to embed the challenge P into the commitment key ck. $\mathcal{B}_2$ flips a coin $b_{\mathsf{dl}} \in \{0, 1\}$.

- If $b_{\mathsf{dl}} = 0$, it sets $\mathsf{ck} = (H, Y, Z) := (G^{x_1}, P, G^{x_2})$ for random $x_1, x_2 \in \mathbb{Z}_p^*$.
- If $b_{\mathsf{dl}} = 1$, it sets $\mathsf{ck} = (H, Y, Z) := (G^{x_1}, G^{x_2}, P)$ for random $x_1, x_2 \in \mathbb{Z}_p^*$.

When the game does not abort, the distribution of ck in this simulation is statistically identical to that in Hyb_5'.

Suppose the event $E_{\mathsf{dl}}{}'$ occurs, and $\mathcal{B}_2$ extracts a witness $\mathbb{w} = (a, b, c)$ from a NIZK proof submitted by $\mathcal{A}$. The witness satisfies $G^a Y^b Z^c = 1$. $\mathcal{B}_2$ uses this witness to solve for $\log_G P$.

- If $b_{\mathsf{dl}} = 0$ and $b \neq 0$, the equation becomes $G^a P^b (G^{x_2})^c = 1$, which implies $P^b = G^{-(a+cx_2)}$. $\mathcal{B}_2$ computes and outputs $\log_G P = -b^{-1}(a + cx_2)$.
- If $b_{\mathsf{dl}} = 1$ and $c \neq 0$, the equation becomes $G^a (G^{x_2})^b P^c = 1$, which implies $P^c = G^{-(a+bx_2)}$. $\mathcal{B}_2$ computes and outputs $\log_G P = -c^{-1}(a + bx_2)$.

Since $\mathbb{w} \in \mathcal{R}_{\mathsf{dl}}$, it cannot be that $b = c = 0$ (as this would imply $G^a = 1$ and $a = 0$, contradicting that the witness is for a non-trivial relation). Thus, at least one of b or c must be non-zero. The reduction $\mathcal{B}_2$ fails only if its choice of b_{dl} does not align with the non-zero component of the witness (e.g., it chose $b_{\mathsf{dl}} = 0$ but the witness has $b = 0$). This occurs with probability at most $1/2$.

The reduction $\mathcal{B}_2$ successfully solves the DL problem if $E_{\mathsf{dl}}{}'$ occurs and it does not abort due to $P = 1$ or a mismatched b_{dl}. The success probability of $\mathcal{B}_2$ is therefore at least $\frac{1}{2}\Pr[E_{\mathsf{dl}}{}' \wedge P \neq 1] = \frac{1}{2}\Pr[E_{\mathsf{dl}}{}'](1 - 1/p)$. This gives us the bound: $\Pr[E_{\mathsf{dl}}{}'] \leqslant 2(1 - 1/p)^{-1}\mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}_2}(\lambda)$.

Combining all the bounds, we get the final difference between the hybrids:

$$|\epsilon_4 - \epsilon_5| \leqslant \Pr[E_{\mathsf{dl}}] \leqslant \Pr[E_{\mathsf{dl}}{}'] + \frac{2}{p} \leqslant \frac{2p}{p-1} \cdot \mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}_2}(\lambda) + \frac{2}{p}.$$

Before moving to the next hybrid, we state a useful lemma regarding the list Opened, containing the opening of commitments.

Lemma 6.3. *For any session $\widetilde{\mathsf{ctnt}}$, if $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ is not empty, it contains valid openings $(r_j, \mathbf{s}_j)$ for every commitment $\mathbf{C}_j$ (i.e. $\mathbf{C}_j = \mathsf{BCJ.Commit}(\mathsf{ck}, G^{r_j}; \mathbf{s}_j)$) from a signer $j \in \mathsf{SS}$ that is either corrupted or impersonated. Furthermore, once all honest signers in SS complete their second round for session $\widetilde{\mathsf{ctnt}}$, the list contains exactly the valid openings for all corrupted signers: $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] = \{(r_j, \mathbf{s}_j)\}_{j \in \mathsf{sCS}}$.*

Proof of Theorem 6.3. The list $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ is populated with openings $(r_j, \mathbf{s}_j)$ for all corrupted and impersonated signers in SS as part of $\mathcal{O}_{\mathsf{Sign}_2}$. The validity of these openings is guaranteed. When a signer i gets corrupted, its openings $(r_i, \mathbf{s}_i)$ also get added by $\mathcal{O}_{\mathsf{Corrupt}}$. For an impersonated honest signer j, the opening is the one honestly generated in $\mathcal{O}_{\mathsf{Sign}_1}$ and is valid by construction. For a corrupted signer j, the opening is extracted from its NIZK proof, and the game aborts in Hyb_5 unless the extracted witness is valid for $\mathbf{C}_j$ (i.e., satisfies $\mathcal{R}_{\mathsf{aux}}$). Thus, any entry added in $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ is a valid opening.

An honest signer i completes its second round for $\widetilde{\mathsf{ctnt}}$ only if all received messages are consistent with its view. Therefore, if all honest signers complete this round for $\widetilde{\mathsf{ctnt}}$, no honest signer $j \in \mathsf{sHS}$ could have been impersonated, that is $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ must be empty. As a result, $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ only contains openings corresponding to the set of corrupted signers sCS. This concludes the proof. □

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

7 : **parse** $(\mathbf{C}_j, \pi_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}$

8 : **parse** $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$

9 : **for** $j \in \mathsf{SS}$ **do**

10 : $\quad \mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j); \quad \mathbb{t}_j := j$

11 : $\quad$ **require** $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$

12 : $\widetilde{\mathsf{ctnt}} := 0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}$

13 : **if** $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \perp$ **then**

14 : $\quad \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$

15 : $\quad \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$

16 : $\quad \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$
$\quad\quad \leftarrow \{j \in \mathsf{sHS} | \mathsf{HonestMsg}[j, \mathbf{C}_j] \neq (\pi_j, \cdot, \cdot)\}$

17 : $\quad$ **for** $j \in \mathsf{sCS} \cup \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **do**

18 : $\quad\quad$ **if** $\mathsf{HonestMsg}[j, \mathbf{C}_j] = (\pi_j, \cdot, \cdot)$ **do**

19 : $\quad\quad\quad$ **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$

20 : $\quad\quad$ **else**

21 : $\quad\quad\quad \mathbb{w} \leftarrow \mathsf{ZKSim}(\mathsf{ext}, (\mathsf{ck}, \mathbf{C}_j), \pi_j, j)$

22 : $\quad\quad\quad$ **abort if** $((\mathsf{ck}, \mathbf{C}_j), \mathbb{w}) \notin \mathcal{R}_{\mathsf{aux}}$

23 : $\quad\quad\quad$ **parse** $(r_j, \mathbf{s}_j) \leftarrow \mathbb{w}$

24 : $\quad\quad \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$

25 : $\widetilde{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

26 : $\widetilde{\mathbf{s}}_i := \mathbf{s}_i + \widetilde{\Delta}_i$

27 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\mathbf{s}}_i$

28 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \backslash \{i\}$

29 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i)$

30 : $\mathsf{pm}_{2,i} := \widetilde{\mathbf{s}}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

Figure 13: The fifth hybrid. Differences from the previous hybrid are highlighted in blue.

Hyb_6. In this hybrid, we introduce a new abort condition to handle the unlikely event that the adversary guesses a secret seed shared between two honest parties. The game now aborts if the adversary $\mathcal{A}$ makes a query to the random oracle $\mathsf{H}_{\mathsf{mask}}$ that matches $\mathsf{seed}_{i,j} = i\|j\|\mathsf{rand}_{i,j}$ for any pair of honest signers $(i, j) \in \mathsf{HS}^2$. This change is formalized in Fig. 14 and will be useful in further hybrids to argue that zero shares remain properly hidden.

The difference in the adversary's advantage is bounded by the probability of this new abort event. The seeds $\mathsf{rand}_{i,j}$ are chosen uniformly at random from $\{0,1\}^\lambda$ and remain hidden from the adversary. Therefore, for any specific pair of honest signers (i, j), the probability that $\mathcal{A}$ guesses $\mathsf{rand}_{i,j}$ in any single query is $1/2^\lambda$. Let $Q_{\mathsf{H}_{\mathsf{mask}}}$ be the total number of queries to $\mathsf{H}_{\mathsf{mask}}$. By a union bound over all possible queries, the probability of the game aborting is negligible:

$$|\epsilon_5 - \epsilon_6| \leqslant \frac{Q_{\mathsf{H}_{\mathsf{mask}}}}{2^\lambda}.$$

$\mathsf{H}_{\mathsf{mask}}(\mathsf{seed}, \mathsf{inp})$

1 : **if** (Query from $\mathcal{A}$) $\wedge$ ($i \| j \| \mathsf{rand} \leftarrow \mathsf{seed}$ correctly parses) **then**
2 : **abort if** $(i,j) \in \mathsf{HS}^2 \wedge \mathsf{rand} = \mathsf{rand}_{i,j}$
3 : **if** $\mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}, \mathsf{inp}] = \bot$ **then**
4 : **if** (first entry of inp is 0) **then**
5 : $m \xleftarrow{\$} \mathbb{Z}_p^2$
6 : **else**
7 : $m \xleftarrow{\$} \mathbb{Z}_p$
8 : $\mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}, \mathsf{inp}] \leftarrow m$
9 : **return** $\mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}, \mathsf{inp}]$

Figure 14: The sixth hybrid. Differences from the previous hybrid are highlighted in blue.

Hyb_7**.** This hybrid modifies how the masked opening $\widetilde{\mathbf{s}}_i$ is generated in $\mathcal{O}_{\mathsf{Sign}_2}$, a crucial step towards simulation. While the previous hybrid computed $\widetilde{\mathbf{s}}_i$ honestly as $\mathbf{s}_i + \widetilde{\Delta}_i$, this hybrid generates it randomly for all but one honest signer in each session. The last honest signer computes its share to ensure consistency. This change is detailed in Fig. 15.

To show that this change is indistinguishable, we use a hybrid argument over the Q_{S} signing sessions. It is sufficient to show that for any single session, the adversary's view is statistically identical. We analyze the distribution of the masked openings and the consistency of the view upon corruption.

Distribution of Masked Openings ($\widetilde{\mathbf{s}}_i$). We analyze the distribution of $\widetilde{\mathbf{s}}_i$ in two cases for a given session $\widetilde{\mathsf{ctnt}}$.

Case 1: Signer i is not the last honest signer, i.e. $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \neq \{i\}$ when $\mathcal{O}_{\mathsf{Sign}_2}$ is called. In Hyb_6, the masked opening is computed as $\widetilde{\mathbf{s}}_i = \mathbf{s}_i + \widetilde{\Delta}_i$. The masking term $\widetilde{\Delta}_i = \sum_{j \in \mathsf{SS} \setminus \{i\}} (\tilde{m}_{i,j} - \tilde{m}_{j,i})$ (where $\tilde{m}_{i,j} = \mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{i,j}, \widetilde{\mathsf{ctnt}})$) acts as a one-time pad. Due to the abort conditions from previous hybrids, the adversary does not query $(\mathsf{seed}_{i,j}, \widetilde{\mathsf{ctnt}})$ for honest signers pair (i,j) (Hyb_6) and the random oracle input $\widetilde{\mathsf{ctnt}}$ is queried for the first time as honest commitments are never sampled twice (Hyb_2). Thus, the $\tilde{m}_{i,j}$ and $\tilde{m}_{j,i}$ for $j \in \mathsf{sHS} \setminus \{i\}$ are freshly sampled during the signing session and ensure that $\widetilde{\Delta}_i$ is uniformly random, which makes $\widetilde{\mathbf{s}}_i$ uniformly random. In Hyb_7, $\widetilde{\mathbf{s}}_i$ is explicitly sampled uniformly at random. Therefore, the distribution is identical in both hybrids.

Case 2: Signer i is the last honest signer. In Hyb_6, $\widetilde{\mathbf{s}}_i$ is determined by the other shares, since $\sum_{j \in \mathsf{SS}} \widetilde{\Delta}_j = 0$. This implies $\widetilde{\mathbf{s}}_i$ is a deterministic function of values already fixed from the adversary's perspective:

$$\widetilde{\mathbf{s}}_i = \sum_{j \in \mathsf{sHS}} \mathbf{s}_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \widetilde{\mathbf{s}}_j - \sum_{j \in \mathsf{sCS}} \widetilde{\Delta}_j. \tag{3}$$

In Hyb_7, the game computes $\widetilde{\mathbf{s}}_i$ for the last signer using this exact formula. Since all inputs are identically distributed in both hybrids, the resulting $\widetilde{\mathbf{s}}_i$ is as well.

Consistency upon Corruption. When a signer i is corrupted, the game must provide the adversary with all of its internal randomness, including the values used to compute the masking term $\widetilde{\Delta}_i$. In Hyb_7, if signer i has already participated in a session, its $\widetilde{\mathbf{s}}_i$ might have been generated randomly. The $\mathcal{O}_{\mathsf{Corrupt}}$ logic ensures consistency by retroactively computing what $\widetilde{\Delta}_i$ *must have been* (i.e., $\widetilde{\Delta}_i = \widetilde{\mathbf{s}}_i - \mathbf{s}_i$) and programming the random oracle $\mathsf{H}_{\mathsf{mask}}$ accordingly using the $\mathsf{ProgramZeroShare}$ subroutine. This ensures that the adversary's view is consistent with an honest execution.

Formally, since the distribution of $\widetilde{\mathbf{s}}_i$ is identical in both hybrids, it suffices to show that the conditional distribution of the underlying randomness programmed into the oracle is also identical under fixed $\widetilde{\mathbf{s}}_j$ for $j \in \mathsf{sHS}\backslash\{i\}$, and $\widetilde{\mathbf{s}}_i$ in the case of $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$. Also, without loss of generality, we fix masking terms associated with corrupted signers.

First, note that the modifications in Hyb_7 are not executed and do not affect the distribution of $\widetilde{\Delta}_i$ for honest signers i who have not yet participated in the session $\widetilde{\mathsf{ctnt}}$ (i.e., $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$) and are not the last signer (i.e., $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \neq \{i\}$). We thus assume we are not in this case.

Let us first show that the conditional distribution of $\widetilde{\Delta}_i$ is identical in both games. We distinguish the two possible cases remaining:

- **Signer i has participated in $\widetilde{\mathsf{ctnt}}$, but is not the last signer.** In this case, $\widetilde{\Delta}_i$ is uniquely determined from $\widetilde{\mathbf{s}}_i$ and $\mathbf{s}_i$ such that $\widetilde{\mathbf{s}}_i = \mathbf{s}_i + \widetilde{\Delta}_i$ holds in both games
- **Signer i is the last honest signer in $\widetilde{\mathsf{ctnt}}$.** In this case, in Hyb_6, since $\sum_{j\in\mathsf{SS}} \widetilde{\Delta}_j = 0$ holds, the last signer's $\widetilde{\Delta}_i$ satisfies

$$\begin{aligned}\widetilde{\Delta}_i &= -\sum_{j\in\mathsf{SS}\backslash\{i\}} \widetilde{\Delta}_j \\ &= -\sum_{j\in\mathsf{sHS}\backslash\{i\}} (\widetilde{\mathbf{s}}_j - \mathbf{s}_j) - \sum_{j\in\mathsf{sCS}} \widetilde{\Delta}_j \\ &= \sum_{j\in\mathsf{sHS}\backslash\{i\}} \mathbf{s}_i - \sum_{j\in\mathsf{sHS}\backslash\{i\}} \widetilde{\mathbf{s}}_j - \sum_{j\in\mathsf{sCS}} \widetilde{\Delta}_j, \end{aligned} \tag{4}$$

  where $\widetilde{\mathbf{s}}_j$ for $j \in \mathsf{sHS}\backslash\{i\}$ have already been revealed to $\mathcal{A}$ via the signing queries. Thus $\widetilde{\Delta}_i$ in Hyb_6 is uniquely determined from fixed values such that Eq. (4) holds. In Hyb_7, the challenger stores $\widetilde{\mathbf{s}}_j$ in $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$ for $\mathsf{sHS}\backslash\{i\}$ in the second round and sets $\sum_{j\in\mathsf{sHS}\backslash\{i\}} \mathbf{s}_i$ in $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}]$ in $\mathcal{O}_{\mathsf{Corrupt}}$. Thus $\widetilde{\Delta}_i$ in Hyb_7 is also uniquely determined from fixed values such that Eq. (4) holds. Therefore the conditional distributions of Δ_i are identical in both games.

Now, it remains to study the conditional distribution of $(\tilde{m}_{i,j}, \tilde{m}_{j,i})_{j\in\mathsf{sHS}\backslash\{i\}}$. In Hyb_6, $(\tilde{m}_{i,j}, \tilde{m}_{j,i})_{j\in\mathsf{sHS}}$ are distributed uniformly conditioned on $\widetilde{\Delta}_i = \sum_{j\in\mathsf{SS}\backslash\{i\}} (\tilde{m}_{i,j} - \tilde{m}_{j,i})$. In Hyb_7, $(\tilde{m}_{i,j}, \tilde{m}_{j,i})_{j\in\mathsf{sHS}\backslash\{i\}}$ are programmed with $\mathsf{ProgramZeroShare}$ exactly so as to as to revert this distribution from the knowledge of already corrupted masks and $\widetilde{\Delta}_i$. Indeed, it selects $a \in \mathsf{sHS}\backslash\{i\}$ and samples $\tilde{m}_{i,j}, \tilde{m}_{j,i}$ for $j \in \mathsf{sHS}\backslash\{i, a\}$ and $\tilde{m}_{a,i}$ uniformly at random, then sets $\tilde{m}_{i,a}$ to satisfy the equation for $\widetilde{\Delta}_i$.

In all cases, the conditional distribution of the values revealed upon corruption is identical. Combining these arguments, the adversary's view remains statistically identical between the two hybrids.

$$\epsilon_6 = \epsilon_7$$

Hyb_8. This hybrid introduces an explicit consistency check to enforce that the aggregated commitment randomness R used in the third round is consistent with the randomnesses r_j of individual signers extracted earlier. The game aborts if the adversary provides an inconsistent R. We will show that this aborts occurs with negligible probability as it corresponds to breaking the binding property of the commitment scheme. This change is formalized in Fig. 17.

The core idea is for the challenger to pre-compute the expected value of the aggregated randomness R and check if the adversary's value in $\mathcal{O}_{\mathsf{Sign}_3}$ matches. Specifically, the challenger now maintains two new lists, $\mathsf{SumComRnd}[\cdot]$ and $\mathsf{SumCom}[\cdot]$, to track the sum of randomnesses r_i for honest signers and the expected product of G^{r_i} for all signers, respectively.

- In $\mathcal{O}_{\mathsf{Sign}_2}$, when the last honest signer for a session $\widetilde{\mathsf{ctnt}}$ is queried, the challenger computes the total commitment randomness. It calculates $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] = \sum_{j\in\mathsf{sHS}} r_j$ and then $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] =$

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.
7 : **parse** $(\mathbf{C}_j, \pi_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}}$
8 : **parse** $(\mathbf{C}_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{st}_i$
9 : **for** $j \in \mathsf{SS}$ **do**
10 : $\mathbb{x}_j := (\mathsf{ck}, \mathbf{C}_j)$; $\mathbb{t}_j := j$
11 : **require** $\Pi_{\mathsf{aux}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{aux}}}(\mathbb{x}_j, \pi_j, \mathbb{t}_j) = 1$
12 : $\widetilde{\mathsf{ctnt}} := 0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}$
13 : **if** $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] = \bot$ **then**
14 : $\mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$
15 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$
16 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$
$\leftarrow \{j \in \mathsf{sHS} | \mathsf{HonestMsg}[j, \mathbf{C}_j] \neq (\pi_j, \cdot, \cdot)\}$
17 : **for** $j \in \mathsf{sCS} \cup \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **do**
18 : **if** $\mathsf{HonestMsg}[j, \mathbf{C}_j] = (\pi_j, \cdot, \cdot)$ **do**
19 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$
20 : **else**
21 : $\mathbb{w} \leftarrow \mathsf{ZKSim}(\mathsf{ext}, (\mathsf{ck}, \mathbf{C}_j), \pi_j, j)$
22 : **require** $((\mathsf{ck}, \mathbf{C}_j), \mathbb{w}) \in \mathcal{R}_{\mathsf{aux}}$
23 : **parse** $(r_j, \mathbf{s}_j) \leftarrow \mathbb{w}$
24 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$
25 : **if** $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \neq \{i\}$ **then**
26 : $\widetilde{\mathbf{s}}_i \xleftarrow{\$} \mathbb{Z}_p^2$
27 : **else** // the last signer in the second round with $\widetilde{\mathsf{ctnt}}$
28 : **for** $j \in \mathsf{sCS}$ **do**
29 : $\widetilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
30 : **for** $j \in \mathsf{sHS}$ **do**
31 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$
32 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}} \mathbf{s}_j$
33 : $\widetilde{\mathbf{s}}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \widetilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$
34 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\mathbf{s}}_i$
35 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$
36 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \widetilde{\mathbf{s}}_i)$
37 : $\mathsf{pm}_{2,i} := \widetilde{\mathbf{s}}_i$
// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

1 : **require** $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T-1]\!]$
2 : **for** $\mathbf{C}_i$ s.t. $\mathsf{HonestMsg}[i, \mathbf{C}_i] \neq \bot$ **do**
3 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
4 : **parse** $\mathsf{sid} \leftarrow \mathsf{CtntSid}[i, \mathbf{C}_i]$
5 : $\rho_i \leftarrow \mathsf{ZKSim}(\mathsf{rvl}, (\mathsf{ck}, \mathbf{C}_i), (r_i, \mathbf{s}_i), i, \pi)$
6 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$
7 : **for** $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$ **do**
8 : **parse** $0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}} \leftarrow \widetilde{\mathsf{ctnt}}$
9 : **if** $i \notin \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **then**
10 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
11 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$
12 : **if** $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**
13 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
14 : $\widetilde{\Delta}_i \leftarrow \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$
15 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\Delta}_i$
16 : **if** $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \neq \bot$ **then**
17 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$
18 : **if** $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**
19 : **for** $j \in \mathsf{sCS}$ **do**
20 : $\widetilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
21 : **for** $j \in \mathsf{sHS}\setminus\{i\}$ **do**
22 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$
23 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathbf{s}_j$
24 : $\widetilde{\Delta}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \widetilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$
25 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\Delta}_i$
26 : **if** $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**
27 : $\mathsf{ProgramZeroShare}(\widetilde{\mathsf{ctnt}}, i, \mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i], \mathsf{sCS}, \mathsf{sHS})$
28 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$
29 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$
30 : $\mathsf{HS} := \mathsf{HS}\setminus\{i\}$
31 : $\mathsf{CS} := \mathsf{CS} \cup \{i\}$
32 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

Figure 15: The seventh hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes lists $\mathsf{SumRnd}[\cdot]$, $\mathsf{MaskedRnd}[\cdot]$, $\mathsf{Mask}_{\mathbf{s}}[\cdot] := \bot$ at the beginning of the game.

ProgramZeroShare($\widetilde{\mathsf{ctnt}}, i, \Delta^*, \mathsf{sCS}, \mathsf{sHS}$):

require $\Delta^* \neq \bot \wedge |\mathsf{sHS}| > 1$
if $\Delta^* \in \mathbb{Z}_p$ **then**
  $t := 1$ // $\Delta^* = \mathsf{Mask}_s[\widetilde{\mathsf{ctnt}}, i]$
else
  $t := 2$ // $\Delta^* = \mathsf{Mask}_z[\mathsf{ctnt}, i]$
for $j \in \mathsf{sCS}$
  $m_{i,j} \leftarrow \mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{i,j}, \widetilde{\mathsf{ctnt}})$
  $m_{j,i} \leftarrow \mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{j,i}, \widetilde{\mathsf{ctnt}})$
pick a **from** $\mathsf{sHS}\backslash\{i\}$
for $j \in \mathsf{sHS}\backslash\{i, a\}$ **do**
  $m_{i,j} \xleftarrow{\$} \mathbb{Z}_p^t,\ \mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}_{i,j}, \widetilde{\mathsf{ctnt}}] \leftarrow m_{i,j}$
  $m_{j,i} \xleftarrow{\$} \mathbb{Z}_p^t,\ \mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}_{j,i}, \widetilde{\mathsf{ctnt}}] \leftarrow m_{j,i}$
$m_{a,i} \xleftarrow{\$} \mathbb{Z}_p^t,\ \mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}_{a,i}, \widetilde{\mathsf{ctnt}}] \leftarrow m_{a,i}$
$\mathsf{T}_{\mathsf{H}_{\mathsf{mask}}}[\mathsf{seed}_{i,a}, \widetilde{\mathsf{ctnt}}] \leftarrow \Delta^* - \sum_{j \in \mathsf{SS}\backslash\{i,a\}} (m_{i,j} - m_{j,i}) + m_{a,i}$

Figure 16: A subroutine for programming the random oracle $\mathsf{H}_{\mathsf{mask}}$ to open zero shares $\widetilde{\Delta}_i \in \mathbb{Z}_p^2$ and $\Delta_i \in \mathbb{Z}_p$ consistently. This is the similar algorithm used in [KRT24].

$G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot \prod_{j \in \mathsf{sCS}} G^{r_j}$. Note that the values r_j for honest and corrupted signers are known to the challenger from $\mathsf{HonestMsg}$ and Opened, respectively, due to Theorem 6.3.

- In $\mathcal{O}_{\mathsf{Sign}_3}$, a new check is added: the game aborts if the R opened does not match the pre-computed $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$.
- In $\mathcal{O}_{\mathsf{Corrupt}}$, if the signer i is corrupted, for any $\widetilde{\mathsf{ctnt}}$ such that all honest signers have completed the second round $\widetilde{\mathsf{ctnt}}$, the challenger removes r_i from $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ to keep the list consistent. Additionally, for any $\widetilde{\mathsf{ctnt}}$ such that i is the last remaining signer, the challenger computes $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ and $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$ as in $\mathcal{O}_{\mathsf{Sign}_2}$ regarding i as a corrupted signer.

The difference in the adversary's advantage between Hyb_7 and Hyb_8 is bounded by the probability of the game aborting. We show that this abort event can be used to construct an adversary $\mathcal{B}_3$ that breaks the binding property of the commitment scheme BCJ.

Reduction to Binding. The reduction adversary $\mathcal{B}_3$ receives a commitment key ck from the binding experiment, and runs the adversary $\mathcal{A}$ while simulating its view in Hyb_8. When the game aborts in $\mathcal{O}_{\mathsf{Sign}_3}$ because $R \neq \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$, $\mathcal{B}_3$ uses the information from the game to extract two valid openings for the same commitment, thus breaking binding.

The extraction process depends on whether all honest signers have completed the second round.

Case 1: $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \neq \bot$**.** This case occurs when the last honest signer has completed the second round, and thus the challenger has computed the expected aggregated randomness $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$. When the game aborts, we have $R \neq \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$. $\mathcal{B}_3$ can construct two valid openings for the aggregated commitment $\mathbf{C} = \prod_{j \in \mathsf{SS}} \mathbf{C}_j$:

1. The first opening is $(R, \mathbf{s})$, which is provided by the adversary $\mathcal{A}$ in its call to $\mathcal{O}_{\mathsf{Sign}_3}$. This opening is verified as valid within the oracle before the abort check.

2. The second opening is $(\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}], \mathbf{s}')$. $\mathcal{B}_3$ computes this opening itself. It parses the individual openings $(r_j, \mathbf{s}_j)$ for all corrupted signers $j \in \mathsf{sCS}$ from the list $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$. It then calculates the total opening value as $\mathbf{s}' = \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] + \sum_{j \in \mathsf{sCS}} \mathbf{s}_j$.

Due to the homomorphic property of the commitment scheme, $(\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}], \mathbf{s}')$ is a valid opening of $\mathbf{C}$. Since $R \neq \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$, $\mathcal{B}_3$ has found two different openings for the same commitment $\mathbf{C}$ and thus breaks the binding property.

Case 2: $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] = \bot$**.** This case occurs if not all honest signers have completed the second round. Here, the challenger has not yet computed the final value for $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$. If the game aborts, it must be because the adversary provided an inconsistent partial commitment for a single signer i. $\mathcal{B}_3$ proceeds as follows:

1. It collects all available individual openings $(r_j, \mathbf{s}_j)$ for every signer $j \in \mathsf{SS} \backslash \{i\}$. These are available from $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$ for corrupted/impersonated signers due to Theorem 6.3 and from $\mathsf{HonestMsg}$ for honest signers. Note that $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \neq \bot$ since at least user i has completed the second round.

2. Using the aggregated values $(R, \mathbf{s})$ from $\mathcal{A}$'s query, $\mathcal{B}_3$ isolates the components corresponding to signer i's commitment $\mathbf{C}_i$:

$$R_i' \leftarrow R / \prod_{j \in \mathsf{SS} \backslash \{i\}} G^{r_j}$$
$$\mathbf{s}_i' \leftarrow \mathbf{s} - \sum_{j \in \mathsf{SS} \backslash \{i\}} \mathbf{s}_j$$

   Due to the homomorphism of the commitment, $(R_i', \mathbf{s}_i')$ constitutes a valid opening for $\mathbf{C}_i$.

3. $\mathcal{B}_3$ also knows the original, honestly generated opening $(G^{r_i}, \mathbf{s}_i)$ for $\mathbf{C}_i$ from the list $\mathsf{HonestMsg}$.

If $R_i' \neq G^{r_i}$, $\mathcal{B}_3$ has found two openings for $\mathbf{C}_i$ and breaks binding.

Probability Analysis. Let E be the event that the game aborts, and $\mathsf{E}_{\mathsf{abort}}$ be the event that $R_i = G^{r_i}$ holds in the case of $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] = \bot$. Then we have

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_3, \mathsf{BCJ}}(\lambda) = \Pr[\mathsf{E} \wedge \neg \mathsf{E}_{\mathsf{abort}}] \geqslant \Pr[\mathsf{E}] - \Pr[\mathsf{E}_{\mathsf{abort}}].$$

In the case where $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] = \bot$, at least one honest signer has not participated in the second round. Furthermore, the values $(r_i, \mathbf{s}_i)$ for the target signer i are perfectly hidden from $\mathcal{A}$ due to the changes in previous hybrids and the perfectly hiding property of the commitment scheme. Therefore, the adversary $\mathcal{A}$ can only guess the correct value of G^{r_i} with probability $1/p$. The event $R_i' = G^{r_i}$ happens with probability at most $1/p$. Therefore, we have $\Pr[\mathsf{E}] \leqslant \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_3, \mathsf{BCJ}}(\lambda) + 1/p$.

Combining all the arguments, we conclude:

$$|\epsilon_7 - \epsilon_8| \leqslant \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_3, \mathsf{BCJ}}(\lambda) + \frac{1}{p}.$$

Hyb_9**.** In this hybrid, we modify the challenger's behavior to program the random oracle H_c. Instead of computing the challenge c as the output of the oracle, the challenger now chooses c in advance and forces the oracle to be consistent with this choice. This is a standard technique in proofs using the random oracle model and is a preparatory step for the final reduction. The changes are formalized in Fig. 18.

Specifically, the modifications are as follows:

- In $\mathcal{O}_{\mathsf{Sign}_2}$, when handling the query for the last honest signer of a session $\widetilde{\mathsf{ctnt}}$, the challenger now first samples a challenge $c \xleftarrow{\$} \mathbb{Z}_p$ uniformly at random.

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-26 in Hyb_7 in Fig. 15.

27 : **else** // the last signer in the second round with $\widetilde{\mathsf{ctnt}}$

28 : **for** $j \in \mathsf{sCS}$ **do**

29 : $\tilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

30 : **for** $j \in \mathsf{sHS}$ **do**

31 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$

32 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}} \mathbf{s}_j$

33 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}} r_j$

34 : **parse** $(r_j, \mathbf{s}_j)_{j\in\mathsf{sCS}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$

35 : $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot \prod_{j\in\mathsf{sCS}} G^{r_j}$

36 : $\tilde{\mathbf{s}}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \tilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$

37 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\mathbf{s}}_i$

38 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$

39 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \tilde{\mathbf{s}}_i)$

40 : $\mathsf{pm}_{2,i} := \tilde{\mathbf{s}}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Sign}_3}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

7 : **parse** $(\tilde{\mathbf{s}}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}}$

8 : **parse** $(\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}, r_i, \tilde{\mathbf{s}}_i) \leftarrow \mathsf{st}_i$

9 : $\mathbf{s} := \sum_{j\in\mathsf{SS}} \tilde{\mathbf{s}}_j$; $\mathbf{C} := \prod_{j\in\mathsf{SS}} \mathbf{C}_j$

10 : $R := \mathsf{RcvMsg}(\mathsf{ck}, \mathbf{C}, \mathbf{s})$

11 : **require** $\mathbf{C} = \mathsf{BCJ.Commit}(\mathsf{ck}, R; \mathbf{s})$

12 : $\widetilde{\mathsf{ctnt}} := 0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}$

13 : **abort if** $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \neq R$

14 : $c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$

15 : $\mathsf{ctnt} := 1\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}\|R$

16 : $\Delta_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt})$

17 : $\tilde{z}_i := c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i$

18 : $\mathsf{st}_i \leftarrow \widetilde{\mathsf{ctnt}}$ // $\widetilde{\mathsf{ctnt}} = 0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}}$

19 : $\mathsf{pm}_{3,i} := \tilde{z}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

// Identical to lines 1-6 in Hyb_7 in Fig. 15.

7 : **for** $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$ **do**

8 : **parse** $0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}} \leftarrow \widetilde{\mathsf{ctnt}}$

9 : **if** $i \notin \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **then**

10 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$

11 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$

12 : **if** $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**

13 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$

14 : $\tilde{\Delta} \leftarrow \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$

15 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\Delta}_i$

16 : **if** $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \neq \bot$ **then**

17 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$

18 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - r_i$

19 : **if** $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**

20 : **for** $j \in \mathsf{sCS}$ **do**

21 : $\tilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

22 : **for** $j \in \mathsf{sHS}\setminus\{i\}$ **do**

23 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$

24 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathbf{s}_j$

25 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j\in\mathsf{sHS}\setminus\{i\}} r_j$

26 : **parse** $(r_j, \mathbf{s}_j)_{j\in\mathsf{sCS}\cup\{i\}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$

27 : $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot \prod_{j\in\mathsf{sCS}\cup\{i\}} G^{r_j}$

28 : $\tilde{\Delta}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \tilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$

29 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\Delta}_i$

30 : **if** $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**

31 : $\mathsf{ProgramZeroShare}(\widetilde{\mathsf{ctnt}}, i, \mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i], \mathsf{sCS}, \mathsf{sHS})$

32 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$

33 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]\setminus\{i\}$

34 : $\mathsf{HS} := \mathsf{HS}\setminus\{i\}$

35 : $\mathsf{CS} := \mathsf{CS} \cup \{i\}$

36 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

Figure 17: The eighth hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes empty lists $\mathsf{SumComRnd}[\cdot], \mathsf{SumCom}[\cdot] := \bot$ at the beginning of the game.

- It then uses this chosen c to compute the values for $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ and $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$, embedding the secret key x in the process. The updated values are:

$$\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow c \cdot x + \sum_{j \in \mathsf{sHS}} r_j$$
$$\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot X^{-c} \cdot \prod_{j \in \mathsf{sCS}} G^{r_j}$$

  Note that the resulting $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$ is still equal to $\prod_{j \in \mathsf{SS}} G^{r_j}$, preserving consistency with the previous hybrid.

- Finally, the challenger programs the random oracle H_c via the subroutine $\mathsf{ProgramHashChall}$. This ensures that any future query to H_c with the inputs $(\mathbf{s}, \mathsf{M}, \prod_{j \in \mathsf{SS}} G^{r_j})$ corresponding to this session will return the pre-selected value of c. The game aborts if this oracle entry has already been defined, which would indicate a hash collision.

The same change is applied in $\mathcal{O}_{\mathsf{Corrupt}}$ when signer i is the last honest signer that has not yet complete the second round for a session $\widetilde{\mathsf{ctnt}}$, regarding signer i as a corrupted signer.

Indistinguishability. The adversary's view remains identical unless the game aborts due to a hash collision during the programming step. We argue that the probability of such an abort is negligible. Due to the modifications in Hyb_7 (individual openings are random until the last honest signer executes round 2) and the perfect hiding property of the commitment scheme, the individual randomness r_i of an honest signer is perfectly hidden from the adversary at the moment of programming. Since each r_i is chosen uniformly from $\mathbb{Z}_p$, the aggregated value $\prod_{j \in \mathsf{SS}} G^{r_j}$ is uniformly distributed from the adversary's perspective and has a min-entropy at least p. Therefore, the probability that the input to H_c for a new signing session collides with any previously defined value is at most $(Q_{\mathsf{H}_c} + Q_{\mathsf{S}})/p$. The difference in advantage is bounded by this collision probability.

$$|\epsilon_8 - \epsilon_9| \leqslant \frac{Q_{\mathsf{S}}(Q_{\mathsf{H}_c} + Q_{\mathsf{S}})}{p}.$$

Hyb_{10}**.** In this hybrid, we introduce several bookkeeping lists (as in Hyb_3) $\mathsf{InitializeSign}$, $\mathsf{UnSignedHS}$, $\mathsf{SignContent}$, $\mathsf{MaskedResp}$, and Chall to keep track of the outputs of the honest signers in the third round of the protocol. This will be useful to show that the honest signers responses $\widetilde{z}_i$ appear uniform to the adversary until the last honest signer executes the final round. This is formalized in Fig. 19. We provide a description and the purpose of each list below.

- $\mathsf{SS} := \mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}]$. This list stores the signer set SS included in the signing context $\widetilde{\mathsf{ctnt}}$ with which an honest signer executes the final round.

- $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \subseteq \mathsf{sHS}$. This list maintains the set of *honest* signers that did not execute the final round with $\widetilde{\mathsf{ctnt}}$ yet. This will be used to determine the last honest signer in the final round, who will be in charge of simulating the last protocol message of the final round.

- $\mathsf{ctnt} := \mathsf{SignContent}[\widetilde{\mathsf{ctnt}}]$. This list stores the signing context ctnt in the final round corresponding to $\widetilde{\mathsf{ctnt}}$. Looking ahead, this will be useful for programming $\mathsf{H}_{\mathsf{mask}}$. Due to the abort condition added in Hyb_8, ctnt is unique to $\widetilde{\mathsf{ctnt}}$.

- $\widetilde{z}_i := \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i]$. This list stores the masked response $\widetilde{z}_i$ of signer i under ctnt, used in its third protocol message. This will be used to remove the dependency of $\widetilde{z}_i$ on the opening $\widetilde{z}_i$ and keep track of its replacement – a fresh uniform sample, until the last honest signer executes the final round.

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j \in \mathsf{SS}})$

// Identical to lines 1-26 in Hyb_7 in Fig. 15.

27 : **else** // the last signer in the second round with $\widetilde{\mathsf{ctnt}}$

28 : **for** $j \in \mathsf{sCS}$ **do**

29 : $\tilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

30 : **for** $j \in \mathsf{sHS}$ **do**

31 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$

32 : $c \xleftarrow{\$} \mathbb{Z}_p$

33 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j \in \mathsf{sHS}} \mathbf{s}_j$

34 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow c \cdot x + \sum_{j \in \mathsf{sHS}} r_i$

35 : **parse** $(r_j, \mathbf{s}_j)_{j \in \mathsf{sCS}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$

36 : $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot X^{-c} \cdot \prod_{j \in \mathsf{sCS}} G^{r_j}$

37 : $\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$

38 : $\tilde{\mathbf{s}}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j \in \mathsf{sCS}} \tilde{\Delta}_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$

39 : $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\mathbf{s}}_i$

40 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

41 : $\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j \in \mathsf{SS}}, r_i, \tilde{\mathbf{s}}_i)$

42 : $\mathsf{pm}_{2,i} := \tilde{\mathbf{s}}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$

1 : **abort if** $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]] \neq \bot$

2 : $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]] \leftarrow c$

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

// Identical to lines 1-18 in Hyb_8 in Fig. 17.

19 : **if** $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**

20 : **for** $j \in \mathsf{sCS}$ **do**

21 : $\tilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

22 : **for** $j \in \mathsf{sHS} \setminus \{i\}$ **do**

23 : **parse** $(\pi_j, r_j, \mathbf{s}_j) \leftarrow \mathsf{HonestMsg}[j, \mathbf{C}_j]$

24 : $c \xleftarrow{\$} \mathbb{Z}_p$

25 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathbf{s}_j$

26 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow c \cdot x + \sum_{j \in \mathsf{sHS} \setminus \{i\}} r_i$

27 : **parse** $(r_j, \mathbf{s}_j)_{j \in \mathsf{sCS} \cup \{i\}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$

28 : $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot X^{-c} \cdot \prod_{j \in \mathsf{sCS} \cup \{i\}} G^{r_j}$

29 : $\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$

30 : $\tilde{\Delta}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j \in \mathsf{sCS}} \tilde{\Delta}_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$

31 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\Delta}_i$

32 : **if** $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**

33 : $\mathsf{ProgramZeroShare}(\widetilde{\mathsf{ctnt}}, i, \mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i], \mathsf{sCS}, \mathsf{sHS})$

34 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

35 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

36 : $\mathsf{HS} := \mathsf{HS} \setminus \{i\}$

37 : $\mathsf{CS} := \mathsf{CS} \cup \{i\}$

38 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

Figure 18: The nineth hybrid. Differences from the previous hybrid are highlighted in blue.

- $c := \mathsf{Chall}[\widetilde{\mathsf{ctnt}}]$. This list maintains the challenge c for the signing session with $\widetilde{\mathsf{ctnt}}$. Looking ahead, this will be used to generate the last signer's $\tilde{z}_i$ consistently later.

Concretely, in $\mathcal{O}_{\mathsf{Sign}_3}$, if signer i is the first honest signer in the final round with $\widetilde{\mathsf{ctnt}}$ (i.e., if $\mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}] = \bot$), it sets $\mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SS}$, $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{sHS}$, $\mathsf{SignContent}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ctnt}$, and $\mathsf{Chall}[\widetilde{\mathsf{ctnt}}] \leftarrow c$. Also, each signer executing the final round stores their $\tilde{z}_i$ in $\mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i]$ and removes i from $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}]$. In $\mathcal{O}_{\mathsf{Corrupt}}$, we remove i from $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}]$ if $i \in \mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}]$.

All the modifications made in this hybrid are conceptual, and thus the view of $\mathcal{A}$ remains identical:

$$\epsilon_9 = \epsilon_{10}.$$

The following helper lemma establishes a key property that will be used in the rest of the proof. It guarantees that the third round of the signing protocol for a given session $\widetilde{\mathsf{ctnt}}$ only proceeds after all honest signers have successfully completed the second round.

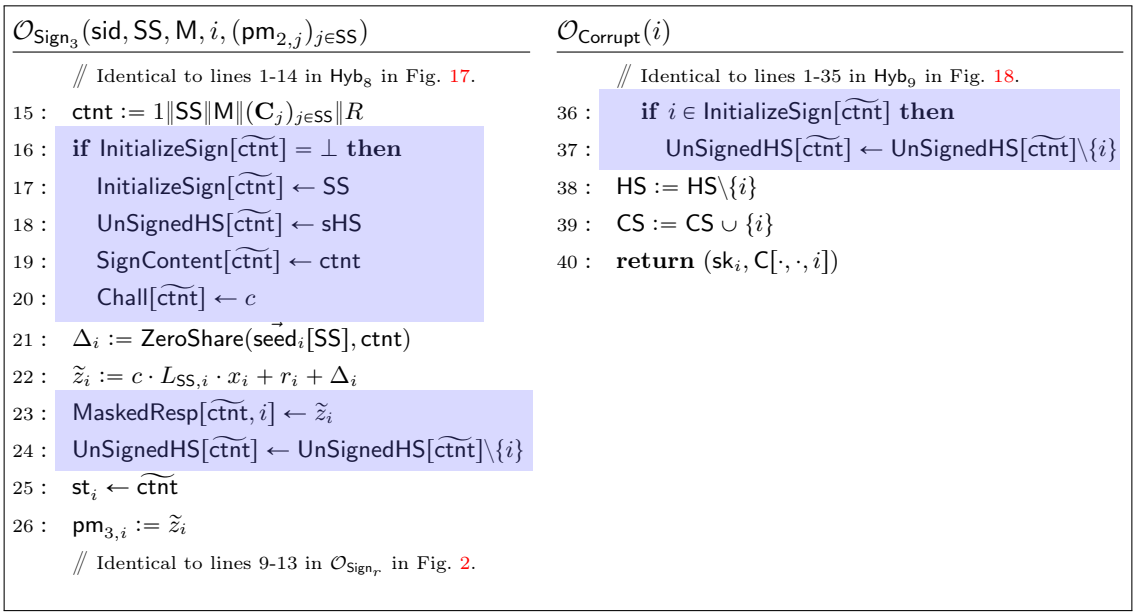

Figure 19: The tenth hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes empty lists $\mathsf{InitializeSign}[\cdot], \mathsf{UnSignedHS}[\cdot], \mathsf{SignContent}[\cdot], \mathsf{MaskedResp}[\cdot], \mathsf{Chall}[\cdot] := \perp$ at the beginning of the game.

Lemma 6.4. *If* $\mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}]$ *is set for a session* $\widetilde{\mathsf{ctnt}}$*, then all honest signers in that session have completed the second round.*

Proof of Theorem 6.4. The list $\mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}]$ is set at the beginning of $\mathcal{O}_{\mathsf{Sign}_3}$. For the game to reach this point without aborting, the check $R = \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$ introduced in Hyb_8 must have passed. This implies that $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$ cannot be $\perp$.

By definition, the value of $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]$ is only computed and set once the last honest signer for the session $\widetilde{\mathsf{ctnt}}$ has either completed the second round (in $\mathcal{O}_{\mathsf{Sign}_2}$) or been corrupted (in $\mathcal{O}_{\mathsf{Corrupt}}$). Therefore, if $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \neq \perp$, it necessarily means that all honest signers have finished their second-round execution for that session. □

Hyb_{11}. This hybrid mirrors the logic of Hyb_7, but applies it to the final round of the signing protocol. We change how the masked response $\widetilde{z}_i$ is generated in $\mathcal{O}_{\mathsf{Sign}_3}$. For all but the last honest signer in a session, $\widetilde{z}_i$ is now sampled uniformly at random, effectively decoupling it from the signer's secret values. The last signer's response is then computed deterministically to ensure the final aggregated signature remains valid. This change is detailed in Fig. 20.

Modifications. The changes are in $\mathcal{O}_{\mathsf{Sign}_3}$ and $\mathcal{O}_{\mathsf{Corrupt}}$:

- **In $\mathcal{O}_{\mathsf{Sign}_3}$:** If signer i is not the last honest signer for the session $\widetilde{\mathsf{ctnt}}$ (i.e., $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \neq \{i\}$), its response $\widetilde{z}_i$ is now chosen uniformly at random from $\mathbb{Z}_p$. If it is the last signer, $\widetilde{z}_i$ is computed consistently using the values from other signers, including the previously stored random responses from other honest signers in $\mathsf{MaskedResp}$: $\widetilde{z}_i := \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - c \cdot \sum_{j\in\mathsf{CS}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j\in\mathsf{CS}} \Delta_j - \sum_{j\in\mathsf{sHS}\backslash\{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]$. This relies on the value $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ computed in Hyb_9, which is guaranteed to be available by Theorem 6.4.
- **In $\mathcal{O}_{\mathsf{Corrupt}}$:** When a signer i is corrupted, we must ensure its internal state is consistent with the simulated (random) values. If i has already participated in the third round for a session $\widetilde{\mathsf{ctnt}}$, its

zero-share Δ_i is retroactively computed as $\mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i] - c \cdot L_{\mathsf{SS},i} \cdot x_i - r_i$. If i was the last honest signer that did not complete a session, its Δ_i is computed deterministically based on the shares of the other participants as $\Delta_i := \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - c \cdot \sum_{j \in \mathsf{sCS} \cup \{i\}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j \in \mathsf{sCS} \cup \{i\}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]$. In both cases, the computed Δ_i is stored in $\mathsf{Mask}_z[\widetilde{\mathsf{ctnt}}, i]$ and the random oracle $\mathsf{H}_{\mathsf{mask}}$ is programmed via $\mathsf{ProgramZeroShare}$ to be consistent with this value.

Indistinguishability. The adversary's view remains statistically identical. The argument follows the same structure as in Hyb_7. We show that for any given session $\widetilde{\mathsf{ctnt}}$, the distribution of all revealed values is unchanged.

Distribution of $\widetilde{z}_i$ in $\mathcal{O}_{\mathsf{Sign}_3}$:

- **Not the last signer:** In Hyb_{10}, $\widetilde{z}_i = c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i$. The zero-share Δ_i depends on oracle outputs $\mathsf{H}_{\mathsf{mask}}(\mathsf{seed}_{i,j}, \mathsf{ctnt})$ that have not been queried before (due to aborts in Hyb_2 and Hyb_6). This makes Δ_i a perfect one-time pad, so $\widetilde{z}_i$ is uniformly random. In Hyb_{11}, $\widetilde{z}_i$ is explicitly sampled uniformly. The distributions are identical.
- **The last signer:** In Hyb_{10}, since $\sum_{j \in \mathsf{SS}} \Delta_i = 0$ holds, the $\widetilde{z}_i$ of the last honest signer satisfies

$$\begin{aligned}
\widetilde{z}_i &= c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i \\
&= c \cdot \left(x - \sum_{j \in \mathsf{SS} \setminus \{i\}} L_{\mathsf{SS},j} \cdot x_j \right) + r_i - \sum_{j \in \mathsf{SS} \setminus \{i\}} \Delta_j \\
&= c \cdot x + \sum_{j \in \mathsf{sHS}} r_i - c \cdot \sum_{j \in \mathsf{sCS}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j \in \mathsf{sCS}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} (c \cdot L_{\mathsf{SS},j} \cdot x_j + \Delta_j + r_j) \\
&= c \cdot x + \sum_{j \in \mathsf{sHS}} r_i - c \cdot \sum_{j \in \mathsf{sCS}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j \in \mathsf{sCS}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]. \qquad (5)
\end{aligned}$$

  where the final equality follows from the fact that all honest signers use the same c in the signing session with $\widetilde{\mathsf{ctnt}}$ such that $c = \mathsf{Chall}[\widetilde{\mathsf{ctnt}}]$ and c is programmed via $\mathsf{ProgramHashChall}$ due to the abort condition added in Hyb_8. Indeed, $\mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]$ have been set to $\widetilde{z}_j$ for all $j \in \mathsf{sHS} \setminus \{i\}$ during their respective signing queries, since i is the last signer. Additionally, due to the modification made in Hyb_9 and Theorem 6.4, $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] = c \cdot x + \sum_{j \in \mathsf{sHS}} r_i$. In Hyb_{11}, the challenger computes $\widetilde{z}_i$ using this exact same function, leveraging the value $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ stored in Hyb_9. Since the inputs to the function are identically distributed, the output is as well.

Distribution of Δ_i in $\mathcal{O}_{\mathsf{Corrupt}}$: The logic is analogous. When a signer i is corrupted, its share Δ_i must be revealed.

- If i had already computed its response $\widetilde{z}_i$, then Δ_i is uniquely determined in both hybrids by the relation $\widetilde{z}_i = c \cdot L_{\mathsf{SS},i} \cdot x_i + r_i + \Delta_i$.
- If i was the last honest signer in a session, its Δ_i is fixed by the $\sum \Delta_j = 0$ constraint:

$$\begin{aligned}
\Delta_i &= - \sum_{j \in \mathsf{SS} \setminus \{i\}} \Delta_i \\
&= - \sum_{j \in \mathsf{sCS}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} (c \cdot L_{\mathsf{SS},j} \cdot x_j + \Delta_j + r_j) + \sum_{j \in \mathsf{sHS} \setminus \{i\}} (c \cdot L_{\mathsf{SS},j} \cdot x_j + r_j) \\
&= \sum_{j \in \mathsf{sHS} \setminus \{i\}} c \cdot L_{\mathsf{SS},j} \cdot x_j + \sum_{j \in \mathsf{sHS} \setminus \{i\}} r_j - \sum_{j \in \mathsf{sCS}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j] \\
&= c \cdot (x - \sum_{j \in \mathsf{sCS} \cup \{i\}} L_{\mathsf{SS},j} \cdot x_j) + \sum_{j \in \mathsf{sHS} \setminus \{i\}} r_j - \sum_{j \in \mathsf{sCS}} \Delta_j - \sum_{j \in \mathsf{sHS} \setminus \{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]
\end{aligned}$$

$\mathcal{O}_{\mathsf{Sign}_3}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-20 in Hyb_{10} in Fig. 19.

21 : **if** $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \neq \{i\}$ **then**

22 : $\quad \tilde{z}_i \xleftarrow{\$} \mathbb{Z}_p$

23 : **else** // the last signer in the third round with $\widetilde{\mathsf{ctnt}}$

24 : $\quad$ **for** $j \in \mathsf{sCS}$ **do**

25 : $\quad\quad \Delta_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \mathsf{ctnt})$

26 : $\quad \tilde{z}_i := \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - c \cdot \sum_{j\in\mathsf{sCS}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j\in\mathsf{sCS}} \Delta_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]$

27 : $\mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{z}_i$

28 : $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

29 : $\mathsf{st}_i \leftarrow \widetilde{\mathsf{ctnt}}$

30 : $\mathsf{pm}_{3,i} := \tilde{z}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

// Identical to lines 1-35 in Hyb_9 in Fig. 18.

36 : **if** $i \in \mathsf{InitializeSign}[\widetilde{\mathsf{ctnt}}]$ **then**

37 : $\quad$ **parse** $c \leftarrow \mathsf{Chall}[\widetilde{\mathsf{ctnt}}]$

38 : $\quad$ **if** $\mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**

39 : $\quad\quad$ **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$

40 : $\quad\quad \Delta_i \leftarrow \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, i] - c \cdot L_{\mathsf{SS},i} \cdot x_i - r_i$

41 : $\quad\quad \mathsf{Mask}_z[\widetilde{\mathsf{ctnt}}, i] \leftarrow \Delta_i$

42 : $\quad\quad$ **if** $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**

43 : $\quad\quad\quad$ **for** $j \in \mathsf{sCS}$ **do**

44 : $\quad\quad\quad\quad \Delta_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \mathsf{ctnt})$

45 : $\quad\quad\quad \Delta_i := \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - c \cdot \sum_{j\in\mathsf{sCS}\cup\{i\}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j\in\mathsf{sCS}} \Delta_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]$

46 : $\quad\quad\quad \mathsf{Mask}_z[\widetilde{\mathsf{ctnt}}, i] \leftarrow \Delta_i$

47 : $\quad\quad\quad$ **if** $\mathsf{Mask}_z[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**

48 : $\quad\quad\quad\quad$ **parse** $\mathsf{ctnt} \leftarrow \mathsf{SignContent}[\widetilde{\mathsf{ctnt}}]$

49 : $\quad\quad\quad\quad \mathsf{ProgramZeroShare}(\mathsf{ctnt}, i, \mathsf{Mask}_z[\widetilde{\mathsf{ctnt}}, i], \mathsf{sCS}, \mathsf{sHS})$

50 : $\quad\quad\quad \mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

51 : $\mathsf{HS} := \mathsf{HS} \setminus \{i\}$

52 : $\mathsf{CS} := \mathsf{CS} \cup \{i\}$

53 : **return** $(\mathsf{sk}_i, \mathsf{C}[\cdot, \cdot, i])$

Figure 20: The eleventh hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes an empty list $\mathsf{Mask}_z[\cdot] := \bot$ at the beginning of the game.

$$= c \cdot x + \sum_{j\in\mathsf{sHS}\setminus\{i\}} r_j - c \cdot \sum_{j\in\mathsf{sCS}\cup\{i\}} L_{\mathsf{SS},j} \cdot x_j - \sum_{j\in\mathsf{sCS}} \Delta_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedResp}[\widetilde{\mathsf{ctnt}}, j]. \tag{6}$$

where the third equality follows the same argument with Eq. (5). Again, we can show (see Eq. (6)) that the formula used to compute it in Hyb_{11} is equivalent to its implicit definition in Hyb_{10}. The challenger has all necessary values, including the modified $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] = c \cdot x + \sum_{j\in\mathsf{sHS}\setminus\{i\}} r_j$ from which r_i was removed in Hyb_8 before computing Δ_i.

Since the distributions of all outputs in $\mathcal{O}_{\mathsf{Sign}_3}$ and $\mathcal{O}_{\mathsf{Corrupt}}$ are identical, and the programming of $\mathsf{H}_{\mathsf{mask}}$ ensures the underlying randomness is consistent, the adversary's view does not change. Therefore:

$$\epsilon_{10} = \epsilon_{11}.$$

Hyb_{12}. This hybrid forms the core of our simulation strategy. We now leverage the restricted product equivocability of the commitment scheme BCJ. Instead of generating commitments honestly, the challenger now simulates them using the CSim algorithm. This allows the challenger to remove the dependencies on the secret key by equivocating the product of the commitments for all honest signers to a simulated R in the second round, and to equivocate individual commitments on corruptions to release their openings. The changes are detailed in Figs. 21 and 22.

Concretely, we introduce the following modifications:

- **Setup:** The commitment key ck is now generated using $\mathsf{EqvSetup}$ instead of $\mathsf{CKSetup}$. The challenger also initializes counters $\mathsf{ctr}_i \leftarrow 1$ for each signer to keep track of commitments, as required by the commitment restricted product equivocability game.
- **In $\mathcal{O}_{\mathsf{Sign}_1}$:** Honest commitments $\mathbf{C}_i$ are now generated via $\mathsf{CSim}(\mathsf{cmt}, \mathsf{td}, i, \mathsf{ctr}_i)$, and we set $\mathsf{HonestMsg}[i, \mathbf{C}_i] := (\pi_i, \bot, \bot)$. Importantly, the signing randomness r_i and opening $\mathbf{s}_i$ are no longer generated in this round. The counter ctr_i is incremented after each commitment generation and stored in $\mathsf{ComCtr}[i, \mathbf{C}_i]$.
- **In $\mathcal{O}_{\mathsf{Sign}_2}$ (last signer):** When the last honest signer for a session $\widetilde{\mathsf{ctnt}}$ is queried, instead of computing the aggregated randomness from individual values, the challenger equivocates the product of all honest signers' commitments for that session: it calls $\mathsf{CSim}(\mathsf{prod\text{-}eqv}, \mathsf{td}, c, \mathsf{sHS}, (j, k_j)_{j \in \mathsf{sHS}})$, where $k_j = \mathsf{ComCtr}[j, \mathbf{C}_j]$. This call returns an aggregated opening $\mathbf{s}$ and an aggregated randomness z, which are then stored in $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}]$ and $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]$ respectively. This step effectively generates the required signing randomness on-the-fly without ever knowing the individual components.
- **In $\mathcal{O}_{\mathsf{Corrupt}}$:** When a signer i is corrupted, the challenger must reveal its internal randomness. It now uses individual equivocation by calling $\mathsf{CSim}(\mathsf{eqv}, \mathsf{td}, i, k_i)$ to generate a valid opening $(r_i, \mathbf{s}_i)$ for commitments $\mathbf{C}_i$ simulated for signer i, and $k_i = \mathsf{ComCtr}[i, \mathbf{C}_i]$. This opening is then used to update $\mathsf{HonestMsg}[i, \mathbf{C}_i] \leftarrow (\pi_i, r_i, \mathbf{s}_i)$. If the corrupted signer was the last one needed to complete the second round, the challenger also performs a product equivocation for the remaining honest signers: it computes $(\mathbf{s}, z)$ equivocating $\prod_{j \in \mathsf{sHS} \setminus \{i\}} \mathbf{C}_j$ with CSim and stores $\mathbf{s}$ and z as in $\mathcal{O}_{\mathsf{Sign}_2}$.

Indistinguishability. The indistinguishability of this hybrid from Hyb_{11} relies on the restricted product equivocability property of the commitment scheme, as defined in Theorem 3.10 and Fig. 5. We construct a reduction $\mathcal{B}_4$ that uses the adversary $\mathcal{A}$ distinguishing Hyb_{11} and Hyb_{12} to break this security property.

First, we introduce two intermediate hybrids Hyb'_{11}, which is identical to Hyb_{11} except that x is chosen from $\mathbb{Z}_p^*$ and the commitment key ck is generated using $\mathsf{EqvSetup}$, and Hyb'_{12}, which is identical to Hyb_{12} except that x is chosen from $\mathbb{Z}_p^*$. It is easy to check that the statistical distance between $x \xleftarrow{\$} \mathbb{Z}_p$ and $x \xleftarrow{\$} \mathbb{Z}_p^*$ is at most $2/p$. Thus due to Theorem 3.10 and hybrid argument, we have $|\epsilon_{11} - \epsilon'_{11}| \leqslant 4/p$ and $|\epsilon_{12} - \epsilon'_{12}| \leqslant 2/p$.

The reduction $\mathcal{B}_4$ receives a commitment key ck, $\mathcal{T} = [N]$, and the secret key $x \in \mathbb{Z}_p^*$ from the equivocability challenger and uses it to simulate the view in both hybrids for $\mathcal{A}$. When we need to perform a commitment operation, $\mathcal{B}_4$ forwards the request to its external oracles ($\mathcal{O}_{\mathsf{com}}$, $\mathcal{O}_{\mathsf{eqv}}$, $\mathcal{O}_{\mathsf{prod\text{-}eqv}}$).

- If $\mathcal{B}_4$ is in the **real world** of the equivocability game, its oracles behave like an honest commitment scheme. Its simulation for $\mathcal{A}$ is therefore perfectly distributed according to Hyb'_{11}.
- If $\mathcal{B}_4$ is in the **ideal world**, its oracles perform simulation and equivocation. Its simulation for $\mathcal{A}$ is perfectly distributed according to Hyb'_{12}.

For this reduction to be valid, we must ensure that the queries made by $\mathcal{B}_4$ respect the restrictions of the restricted product equivocability game. We verify two conditions:

1. **Each commitment $\mathbf{C}_i$ is used in at most one individual equivocation, followed by at most one product equivocation.** First, assume that $\mathbf{C}_i$ was used in a product equivocation. This can only happen when signer i completes its second round $\widetilde{\mathsf{ctnt}}$, and thus $\mathbf{C}_i$ cannot be involved in the product equivocation for another $\widetilde{\mathsf{ctnt}}'$ including $\mathbf{C}_i$ and $i \notin \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}']$ since signer i will remain in $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}']$ as long as it's not corrupted.

   Additionally, an individual equivocation on $\mathbf{C}_i$ only happens when signer i is corrupted. After this, i is no longer in the honest set HS, so $\mathbf{C}_i$ will not be part of any future product equivocations as it only runs for commitments from $\mathsf{sHS} \setminus \{i\}$.

2. **For each product equivocation, at least one honest commitment is not individually equivocated.** A product equivocation involves the set of honest signers sHS for a session. For all of these commitments to be individually equivocated later, all signers in sHS would need to be corrupted. This is impossible, as the adversary can corrupt at most $T-1$ signers. The condition holds.

Since the simulation is perfect and the conditions are met, the probability that $\mathcal{A}$ distinguishes the hybrids Hyb'_{11} and Hyb'_{12} is exactly the advantage of $\mathcal{B}_4$ in breaking the equivocability property.

$$\left|\epsilon'_{11} - \epsilon'_{12}\right| \leqslant \mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{B}_4,\mathsf{CSim},[N]}(1^\lambda).$$

Combining the bounds, we get:

$$|\epsilon_{11} - \epsilon_{12}| \leqslant \mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{B}_4,\mathsf{CSim},[N]}(1^\lambda) + \frac{6}{p}.$$

Hyb_{13}. This hybrid completes the process of removing the secret key x from the simulation. The challenger no longer generates x or its shares at the beginning of the game. Instead, the public key X is chosen directly, and each secret share x_i is generated on-the-fly only when a signer i is corrupted. This change is formalized in Fig. 23.

The adversary's view is statistically identical to the previous hybrid. This relies on two key observations:

1. **Signing oracles are independent of secrets:** As a result of the changes in Hyb_{12} and earlier hybrids, the signing oracles for honest parties no longer use the master secret x or any of the individual shares x_i. All operations are performed using simulated or equivocated values.

2. **Corrupted shares are uniformly random:** In a (T, N) Shamir's secret sharing scheme, any set of up to $T-1$ shares are uniformly and independently random. Since the adversary $\mathcal{A}$ can corrupt at most $T-1$ signers, the shares it obtains in Hyb_{12} are computationally indistinguishable from uniformly random values. Generating them uniformly at random in Hyb_{13} is therefore a statistically identical process from the adversary's perspective.

Since the adversary's view is unchanged, we have:

$$\epsilon_{12} = \epsilon_{13}.$$

Hyb_{14}. This final hybrid introduces the "guessing" step required for the reduction. The challenger tries to guess which random oracle query made by the adversary will be used to create a non-trivial forgery. This change is formalized in Fig. 24.

Formally, we introduce the following changes:

- The challenger picks a random integer q^* from $[1, Q_{\mathsf{H}_c} + Q_{\mathsf{S}}]$ at the beginning of the game. This q^* is the challenger's guess for which oracle query will be critical for the forgery.
- When the q^*-th query to H_c is made as part of a signing session with $\widetilde{\mathsf{ctnt}}$ (via $\mathsf{ProgramHashChall}$), the challenger overwrites the normally computed challenge c with a freshly sampled c'. It records that this session (identified by $\widetilde{\mathsf{ctnt}}$) is the "guessed" session by setting $\mathsf{GuessPoint} \leftarrow \widetilde{\mathsf{ctnt}}$.
- The game now aborts if the last honest signer for the guessed session $\mathsf{GuessPoint}$ completes the protocol, or gets corrupted. We can safely ignore this case as the forgery would not be non-trivial, since the corresponding signature would have been legitimately obtained.
- The winning condition is tightened: the adversary now only wins if its forgery corresponds to the guessed query q^*.

$\mathsf{Game}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T)$

// Identical to lines 1-5 in Hyb_1 in Fig. 9.

Initialize empty sets and lists.

$(\mathsf{ctr}_i)_{i\in[N]} := 1$

$x \xleftarrow{\$} \mathbb{Z}_p;\ X \leftarrow G^x$

$\mathsf{ck} \xleftarrow{\$} \mathsf{EqvSetup}(1^\lambda, X)$

$\mathsf{tspar} := (\mathsf{ck}, N, T)$

// Identical to lines from 8 in Fig. 9.

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{sid}, i)$

require $\mathsf{sid} \in [\mathsf{ctr}_{\mathsf{sid}}] \wedge \mathsf{R}[\mathsf{sid}, i] = 0$

parse $(((\mathsf{ck}, N, T), X), x_i, \vec{\mathsf{seed}}_i) \leftarrow (\mathsf{vk}, \mathsf{sk}_i)$

$\mathbf{C}_i := \mathsf{CSim}(\mathsf{cmt}, \mathsf{td}, i, \mathsf{ctr}_i)$

require $\mathbf{C}_i \notin \mathsf{Cmt}_i$

$\mathsf{Cmt}_i \leftarrow \mathsf{Cmt}_i \cup \{\mathbf{C}_i\}$

$\pi_i \leftarrow \mathsf{ZKSim}(\mathsf{prv}, (\mathsf{ck}, \mathbf{C}_i), i)$

$\mathsf{HonestMsg}[i, \mathbf{C}_i] \leftarrow (\pi_i, \bot, \bot)$

$\mathsf{ComCtr}[i, \mathbf{C}_i] \leftarrow \mathsf{ctr}_i$

$\mathsf{ctr}_i \leftarrow \mathsf{ctr}_i + 1$

$\mathsf{CtntSid}[i, \mathbf{C}_i] \leftarrow \mathsf{sid}$

$\mathsf{st}_i \leftarrow \bot$

$\mathsf{pm}_{1,i} := (\mathbf{C}_i, \pi_i)$

$\mathsf{RM}[\mathsf{sid}, 1, i] := \mathsf{pm}_{1,i}; \mathsf{St}[\mathsf{sid}, i] := \mathsf{st}_i, \mathsf{R}[\mathsf{sid}, i] = 1$

return $\mathsf{pm}_{1,i}$

$\mathcal{O}_{\mathsf{Sign}_3}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-6 in Hyb_{11} in Fig. 20.

parse $(\tilde{\mathbf{s}}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}}$

parse $(\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}}) \leftarrow \mathsf{st}_i$

// Identical to lines from 9 in Hyb_{11} in Fig. 20.

$\mathcal{O}_{\mathsf{Sign}_2}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{1,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-8 and 9-26 in Hyb_9 in Fig. 18.

else // the last signer in the second round with $\widetilde{\mathsf{ctnt}}$

for $j \in \mathsf{sCS}$ **do**

$\tilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$

$c \xleftarrow{\$} \mathbb{Z}_p$

for $j \in \mathsf{sHS}$ **do**

parse $k_j \leftarrow \mathsf{ComCtr}[j, \mathbf{C}_j]$

$(\mathbf{s}, z) \leftarrow \mathsf{CSim}(\mathsf{prod\text{-}eqv}, \mathsf{td}, c, \mathsf{sHS}, (j, k_j)_{j\in\mathsf{sHS}})$

$\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathbf{s}$

$\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow z$

parse $(r_j, \mathbf{s}_j)_{j\in\mathsf{sCS}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$

$\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot X^{-c} \cdot \prod_{j\in\mathsf{sCS}\cup\{i\}} G^{r_j}$

$\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$

$\tilde{\mathbf{s}}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \tilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\setminus\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$

$\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \tilde{\mathbf{s}}_i$

$\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \setminus \{i\}$

$\mathsf{st}_{2,i} \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{C}_j)_{j\in\mathsf{SS}})$

$\mathsf{pm}_{2,i} := \tilde{\mathbf{s}}_i$

// Identical to lines 9-13 in $\mathcal{O}_{\mathsf{Sign}_r}$ in Fig. 2.

Figure 21: The first part of the twelfth hybrid. Differences from the previous hybrid are highlighted in blue.

The adversary's advantage in this hybrid is scaled down by the probability of the challenger guessing correctly. The key idea is that from the adversary's perspective, the guess q^* is perfectly hidden. While the programmed c for the q^*-th session is no longer consistent with the simulated commitment and openings for $\widetilde{\mathsf{ctnt}}$, this is fine as long as not all honest signers complete the signing session for $\widetilde{\mathsf{ctnt}}$ as then the adversary would never obtain a valid signature under $\widetilde{\mathsf{ctnt}}$ (the $\tilde{z}_j$ of honest signers who are not the last signer in the final round are just random values in $\mathbb{Z}_p$). The challenger aborts if the last honest signer completes or is corrupted, ensuring that the adversary never sees a valid signature for this session.

The only observable difference would be if the game aborts because a guessed signing session is completed. However, if the adversary does not complete the guessed session, its view is identical to Hyb_{13}.

Below, we check that the winning condition of the unforgeability game requires the adversary to produce a forgery for a message M^* where the signature's challenge c_j was not obtained from a completed signing

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

1 : **require** $[\![i \in \mathsf{HS}]\!] \wedge [\![|\mathsf{CS}| \leqslant T-1]\!]$
2 : **for** $\mathbf{C}_i$ s.t. $\mathsf{HonestMsg}[i, \mathbf{C}_i] \neq \bot$ **do**
3 : **parse** $(\pi, \bot, \bot) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
4 : **parse** $k_i \leftarrow \mathsf{ComCtr}[i, \mathbf{C}_i]$
5 : $(\mathbf{s}_i, r_i) \leftarrow \mathsf{CSim}(\mathsf{eqv}, \mathsf{td}, i, k_i)$
6 : $\mathsf{HonestMsg}[i, \mathbf{C}_i] \leftarrow (\pi_i, r_i, \mathbf{s}_i)$
7 : **parse** $\mathsf{sid} \leftarrow \mathsf{CtntSid}[i, \mathbf{C}_i]$
8 : $\rho_i \leftarrow \mathsf{ZKSim}(\mathsf{rvl}, (\mathsf{ck}, \mathbf{C}_i), (r_i, \mathbf{s}_i), i, \pi)$
9 : $\mathsf{C}[\mathsf{sid}, 1, i] \leftarrow (r_i, \mathbf{s}_i, \rho_i)$
10 : **for** $\widetilde{\mathsf{ctnt}}$ s.t. $i \in \mathsf{InitializeOpen}[\widetilde{\mathsf{ctnt}}]$ **do**
11 : **parse** $0\|\mathsf{SS}\|\mathsf{M}\|(\mathbf{C}_j)_{j\in\mathsf{SS}} \leftarrow \widetilde{\mathsf{ctnt}}$
12 : **if** $i \notin \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]$ **then**
13 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
14 : $\mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}] \cup \{(r_j, \mathbf{s}_j)\}$
15 : **if** $\mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**
16 : **parse** $(\pi_i, r_i, \mathbf{s}_i) \leftarrow \mathsf{HonestMsg}[i, \mathbf{C}_i]$
17 : $\widetilde{\Delta} \leftarrow \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$
18 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\Delta}_i$
19 : **if** $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \neq \bot$ **then**
20 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \mathbf{s}_i$
21 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] - r_i$
22 : **if** $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**
23 : **for** $j \in \mathsf{sCS}$ **do**
24 : $\widetilde{\Delta}_j := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_j[\mathsf{SS}], \widetilde{\mathsf{ctnt}})$
25 : $c \xleftarrow{\$} \mathbb{Z}_p$
26 : **for** $j \in \mathsf{sHS}\backslash\{i\}$ **do**
27 : **parse** $k_j \leftarrow \mathsf{ComCtr}[j, \mathbf{C}_j]$
28 : $(\mathbf{s}, z) \leftarrow \mathsf{CSim}(\mathsf{prod\text{-}eqv}, \mathsf{td}, c, \mathsf{sHS}\backslash\{i\}, (j, k_j)_{j\in\mathsf{sHS}\backslash\{i\}})$
29 : $\mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathbf{s}$
30 : $\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}] \leftarrow z$
31 : **parse** $(r_j, \mathbf{s}_j)_{j\in\mathsf{sCS}\cup\{i\}} \leftarrow \mathsf{Opened}[\widetilde{\mathsf{ctnt}}]$
32 : $\mathsf{SumCom}[\widetilde{\mathsf{ctnt}}] \leftarrow G^{\mathsf{SumComRnd}[\widetilde{\mathsf{ctnt}}]} \cdot X^{-c} \cdot \prod_{j\in\mathsf{sCS}\cup\{i\}} G^{r_j}$
33 : $\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$
34 : $\widetilde{\Delta}_i := \mathsf{SumRnd}[\widetilde{\mathsf{ctnt}}] - \sum_{j\in\mathsf{sCS}} \widetilde{\Delta}_j - \sum_{j\in\mathsf{sHS}\backslash\{i\}} \mathsf{MaskedRnd}[\widetilde{\mathsf{ctnt}}, j]$
35 : $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \leftarrow \widetilde{\Delta}_i$
36 : **if** $\mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i] \neq \bot$ **then**
37 : $\mathsf{ProgramZeroShare}(\widetilde{\mathsf{ctnt}}, i, \mathsf{Mask}_{\mathbf{s}}[\widetilde{\mathsf{ctnt}}, i], \mathsf{sCS}, \mathsf{sHS})$
38 : $\mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{UnOpenedHS}[\widetilde{\mathsf{ctnt}}]\backslash\{i\}$
39 : $\mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}] \leftarrow \mathsf{ImpersonatedHS}[\widetilde{\mathsf{ctnt}}]\backslash\{i\}$
// Identical to lines from 36 in Hyb_{11} in Fig. 20.

Figure 22: The second part of the twelfth hybrid. Differences from the previous hybrid are highlighted in blue.

session (i.e., all honest users complete the final round). If adversary produced two forgeries which include the same $c_j = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}*, R)$, z_j also must be the same since there is only one z_j satisfying $R = g^{z_j} \cdot X^{-c_j}$. Thus forgeries produced by $\mathcal{A}$ have distinct corresponding random oracle queries H_c. Also, $\mathsf{FinishedSSIDs}(\mathsf{M}^*)$ returns a set of $\widetilde{\mathsf{ctnt}}$ with which all honest signer complete the signing session. Therefore, when the adversary succeeds in Hyb_{13}, there must be at least one such a non-trivial forgery including c_j under which at least one honest signer does not complete the signing session. The probability that the challenger correctly guesses the oracle query corresponding to this forgery is at least $1/(Q_{\mathsf{H}_c} + Q_{\mathsf{S}})$. If the guess is correct, the game does not abort, and the adversary's winning output will satisfy the new winning condition.

Therefore, the probability of winning in the previous hybrid is bounded by the probability of winning in this hybrid, scaled by the number of possible guesses:

$$\epsilon_{13} \leqslant (Q_{\mathsf{H}_c} + Q_{\mathsf{S}}) \cdot \epsilon_{14}.$$

Reduction from $\mathsf{SelfTargetDL}$. Finally, we show that there exists an adversary $\mathcal{B}_5$ that solves the $\mathsf{SelfTargetDL}$ problem if $\mathcal{A}$ wins Hyb_{14}. The procedure of $\mathcal{B}_5$ is as follows: $\mathcal{B}_5$ is given X and can access an oracle H. It simulates Hyb_{14} to run $\mathcal{A}$. Specifically, it embeds the received X into X, instead of randomly sampling. When $\mathcal{A}$ makes random oracle queries to H_c, it passes its query to H and sets the response c to $\mathsf{T}_{\mathsf{H}_c}$ if $\mathsf{T}_{\mathsf{H}_c}$ is

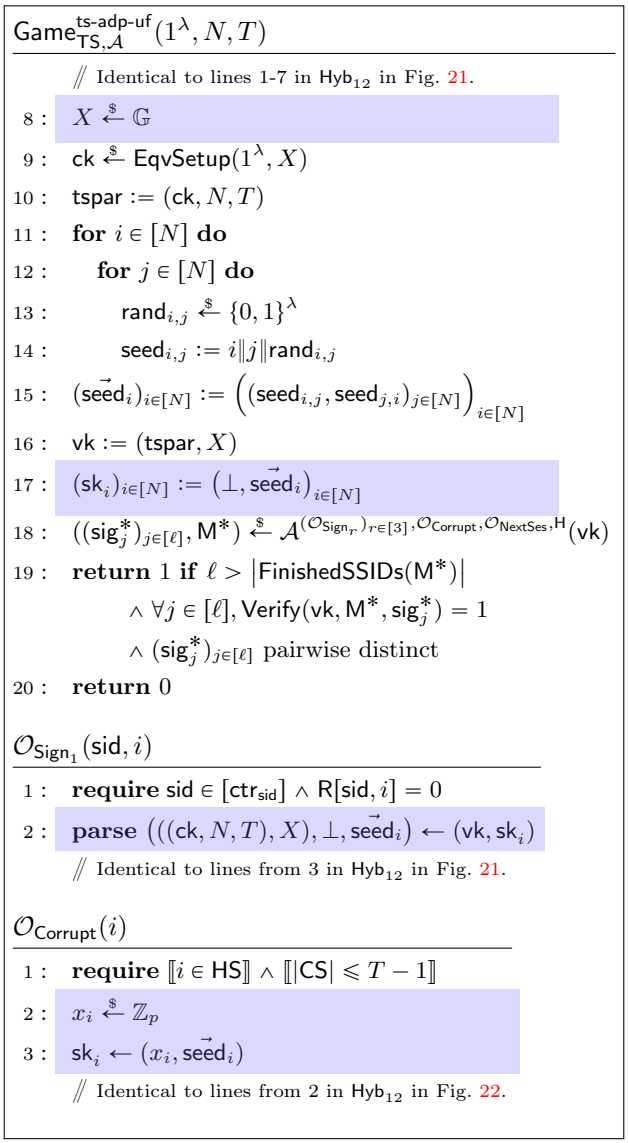

Figure 23: The thirteenth hybrid. Differences from the previous hybrid are highlighted in blue.

not defined. Also, in $\mathsf{ProgramHashChall}$, it queries $(X, \mathsf{M}, \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}])$ to H instead of sampling c'. After $\mathcal{A}$ outputs forgeries, when $\mathcal{B}_5$ verifies a forgery $\mathsf{sig}^*_j = (c_j, z_j)$ such that $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}^*, G^{z_j} \cdot X^{c_j}]$ is not defined, it samples c'_j uniformly at random and regards it as a valid signature if $c'_j = c_j$ holds. We denote the event that such a forgery pass the verification by $\mathsf{E}_{\mathsf{bad}}$. If $\mathcal{A}$ wins the game, it outputs (M^*, c_j, z_j) where (c_j, z_j) is sig^*_j such that c_j is the q^*-th programmed H_c value. As showed in the previous hybrid, $\mathcal{A}$'s outputs must include such sig^*_j when it wins the game, and the q^*-th programmed value is produced by H. Thus, (M^*, c_j, z_j) is a valid solution of the $\mathsf{SelfTargetDL}$ problem. It is easy to check that the probability that $\mathcal{A}$ occurs $\mathsf{E}_{\mathsf{bad}}$ is trivially at most $1/p$. Therefore, we have

$$\epsilon_{14} \leqslant \mathsf{Adv}^{\mathsf{SelfTargetDL}}_{\mathcal{B}_5}(\lambda) + \frac{1}{p}.$$

Note that the number of H queries made by $\mathcal{B}_5$ is at most $\mathsf{H}_c + 1$.

$\mathsf{Game}^{\text{ts-adp-uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T)$

// Identical to lines 1-17 in Hyb_{13} in Fig. 23.

18 : $\mathsf{ctr}_c := 1$

19 : $q^* \xleftarrow{\$} [Q_{\mathsf{H}_c} + Q_{\mathsf{S}}]$

20 : $((\mathsf{sig}^*_j)_{j\in[\ell]}, \mathsf{M}^*) \xleftarrow{\$} \mathcal{A}^{(\mathcal{O}_{\mathsf{Sign}_r})_{r\in[3]}, \mathcal{O}_{\mathsf{Corrupt}}, \mathcal{O}_{\mathsf{NextSes}}, \mathsf{H}}(\mathsf{vk})$

21 : **parse** $(c_j, z_j)_{j\in[\ell]} \leftarrow (\mathsf{sig}^*_j)_{j\in[\ell]}$

22 : $\mathsf{CTR} := \{\alpha \in [\mathsf{ctr}_c] \mid \exists j \in [\ell],\ \mathsf{Ctr}_c[\mathsf{vk}, \mathsf{M}^*, G^{z_j} X^{-c_j}] = \alpha\}$

23 : **return** 1 **if** $\ell > \left|\mathsf{FinishedSSIDs}(\mathsf{M}^*)\right|$

$\wedge\ \forall j \in [\ell], \mathsf{Verify}(\mathsf{vk}, \mathsf{M}^*, \mathsf{sig}^*_j) = 1$

$\wedge\ (\mathsf{sig}^*_j)_{j\in[\ell]}$ pairwise distinct

24 : $\wedge\ q^* \in \mathsf{CTR}$

25 : **return** 0

$\mathsf{H}_c(\mathsf{vk}, \mathsf{M}, R)$

1 : **if** $[\![\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, R] = \bot]\!]$ **then**

2 : $c \xleftarrow{\$} \mathbb{Z}_p$

3 : $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, R] \leftarrow c$

4 : $\mathsf{Ctr}_c[\mathsf{vk}, \mathsf{M}, R] \leftarrow \mathsf{ctr}_c$

5 : $\mathsf{ctr}_c \leftarrow \mathsf{ctr}_c + 1$

6 : **return** $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, R]$

$\mathsf{ProgramHashChall}(\mathsf{vk}, \mathsf{M}, c, \widetilde{\mathsf{ctnt}})$

1 : **require** $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]] = \bot$

2 : **if** $\mathsf{ctr}_c = q^*$ **then**

3 : $c' \xleftarrow{\$} \mathbb{Z}_p$

4 : $c \leftarrow c'$

5 : $\mathsf{GuessPoint} \leftarrow \widetilde{\mathsf{ctnt}}$

6 : $\mathsf{Ctr}_c[\mathsf{vk}, \mathsf{M}, R] \leftarrow \mathsf{ctr}_c$

7 : $\mathsf{ctr}_c \leftarrow \mathsf{ctr}_c + 1$

8 : $\mathsf{T}_{\mathsf{H}_c}[\mathsf{vk}, \mathsf{M}, \mathsf{SumCom}[\widetilde{\mathsf{ctnt}}]] \leftarrow c$

$\mathcal{O}_{\mathsf{Sign}_3}(\mathsf{sid}, \mathsf{SS}, \mathsf{M}, i, (\mathsf{pm}_{2,j})_{j\in\mathsf{SS}})$

// Identical to lines 1-22 in Hyb_{12} in Fig. 21.

23 : **else** // the last signer in the third round with $\widetilde{\mathsf{ctnt}}$

24 : **abort if** $\mathsf{GuessPoint} = \widetilde{\mathsf{ctnt}}$

// Identical to lines from 24 in Hyb_{12} in Fig. 21.

$\mathcal{O}_{\mathsf{Corrupt}}(i)$

// Identical to lines 1-3 in Hyb_{13} in Fig. 23,

// lines 2-39 in Hyb_{12} in Fig. 22,

// and lines 36-41 in Hyb_{11} in Fig. 20.

48 : **if** $\mathsf{UnSignedHS}[\widetilde{\mathsf{ctnt}}] = \{i\}$ **then**

49 : **abort if** $\mathsf{GuessPoint} = \widetilde{\mathsf{ctnt}}$

// Identical to lines from 43 in Hyb_{11} in Fig. 20.

Figure 24: The fourteenth hybrid. Differences from the previous hybrid are highlighted in blue. This game initializes a value $\mathsf{GuessPoint} := \bot$ and a empty list $\mathsf{Ctr}_c[\cdot] := \bot$ at the beginning pf the game.

Combining all bounds above, we obtain

$$\begin{aligned}\mathsf{Adv}^{\text{ts-adp-uf}}_{\mathsf{Rackle},\mathcal{A}}(1^\lambda, N, T) \leqslant\ & \mathsf{Adv}^{\mathsf{SIM\text{-}EXT}}_{\Pi_{\mathsf{aux}},\mathcal{B}_1,\mathsf{ZKSim}}(1^\lambda) + \frac{2p}{p-1}\mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}_2}(\lambda) + \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_3,\mathsf{BCJ}}(\lambda) \\ & + \mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{B}_4,\mathsf{CSim},[N]}(1^\lambda) + (Q_{\mathsf{H}_c} + Q_{\mathsf{S}}) \cdot \mathsf{Adv}^{\mathsf{SelfTargetDL}}_{\mathcal{B}_5}(\lambda) \\ & + \frac{N \cdot Q_{\mathsf{S}}^2}{p^2} + \frac{Q_{\mathsf{H}_{\mathsf{mask}}}}{2^\lambda} + \frac{Q_{\mathsf{S}}(Q_{\mathsf{H}_c} + Q_{\mathsf{S}}) + 10}{p}\end{aligned}$$

where $\mathsf{Time}(\mathcal{B}_1), \mathsf{Time}(\mathcal{B}_2), \mathsf{Time}(\mathcal{B}_3), \mathsf{Time}(\mathcal{B}_4), \mathsf{Time}(\mathcal{B}_5) \approx \mathsf{Time}(\mathcal{A})$. Applying Theorems 3.5, 3.8, 4.1

and 5.6, $r = \lceil \lambda/b \rceil$ for our instantiation of Π_{aux} (c.f. Section C), and assuming the DL problem for GenG is $(\varepsilon_{dl}, T_{dl})$-hard, we have

$$\mathsf{Adv}^{\mathsf{ts\text{-}adp\text{-}uf}}_{\mathsf{Rackle},\mathcal{A}}(1^\lambda, N, T) \leqslant (Q_{\mathsf{H}_c} + Q_{\mathsf{S}}) \cdot \sqrt{(Q_{\mathsf{H}_c} + 1) \cdot \varepsilon_{dl}} + \frac{3p-1}{p-1} \cdot \varepsilon_{dl} + \frac{Q_{\mathsf{H}_{\mathsf{aux}}} + Q_{\mathsf{H}_{\mathsf{mask}}}}{2^\lambda} + \frac{Q_{\mathsf{S}}(Q_{\mathsf{H}_c} + Q_{\mathsf{S}} + Q_{\mathsf{H}_{\mathsf{aux}}} \cdot 2^t) + Q_{\mathsf{H}_c} + 10}{p} + \frac{N \cdot Q_{\mathsf{S}}^2}{p^2}$$

where t is a parameter of Π_{aux} (see Section 5.2) and $2 \cdot \mathsf{Time}(\mathcal{A}) \approx T_{dl}$. This completes the proof. □

Acknowledgement. We thank Shuichi Katsumata for helpful discussions and advice throughout the project. We also thank the anonymous reviewers of Eurocrypt for their valuable feedback and suggestions.

References

[ABHR25] Karen Azari, Cecilia Boschini, Kristina Hostáková, and Michael Reichle. Security amplification of threshold signatures in the standard model. To appear at TCC'25., 2025.

[AF04] Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2004.

[BCJ08] Ali Bagherzandi, Jung Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008*, pages 449–458. ACM Press, October 2008.

[BCK+22] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022.

[BD21] Mihir Bellare and Wei Dai. Chain reductions for multi-signatures and the HBMS scheme. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 650–678. Springer, Cham, December 2021.

[BDLR25a] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 390–422. Springer, Cham, August 2025.

[BDLR25b] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 304–334. Springer, Cham, May 2025.

[BFW15] David Bernhard, Marc Fischlin, and Bogdan Warinschi. Adaptive proofs of knowledge in the random oracle model. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 629–649. Springer, Berlin, Heidelberg, March / April 2015.

[BHLS25] Dan Boneh, Iftach Haitner, Yehuda Lindell, and Gil Segev. Exponent-VRFs and their applications. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 195–224. Springer, Cham, May 2025.

[BLSW24] Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. HARTS: High-threshold, adaptively secure, and robust threshold Schnorr signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 104–140. Springer, Singapore, December 2024.

[BLT+24] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024.

[BW25] Renas Bacho and Benedikt Wagner. T-spoon: Tightly secure two-round multi-signatures with key aggregation. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 256–290. Springer, Cham, August 2025.

[CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Pairing-free blind signatures from CDH assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 174–209. Springer, Cham, August 2024.

[CGRS23] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. Practical Schnorr threshold signatures without the algebraic group model. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 743–773. Springer, Cham, August 2023.

[Che25] Yanbo Chen. Round-efficient adaptively secure threshold signatures with rewinding, 2025.

[CKK+25] Elizabeth C. Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. On the adaptive security of FROST. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 480–511. Springer, Cham, August 2025.

[CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.

[CS25] Elizabeth C. Crites and Alistair Stewart. A plausible attack on the adaptive security of threshold schnorr signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 457–479. Springer, Cham, August 2025.

[DEF+19] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy*, pages 1084–1101. IEEE Computer Society Press, May 2019.

[Des90] Yvo Desmedt. Abuses in cryptography and how to fight them. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 375–389. Springer, New York, August 1990.

[DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990.

[DKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.

[Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 152–168. Springer, Berlin, Heidelberg, August 2005.

[GCRS25] Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. Fully-adaptive two-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Paper 2025/1478, 2025.

[GJKR07] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007.

[HK22] Lucjan Hanzlik and Kamil Kluczniak. Explainable arguments. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 59–79. Springer, Cham, May 2022.

[HOS25] Keitaro Hashimoto, Wakaha Ogata, and Yusuke Sakai. Signatures with tight adaptive corruptions from search assumptions. Cryptology ePrint Archive, Report 2025/115, 2025.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[KMP16] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 33–61. Springer, Berlin, Heidelberg, August 2016.

[KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.

[KRW24] Michael Klooß, Michael Reichle, and Benedikt Wagner. Practical blind signatures in pairing-free groups. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 363–395. Springer, Singapore, December 2024.

[Ks22] Yashvanth Kondi and abhi shelat. Improved straight-line extraction in the random oracle model with applications to signature aggregation. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 279–309. Springer, Cham, December 2022.

[Lin22] Yehuda Lindell. Simple three-round multiparty Schnorr signing with full simulatability. Cryptology ePrint Archive, Report 2022/374, 2022.

[LNÖ25] Anja Lehmann, Phillip Nazarian, and Cavit Özbay. Stronger security for threshold blind signatures. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 335–364. Springer, Cham, May 2025.

[Mak22] Nikolaos Makriyannis. On the classic protocol for MPC Schnorr signatures. Cryptology ePrint Archive, Report 2022/1332, 2022.

[NT24] Sela Navot and Stefano Tessaro. One-more unforgeability for multi - and threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 429–462. Springer, Singapore, December 2024.

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999.

[PKN+25] Rafaël Del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 423–456. Springer, Cham, August 2025.

[PS00] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, June 2000.

[PW24] Jiaxin Pan and Benedikt Wagner. Toothpicks: More efficient fork-free two-round multi-signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 460–489. Springer, Cham, May 2024.

[RRJ+22] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous Schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2551–2564. ACM Press, November 2022.

[Sch22] Tim Scheurer. Universally composable verifiable random oracles. Master's thesis, Karlsruher Institut für Technologie (KIT), 2022. 46.23.01; LK 01.

[Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.

[SS01] Douglas R. Stinson and Reto Strobl. Provably secure distributed Schnorr signatures and a (t, n) threshold scheme for implicit certificates. In Vijay Varadharajan and Yi Mu, editors, *ACISP 01*, volume 2119 of *LNCS*, pages 417–434. Springer, Berlin, Heidelberg, July 2001.

[TSS+24] Kaoru Takemure, Yusuke Sakai, Bagus Santoso, Goichiro Hanaoka, and Kazuo Ohta. More efficient two-round multi-signature scheme with provably secure parameters for standardized elliptic curves. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 107(7):966–988, 2024.

A BCJ commitments are Product Equivocal

We provide the full proof of Theorem 4.1.

Proof. We construct a simulator CSim and show that the adversary's view in the ideal game is identically distributed to its view in the real game.

The Simulator CSim. The simulator CSim is stateful and its core idea is to invert the distributions of the exponents on G and X between commitments generation and equivocation. That is, it initially samples arbitrary exponents in commitments as $\mathbf{C}_{\mathbb{t}} = (G^r, G^a X^b)$, and then for equivocation, we compute the proper invert distribution of the openings so as to match the commitments exponents. Note that the X exponent is crucial for product equivocation as we wish to invert the distribution of $z = \beta \cdot x + \sum_{\mathbb{t} \in I} \alpha_{\mathbb{t}}$ while the simulator is only given access to $X = g^x$. The trapdoor $\mathsf{td} = (u_1, u_2, u_3)$ it receives allows to treat the commitment values as variables and solve for the openings later.

- **Commitment Simulation ($\mathsf{CSim}(\mathsf{cmt}, \mathsf{td}, \mathbb{t}, \mathsf{ctr}_{\mathbb{t}})$):** To simulate a commitment $\mathbf{C}_{\mathbb{t}}$, CSim samples $r, a, b \xleftarrow{\$} \mathbb{Z}_p$ and computes $\mathbf{C}_{\mathbb{t}} = (G^r, G^a X^b)$. It stores the tuple (r, a, b) associated with the identifier $(\mathbb{t}, \mathsf{ctr}_{\mathbb{t}})$.
- **Product Equivocation ($\mathsf{CSim}(\mathsf{prod\text{-}eqv}, \mathsf{td}, \beta, I, (\mathbb{t}, i_{\mathbb{t}})_{\mathbb{t} \in I})$):** To equivocate a product of commitments $\{\mathbf{C}_{\mathbb{t}_j}\}_{j \in I}$ for a given β, the simulator retrieves the stored tuples (r_j, a_j, b_j) for each commitment, where j iterates over the indices provided in $(\mathbb{t}, i_{\mathbb{t}})_{\mathbb{t} \in I}$. It then computes and returns $(\mathbf{s}, z)$, where $\mathbf{s} = (s, t)$, by solving the following system of linear equations over $\mathbb{Z}_p$:

  $$s + tu_1 = \sum_{j \in I} r_j; \quad su_2 + z = \sum_{j \in I} a_j; \quad tu_3 - \beta = \sum_{j \in I} b_j$$

  This system matches the exponents of G and X on both sides of the equation $\prod_{j \in I} \mathbf{C}_j = (G^s H^t, Y^s Z^t G^z X^{-\beta})$. Since $u_3 \in \mathbb{Z}_p^*$, the system has a unique solution for (s, t, z): $t = (\sum_{j \in I} b_j + \beta)/u_3$, $s = \sum_{j \in I} r_j - tu_1$, and $z = \sum_{j \in I} a_j - su_2$.
- **Individual Equivocation ($\mathsf{CSim}(\mathsf{eqv}, \mathsf{td}, \mathbb{t}, i)$):** To equivocate an individual commitment $\mathbf{C}_{\mathbb{t}}$ with stored tuple (r, a, b), the simulator must find (s, t, α) such that the simulated opening matches the real commitments:

  $$(G^r,\, G^a X^b) = (G^s H^t,\, Y^s Z^t G^\alpha)$$

  Using $H = G^{u_1}$, $Y = G^{u_2}$, and $Z = X^{u_3}$, we define the following linear system over $\mathbb{Z}_p$ to match the exponents of G and X:

  $$s + tu_1 = r; \quad su_2 + \alpha = a; \quad tu_3 = b$$

  Solving gives $t = b/u_3, s = r - tu_1, \alpha = a - su_2$. The simulator outputs $(\mathbf{s}, \alpha)$ with $\mathbf{s} = (s, t)$.

Indistinguishability of Games. We show by induction on the number of oracle queries k that the adversary's view is identically distributed in both games. The adversary's view in game $\mathcal{G}$, denoted $\mathsf{View}_k^{\mathcal{G}} := (\mathsf{td}^{\mathcal{G}}, (\mathcal{O}_i^{\mathcal{G}}, \mathsf{in}_i^{\mathcal{G}}, \mathsf{out}_i^{\mathcal{G}})_{1 \leqslant i \leqslant k})$, consists of random variables for the public key $\mathsf{ck}^{\mathcal{G}}$ and the transcript of its first k oracle interactions where $\mathcal{O}_i^{\mathcal{G}}$ denotes the i-th oracle chosen by the adversary and $\mathsf{in}_i^{\mathcal{G}}$ and $\mathsf{out}_i^{\mathcal{G}}$ denote respectively its input and output.

Base Case ($k = 0$): The view $\mathsf{View}_0^{\mathsf{Real}} = (\mathsf{ck}^{\mathsf{Real}}), \mathsf{View}_0^{\mathsf{Ideal}} = (\mathsf{ck}^{\mathsf{Ideal}})$ are generated by $\mathsf{EqvSetup}$ in both games, so they are identically distributed.

Inductive Step: Assume that for some $k \geqslant 0$, the distribution of $\mathsf{View}_k^{\mathsf{Real}}$ and $\mathsf{View}_k^{\mathsf{Ideal}}$ are identical (Inductive Hypothesis, IH). We show this holds for $\mathsf{View}_{k+1}^{\mathsf{Real}}, \mathsf{View}_{k+1}^{\mathsf{Ideal}}$. First, in both games, the adversary's choice of the $(k+1)$-th query, $(\mathcal{O}_{k+1}^{\mathcal{G}}, \mathsf{in}_{k+1}^{\mathcal{G}})$, is a probabilistic function of $\mathsf{View}_k^{\mathcal{G}}$. By the IH, the distribution of the history and the adversary's query, $(\mathsf{View}_k^{\mathcal{G}}, \mathcal{O}_{k+1}^{\mathcal{G}}, \mathsf{in}_{k+1}^{\mathcal{G}})$, is identical in both games. Let $\mathsf{out}_{k+1}^{\mathsf{Real}}$ and $\mathsf{out}_{k+1}^{\mathsf{Ideal}}$

be the random variables for the query's output in the real and ideal games, respectively. The adversary's view is $\mathsf{View}^{\mathcal{G}}_{k+1} = (\mathsf{View}^{\mathcal{G}}_{k}, \mathcal{O}^{\mathcal{G}}_{k+1}, \mathsf{in}^{\mathcal{G}}_{k+1}, \mathsf{out}^{\mathcal{G}}_{k+1})$. To prove that $\mathsf{View}^{\mathcal{G}}_{k+1}$ is identically distributed in both games, we must show that for any fixed history v_k and query (o, i) that can be generated with non-zero probability, the conditional distribution of the output is identical in both games. That is, we show that

$$\begin{aligned}&\Pr[\mathsf{out}^{\mathsf{Real}}_{k+1} | (\mathsf{View}^{\mathsf{Real}}_{k}, \mathcal{O}^{\mathsf{Real}}_{k+1}, \mathsf{in}^{\mathsf{Real}}_{k+1}) = (v_k, o, i)] \\ &\qquad = \Pr[\mathsf{out}^{\mathsf{Ideal}}_{k+1} | (\mathsf{View}^{\mathsf{Ideal}}_{k}, \mathcal{O}^{\mathsf{Ideal}}_{k+1}, \mathsf{in}^{\mathsf{Ideal}}_{k+1}) = (v_k, o, i)].\end{aligned}$$

Case 1: The query is to $\mathcal{O}_{\mathsf{com}}(\mathbb{t})$. The output out_{k+1} is a new commitment $\mathbf{C}$. This output is generated from fresh randomness, independent of the history. In the real game, $\mathbf{C} = (G^{s+tu_1}, G^{su_2+\alpha} X^{tu_3})$ for fresh uniform $(s, t, \alpha) \in \mathbb{Z}_p^3$. In the ideal game, $\mathbf{C} = (G^r, G^a X^b)$ for fresh uniform $(r, a, b) \in \mathbb{Z}_p^3$. Consider the mapping $\phi : \mathbb{Z}_p^3 \to \mathbb{Z}_p^3$ defined by $\phi(s, t, \alpha) = (s + tu_1, su_2 + \alpha, tu_3)$. Its matrix has determinant $-u_3 \neq 0$ (since $u_3 \in \mathbb{Z}_p^*$), hence ϕ is a bijection. A bijective linear map on a finite field preserves uniformity. Therefore, $(s + tu_1, su_2 + \alpha, tu_3)$ is uniform in $\mathbb{Z}_p^3$, just like (r, a, b), and the conditional distribution of $\mathbf{C}$ is identical in both games. This is a neat argument.

Case 2: Equivocation oracles (product and individual). In these cases, the output $\mathsf{out}^{\mathcal{G}}_{k+1}$ is computed deterministically from randomness previously sampled by each game and the query input. The core of the argument is to show that for any given history, the next output an adversary sees is drawn from the exact same probability distribution, regardless of whether they are in the real or ideal game.

Let's denote the set of underlying random variables in the real game as $\mathcal{R}_{\text{Real}}$ and in the ideal game as $\mathcal{R}_{\text{Ideal}}$. The outputs are deterministic functions of these variables: $\mathsf{out}^{\text{Real}} = f_{\text{Real}}(\mathcal{R}_{\text{Real}})$ and $\mathsf{out}^{\text{Ideal}} = f_{\text{Ideal}}(\mathcal{R}_{\text{Ideal}})$. By the induction hypothesis, the distributions of $\mathcal{R}_{\text{Real}}$ and $\mathcal{R}_{\text{Ideal}}$ are identical (in our case, uniform).

To prove the outputs are identically distributed, we find a bijection ϕ that maps the random variables from one game to the other, such that $\mathcal{R}_{\text{Ideal}} = \phi(\mathcal{R}_{\text{Real}})$. If we can then show that the output-computing functions are perfectly aligned through this bijection (i.e., $f_{\text{Ideal}}(\phi(\mathcal{R}_{\text{Real}})) = f_{\text{Real}}(\mathcal{R}_{\text{Real}})$), it means that both outputs are just different ways of computing the exact same function on the exact same underlying probability distribution. Therefore, their conditional distributions must be identical.

When applying this, we only need to consider the subset of random variables that are directly used in the computation of the current output $\mathsf{out}^{\mathcal{G}}_{k+1}$. This is because we are analyzing the conditional distribution given a fixed history. The inductive hypothesis guarantees that the distributions of all variables sampled *before* this step are identical across both games. Therefore, we can treat the randomness from the past as fixed constants and focus our bijection only on the "active" set of random variables that determine the value of the current output.

We now apply this reasoning by defining the proper bijections for two sub-cases, depending on whether a product equivocation has occurred.

1. **Individual Equivocation and C was not part of a previous product equivocation.** The bijection ϕ from Case 1 applies here as well. By construction, CSim computes $(s, t, \alpha) := \phi^{-1}(r, a, b)$ for individual equivocations in the ideal game. Since the underlying randomness (s, t, α) (in the real game) and (r, a, b) (in the ideal game) were only used for this commitment, and ϕ is a bijection, the output distributions are identical.

2. **Product Equivocation or Individual Equivocation with C part of a previous product equivocation.** Fix the set of commitments $\{\mathbf{C}_j\}_{j \in I}$ involved in the product equivocation. This case is more complex because the ideal world computes z as part of a system of equations, and we must account for individual equivocations that share the same underlying randomness.

   The core idea is that, by the game rule that table $\mathcal{T}_{\Pi\text{-}\mathsf{eqv}}[j]$ (for any given product equivocation j) never becomes empty, at least one individual commitment in I, say for index j_0, is not opened individually after the product equivocation. This ensures that the corresponding randomness α_{j_0}

remains hidden and effectively randomizes the aggregated value z. We can express all outputs in both games as a function of the aggregated opening (s,t), the response z, and the individual openings $\{(s_j,t_j,\alpha_j)\}_{j\in I\setminus\{j_0\}}$.

In the real game, we define the aggregated values $z := \beta\cdot x + \sum_{j\in I}\alpha_j$, $s := \sum_{j\in I} s_j$, $t := \sum_{j\in I} t_j$. These definitions establish a bijection to replace the original randomness $(s_{j_0},t_{j_0},\alpha_{j_0})$ with (s,t,z). All outputs can then be expressed as a function of the uniform random variables $\{(s_j,t_j,\alpha_j)\}_{j\in I\setminus\{j_0\}}$ and (s,t,z).

In the ideal game, we aggregate the uses of the randomness $\{(r_j,a_j,b_j)\}_{j\in I}$ across the product and individual equivocations. By construction of CSim, we can express all outputs as a function of $\{(s_j,t_j,\alpha_j)\}_{j\in I\setminus\{j_0\}}$ and (s,t,z), where these are defined by the following system:

$$\begin{cases} s+tu_1 = \sum_{j\in I} r_j; \quad su_2+z = \sum_{j\in I} a_j; \quad tu_3-\beta = \sum_{j\in I} b_j & \text{(product equivocation)} \\ r_j = s_j + t_j u_1, \quad a_j = s_j u_2 + \alpha_j, \quad b_j = t_j u_3 & \text{(individual equivocations for } j\in I\setminus\{j_0\}) \end{cases}$$

This system defines a bijection ϕ' between the set of variables $((s_j,t_j,\alpha_j)_{j\in I\setminus\{j_0\}},s,t,z)$ and the simulator's randomness $(r_j,a_j,b_j)_{j\in I}$. The system is linear with a triangular matrix that has non-zero diagonal entries (since $u_3\in\mathbb{Z}_p^*$), guaranteeing a unique solution. This concludes that the conditional distribution of the output is identical in both games.

Since in all cases the conditional distribution of the next output is identical, we conclude by induction that the adversary's final view is identically distributed in both games. Therefore, $\mathsf{Adv}^{\mathsf{RstEqv}}_{\mathcal{A},\mathsf{CSim},\mathcal{T}}(1^\lambda) = 0$. □

B Labeled Randomized Fischlin is Simulation-Extractable

Here, we prove Theorem 5.6. We refer to Section 5.2 for an overview of the security proof.

Proof. We construct a simulator ZKSim and show that the adversary's probability of distinguishing the real and ideal games are negligible.

The Simulator ZKSim. The simulator ZKSim is stateful. Roughly, it simulates proofs π via programming of the random oracle $\mathcal{O}_\mathsf{H}$ and extracts a witness by finding a related Σ-protocol transcript (*i.e.*, a distinct transcript with identical commitment a) amongst simulated proofs or hash queries $\mathcal{Q}$ to $\mathcal{O}_\mathsf{H}$. In more detail, ZKSim first initializes empty tables $\mathsf{T}_\mathsf{sim}[\cdot]=\bot$, $\mathsf{T}_\mathsf{c}[\cdot]=\varnothing$, $\mathsf{T}_\mathsf{rnd}[\cdot]$, and $\mathsf{T}_\mathsf{st}[\cdot]:=\bot$. These tables are filled as follows.

- $\mathsf{T}_\mathsf{sim}[\mathsf{ctr}] = (\mathbb{x},\mathbb{t},\pi)$ contains the simulated proofs in the ctr-th call to $\mathcal{O}_\mathsf{prove}$.
- $\mathsf{T}_\mathsf{c}[\mathsf{ctr}]$ contains all tuples $(i,\mathbb{x},\mathbb{t},c_i,h_i)$ such that during the ctr-th call to $\mathcal{O}_\mathsf{prove}$ on input $(\mathbb{x},\mathbb{t})$, the oracle $\mathcal{O}_\mathsf{H}$ is queried for challenge c_i and the output is $h_i\neq 0^b$ (or $h_i=0^b$ but $\mathcal{O}_\mathsf{prove}$ outputs $\bot$). These calls are not made explicitly during $\mathcal{O}_\mathsf{prove}$ but later programmed in $\mathcal{O}_\mathsf{rvl}$.
- $\mathsf{T}_\mathsf{rnd}[\mathsf{ctr}]$ contains all sampled challenges in the ctr-th call to $\mathcal{O}_\mathsf{prove}$.
- $\mathsf{T}_\mathsf{st}[\mathsf{ctr}]$ contains the Σ-protocol states $\mathsf{st}_{\mathsf{sim},i}$ employed to simulate a_i in the ctr-th call to $\mathcal{O}_\mathsf{prove}$ for $i\in[r]$.

Throughout the simulation, ZKSim keeps track of all of $\mathcal{A}$'s queries $\mathcal{Q}$ made to $\mathcal{O}_\mathsf{H}$ and the value of $\mathsf{ctr}_\mathsf{prv}$. Denote by $\Pi_\Sigma.\mathsf{ZKSim}$ and $\Pi_\Sigma.\mathsf{ZKExp}$ the simulators due to strong special HVZK of Π_Σ.

- **Hash Simulation ($\mathsf{ZKSim}(\mathsf{hash},x)$):** Simulates $\mathcal{O}_\mathsf{H}$ as in the real game, except ZKSim aborts if on query $x=(\mathbb{x},\mathbb{t},\mathbf{a},i,c_i,z_i)$, there exists some $\mathsf{ctr}\in[\mathsf{ctr}_\mathsf{prv}]$ such that $(i,\mathbb{x},\mathbb{t},c_i,\cdot)\in\mathsf{T}_\mathsf{c}[\mathsf{ctr}]$.

- **Proof Simulation ($\mathsf{ZKSim}(\mathsf{prv}, \mathbb{x}, \mathbb{t})$):** The simulator ZKSim increments $\mathsf{ctr}_{\mathsf{prv}}$ and sets $\mathsf{ctr} \leftarrow \mathsf{ctr}_{\mathsf{prv}}$. For all $i \in [r]$, ZKSim samples pairs $(c_i, h_i) \leftarrow \mathcal{C} \times \{0,1\}^b$, where c_i is sampled without replacement. If $h_i \neq 0^b$, it adds $\{(i, \mathbb{x}, \mathbb{t}, c_i, h_i)\}$ to $\mathsf{T_c}[\mathsf{ctr}_{\mathsf{prv}}]$. Else, if $h_i = 0^b$ it continues to the next i and later, proceeds with the sampled (c_i, h_i) below. If this fails, it outputs $\perp$, but also adds $\{(i, \mathbb{x}, \mathbb{t}, c_i, h_i)\}$ to $\mathsf{T_c}[\mathsf{ctr}_{\mathsf{prv}}]$ for all i with $h_i = 0^b$.

  Then, for $i \in [r]$, it runs $(a_i, z_i, \mathsf{st}_{\mathsf{sim},i}) \leftarrow \Pi_\Sigma.\mathsf{ZKSim}(\mathbb{x}, c_i)$. It stores $(\mathsf{st}_{\mathsf{sim},i})_{i\in[r]}$ in $\mathsf{T_{st}}[\mathsf{ctr}]$ and $(\mathcal{C}_i)_{i\in[r]}$ in $\mathsf{T_{rnd}}[\mathsf{ctr}]$ for later, where $\mathcal{C}_i$ denotes the set of challenges c_i for $i \in [r]$ sampled before. It outputs $\pi = (a_i, c_i, z_i)_{i\in[r]}$.

- **Witness Extraction ($\mathsf{ZKSim}(\mathsf{ext}, \mathbb{x}, \pi, \mathbb{t})$):** The simulator distinguishes two cases.

  Case 1: There exists a tuple $(\mathbb{x}, \mathbb{t}, \pi')$ stored in $\mathsf{T_{sim}}[\mathsf{ctr}]$ for some ctr such that $\pi = (a_i, c'_i, z'_i)_{i\in[r]}$. In this case, we have $\pi \neq \pi'$ but both proofs share the same commitment vector $\mathbf{a}$. Therefore, there exists some $j \in [r]$ such that $(c_j, z_j) \neq (c'_j, z'_j)$.

  Case 2: Otherwise, the tuple $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ is fresh, *i.e.*, no $\mathcal{O}_{\mathsf{prove}}$ query was made in which $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ appears. Let $\mathcal{Q}$ denote all inputs to $\mathcal{O}_{\mathsf{H}}$ so far. The simulator picks $(\mathbb{x}, \mathbb{t}, \mathbf{a}, j, c'_j, z'_j) \in \mathcal{Q}$ such that $(c'_j, z'_j) \neq (c_j, z_j)$.

  The simulator outputs $\mathbb{w} \leftarrow \Pi_\Sigma.\mathsf{ZKExt}((a_j, c_j, z_j), (a_j, c'_j, z'_j))$.

- **Randomness Simulation ($\mathsf{ZKSim}(\mathsf{rvl}, \mathbb{x}, \pi, \mathbb{w}, \mathbb{t}, \pi)$):** The simulator recovers $(\mathcal{C}_i)_{i\in[r]}$ from $\mathsf{T_{rnd}}$. If $\mathsf{T_{st}}[\mathsf{ctr}] = \perp$ (*i.e.*, the ctr-th call to $\mathcal{O}_{\mathsf{prove}}$ output $\perp$), then the simulator samples $\rho_i \leftarrow \mathcal{R}_{\Sigma,\$}$. Else, it recovers $(\mathsf{st}_{\mathsf{sim},i})_{i\in[r]}$ from $\mathsf{T_{st}}$ and sets $\rho_i \leftarrow \Pi_\Sigma.\mathsf{ZKExp}(\mathsf{st}_{\mathsf{sim},i}, \mathbb{w})$ for $i \in [r]$. Then, it sets $(a_i, \mathsf{st}_i) \leftarrow \mathsf{Init}(\mathbb{x}, \mathbb{w}; \rho_i)$ and programs $\mathcal{O}_{\mathsf{H}}$ for all $\{(i, \mathbb{x}, \mathbb{t}, c_i, h_i)\}$ in $\mathsf{T_c}[\mathsf{ctr}_{\mathsf{prv}}]$, *i.e.*, it computes $z_i \leftarrow \mathsf{Resp}(\mathsf{st}_i, c_i)$ and sets $\mathsf{T_H}[\mathbb{x}, \mathbb{t}, \mathbf{a}, i, c_i, z_i] \leftarrow h_i$. Finally, it outputs random coins $\rho = (\rho_i, \mathcal{C}_i)_{i\in[r]}$.

Analysis of $\mathcal{A}$'s advantage. Let us show that our simulator ZKSim is sufficient. Let $\mathcal{A}$ be an PPT adversary. Without loss of generality, we assume that $\mathcal{O}_{\mathsf{rvl}}$ and $\mathcal{O}_{\mathsf{H}}$ is called at most once by $\mathcal{A}$ on identical inputs. Let Q_{H} denote the number of H queries made by $\mathcal{A}$. We proceed with a sequence of hybrid games and denote by ε_i the probability that the ith hybrid outputs 1.

Hyb_1**.** This is the real simulation extractability game. Formally, this is depicted in Fig. 25. By definition, we have

$$\varepsilon_1 = \Pr[\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}real}}_{\Pi,\mathcal{A}}(1^\lambda) = 1].$$

Hyb_2**.** The game aborts its entire execution if $\mathcal{O}_{\mathsf{H}}$ is queried by $\mathcal{A}$ on any of the non-zero H queries within Prove before $\mathcal{O}_{\mathsf{rvl}}$ is called for the simulated proof. The game is given in Fig. 26. Note that we also unroll the definition of $\mathsf{Prove}^{\mathsf{H}}(\mathbb{x}, \mathbb{w}, \mathbb{t}; \rho)$ in $\mathcal{O}_{\mathsf{prove}}$. In more detail, the game initializes a table $\mathsf{T_c}[\cdot] = \varnothing$ at the start. During each $\mathcal{O}_{\mathsf{prove}}$, every sampled c_i in Prove such that $h_i \neq 0^b$, where $h_i = \mathsf{H}(\mathbb{x}, \mathbb{t}, \mathbf{a}, i, c_i, z_i)$, the game stores $\mathsf{T_c}[\mathsf{ctr}] \leftarrow (i, \mathbb{x}, \mathbb{t}, c_i, h_i)$. During $\mathcal{O}_{\mathsf{rvl}}$, the game resets $\mathsf{T_c}[\mathsf{ctr}] \leftarrow \varnothing$. In $\mathcal{O}_{\mathsf{H}}$, the game aborts its entire execution if $\mathcal{A}$ queries $x = (i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$ and there is some counter $i \in [\mathsf{ctr}_{\mathsf{prv}}]$ such that $\mathsf{T_c}[i] = (i, \mathbb{x}, \mathbb{t}, c_i, \cdot)$. Looking ahead, we will delay programming $\mathsf{T_H}$ for all these queries. For this purpose, it will also be convenient to store sampled c_i such that $h_i = 0^b$ in $\mathsf{T_c}[\mathsf{ctr}]$ if $\mathcal{O}_{\mathsf{prove}}$ outputs $\perp$, *i.e.*, more than 2^t iterations are made in the loop.

Observe that the distribution of the game remains identical, therefore it suffices to upper bound the abort probability. Notice that all sampled c_i in $\mathsf{T_c}$ when generating proof π remain information-theoretically hidden from $\mathcal{A}$ (as these are not outputs as part of the proof π) until the randomness ρ used to generate π is revealed in $\mathcal{O}_{\mathsf{rvl}}$. For a fixed $\mathcal{O}_{\mathsf{H}}$ query $x = (i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$, denote the set of challenges c in $\mathsf{T_c}$ at the time by $\mathcal{C}_\$$. (Note that $\mathcal{C}_\$$ also contains challenges c_i that map to 0^b if $\pi = \perp$ is output. Clearly, these are also information-theoretically hidden until the corresponding $\mathcal{O}_{\mathsf{rvl}}$ call.) The probability that $c_i \in \mathcal{C}_\$$ is

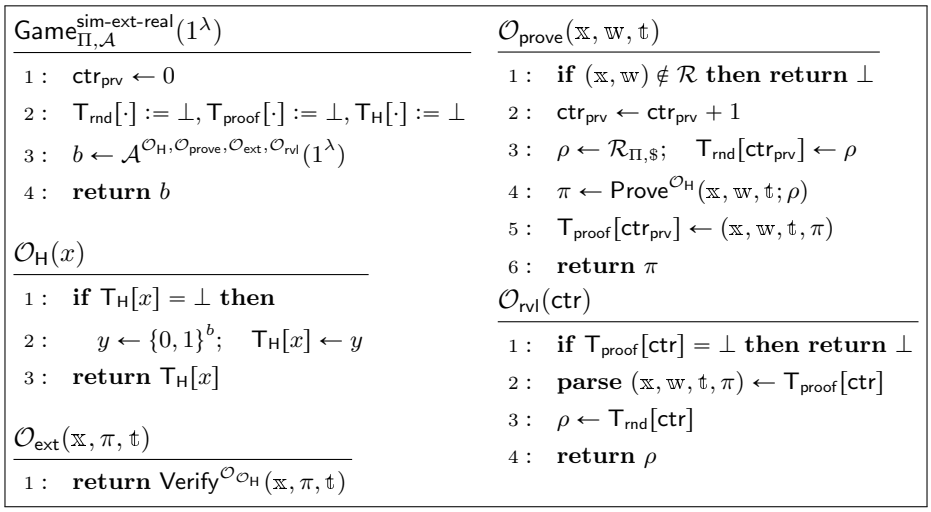

Figure 25: The first hybrid game Hyb_1. Differences from the previous hybrid are highlighted in blue.

at most $|\mathcal{C}_\$|/|\mathcal{C}|$. Since there are at most 2^t challenges c_i sampled and added to $\mathcal{C}_\$$ per $\mathcal{O}_{\mathsf{prove}}$ call, we have $|\mathcal{C}_\$| \leqslant Q_{\mathsf{prv}} \cdot 2^t$. A union bound over all $\mathcal{O}_{\mathsf{H}}$ calls yields

$$|\varepsilon_1 - \varepsilon_2| \leqslant \frac{Q_{\mathsf{H}} \cdot Q_{\mathsf{prv}} \cdot 2^t}{|\mathcal{C}|}.$$

Hyb_3**.** The game programs the random oracle $\mathcal{O}_{\mathsf{H}}$ in $\mathcal{O}_{\mathsf{prove}}$ and $\mathcal{O}_{\mathsf{rvl}}$ as described in Fig. 27. That is, after incrementing $\mathsf{ctr}_{\mathsf{prv}}$ in $\mathcal{O}_{\mathsf{prove}}$, for all $i \in [r]$, it samples pairs $(c_i, h_i) \leftarrow \mathcal{C} \times \{0,1\}^b$, where c_i is sampled without replacement. If $h_i \neq 0^b$, it adds $\{(i, \mathbb{x}, \mathbb{t}, c_i, h_i)\}$ to $\mathsf{T}_{\mathsf{c}}[\mathsf{ctr}_{\mathsf{prv}}]$. Else, if $h_i = 0^b$, it proceeds as before with c_i (*i.e.*, it samples $\mathbf{a}$ via Init and executes the loop with the pairs (c_i, h_i) with $h_i = 0^b$ sampled above) and programs $\mathsf{T}_{\mathsf{H}}[\mathbb{x}, \mathbb{t}, \mathbf{a}, i, c_i, z_i, \mathsf{st}_i] \leftarrow h_i$. Finally, it stores $(\mathsf{st}_i, a_i)_{i\in[r]}$ in table $\mathsf{T}_{\mathsf{st}}[\mathsf{ctr}_{\mathsf{prv}}]$ and outputs π as before. During $\mathcal{O}_{\mathsf{rvl}}$, the game programs executes the loop for all pairs (c_i, h_i) with $h_i \neq 0^b$ and programs H accordingly using $\mathsf{T}_{\mathsf{st}}[\mathsf{ctr}]$.

The distribution of the game does not change since due to the abort condition from the prior game, the delayed programming of T_{H} is not noticeable. Also, notice that the loop in $\mathcal{O}_{\mathsf{prove}}$ has been reorganized but except for the programming of T_{H} and the addition of T_{st}, the oracle $\mathcal{O}_{\mathsf{prove}}$ behaves identically in Hyb_2 and Hyb_3. Therefore, we have

$$\varepsilon_2 = \varepsilon_3.$$

Hyb_4**.** Denote by $\Pi_\Sigma.\mathsf{ZKSim}$ and $\Pi_\Sigma.\mathsf{ZKExp}$ the PPT simulators due to explainable special HVZK of Π_Σ. The game employs $\Pi_\Sigma.\mathsf{ZKSim}$ to compute $\mathbf{a}$ and $(z_1, \ldots, z_r)$ in $\mathcal{O}_{\mathsf{prove}}$. Also, the game employs the witness and $\Pi_\Sigma.\mathsf{ZKExp}$ to compute the state st_i and coins ρ_i for Π_Σ in $\mathcal{O}_{\mathsf{rvl}}$. Given st_i, it recomputes z_i to program T_{H} as in Hyb_3. The changes are formalized in Fig. 28.

The games distribution differs in the way the transcripts $(a_i, c_i, z_i)_{i\in[r]}$ and the coins ρ_i and states st_i are computed: they are either computed via Π_Σ or simulated via $\Pi_\Sigma.\mathsf{ZKSim}$ and $\Pi_\Sigma.\mathsf{ZKExp}$. Under explainable special HVZK, we have

$$\varepsilon_3 = \varepsilon_4.$$

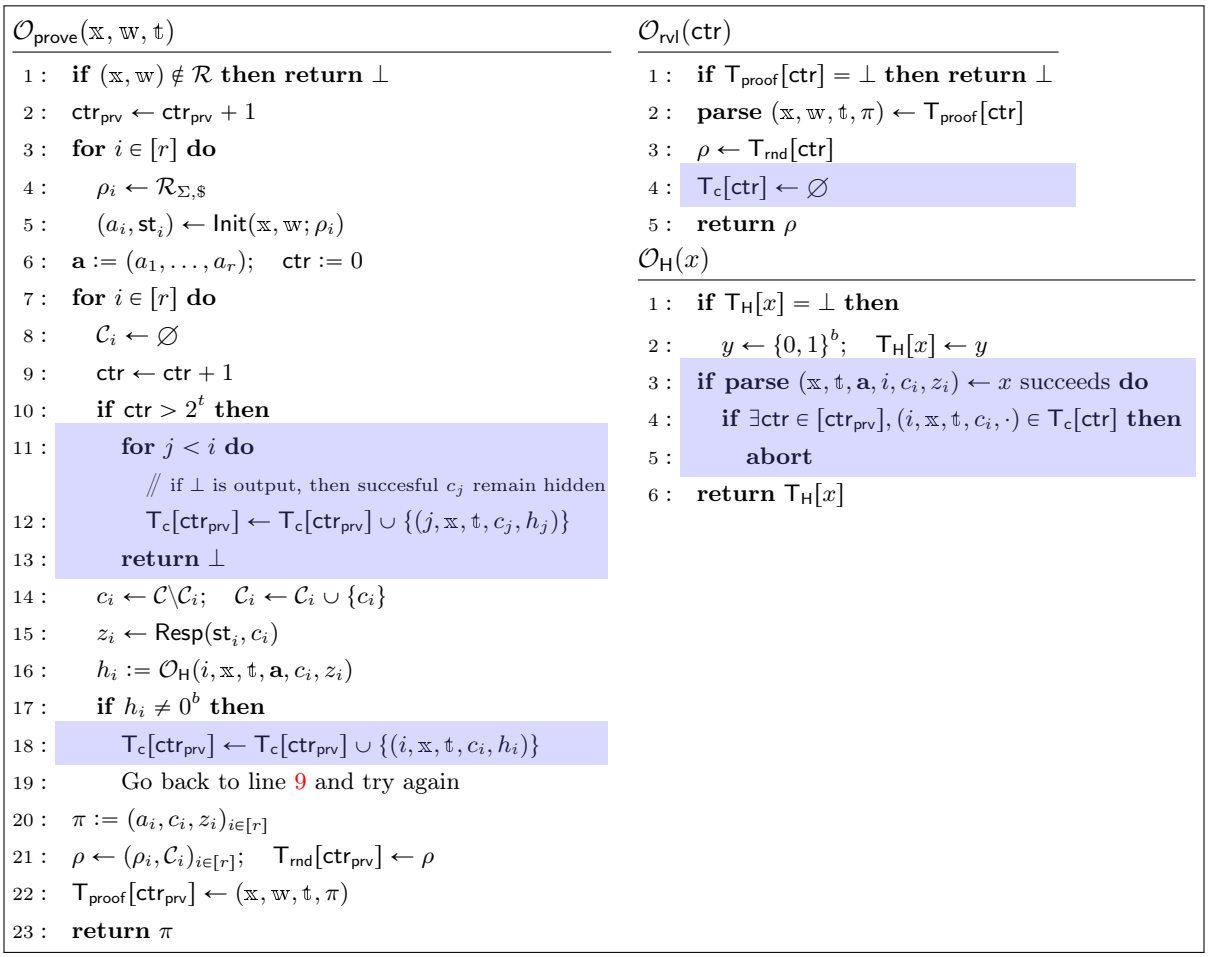

Figure 26: The second hybrid game Hyb_2. Differences from the previous hybrid are highlighted in blue.

Hyb_5. The game extracts a witness $\mathbb{w}$ in the extraction oracle as follows (cf. Fig. 29). Denote by $\Pi_\Sigma.\mathsf{ZKExt}$ the strong 2-special soundness extractor from Π_Σ. Looking ahead, as the SIM-EXT simulator does not have access to the table $\mathsf{T_{proof}}$, the game initializes another table $\mathsf{T_{sim}}$ to store the tuples $(\mathbb{x}, \mathbb{t}, \pi)$ in $\mathcal{O}_{\mathsf{prove}}$. Then, each $\mathcal{O}_{\mathsf{ext}}$ query $(\mathbb{x}, \pi, \mathbb{t})$, the game checks that $b = 1$ and that tuple $(\mathbb{x}, \pi, \mathbb{t})$ is fresh, *i.e.*, it has not been stored in $\mathsf{T_{sim}}$. If so, then the game distinguishes two cases.

Case 1: There exists a tuple $(\mathbb{x}, \mathbb{t}, \pi')$ stored in $\mathsf{T_{sim}}[\mathsf{ctr}]$ for some ctr such that $\pi = (a_i, c'_i, z'_i)_{i \in [r]}$. In this case, we have $\pi \neq \pi'$ but both proofs share the same commitment vector $\mathbf{a}$. Therefore, there exists some $j \in [r]$ such that $(c_j, z_j) \neq (c'_j, z'_j)$.

Case 2: Otherwise, the tuple $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ is fresh, *i.e.*, no $\mathcal{O}_{\mathsf{prove}}$ query was made in which $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ appears. Let $\mathcal{Q}$ denote all inputs to $\mathcal{O}_{\mathsf{H}}$ sofar. The game picks $(\mathbb{x}, \mathbb{t}, \mathbf{a}, j, c'_j, z'_j) \in \mathcal{Q}$ such that $(c'_j, z'_j) \neq (c_j, z_j)$.

Then, the game sets $\mathbb{w} \leftarrow \Pi_\Sigma.\mathsf{ZKExt}((a_j, c_j, z_j), (a_j, c'_j, z'_j))$ and outputs 0 if $(\mathbb{x}, \mathbb{w}) \notin \tilde{R}$.

By strong 2-special soundness of Π_Σ, we have $(\mathbb{x}, \mathbb{w}) \in \tilde{R}$ in both cases if an appropriate transcript (a_j, c'_j, z'_j) is found. Therefore, the game never outputs 0 if $b = 1$ (as in the previous game). In case 1, it is clear that such a transcript must exist. It remains to argue that in the second case, such an $\mathcal{O}_{\mathsf{H}}$ query must exist amongst $\mathcal{Q}$. For this, observe that since $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ is fresh, the random oracle $\mathcal{O}_{\mathsf{H}}$ was never programmed for such inputs. Thus, it can be shown as in [Ks22, Theorem 6.4] that this happens with

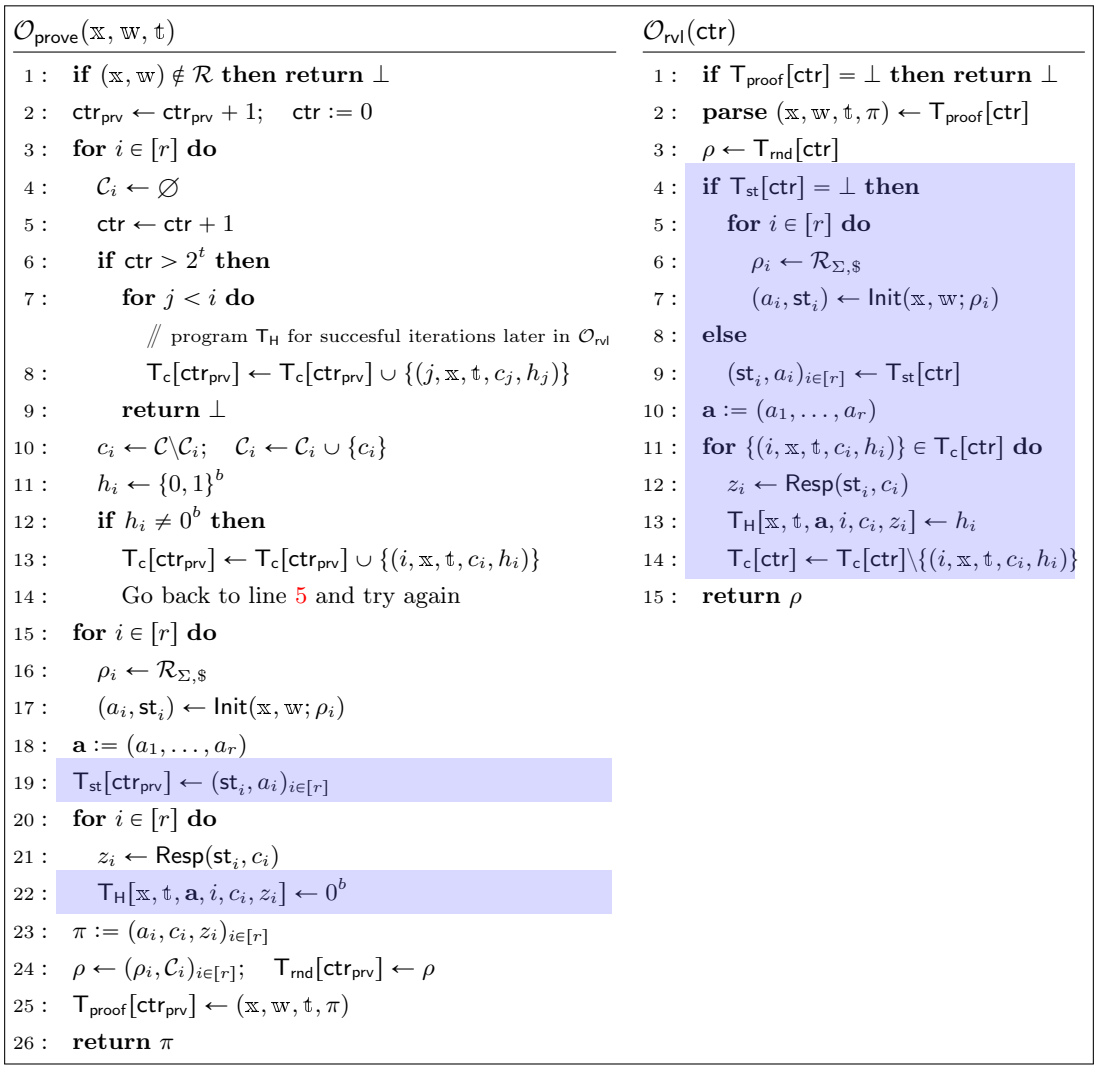

Figure 27: The third hybrid game Hyb_3. Differences from the previous hybrid are highlighted in blue. Note that if $\mathcal{O}_{\mathsf{prove}}$ outputs $\bot$ in the ctr-th run, then $\mathbf{a}$ has not been sampled yet when $\mathcal{O}_{\mathsf{rvl}}(\mathsf{ctr})$ is invoked. Then, in $\mathcal{O}_{\mathsf{rvl}}$ we have $\mathsf{T_{st}}[\mathsf{ctr}] = \bot$. In this case, $\mathcal{O}_{\mathsf{rvl}}$ samples $\mathbf{a}$ via Init as before.

overwhelming probability. In particular, extraction fails if not such query exists. This happens if $\mathcal{A}$ queried $\mathcal{O}_{\mathsf{H}}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i)$ with accepting transcript $(\mathbb{x}, a_i c_i, z_i)$ only once for every $i \in [r]$. However, the proof is only accepting if $\mathcal{O}_{\mathsf{H}}(i, \mathbb{x}, \mathbb{t}, \mathbf{a}, c_i, z_i) = 0^b$ which happens with probability 2^{-b}. Note that here, it is important that $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ is fresh. Therefore, the extractor fails for proof π with probability at most $(2^{-b})^r = 2^{-rb}$. Denote by Q_{H} the number of $\mathcal{O}_{\mathsf{H}}$ queries. Due to a union bound over all possible $\mathbf{a}$ which $\mathcal{A}$ may have tried, it follows that extraction fails except with probability $Q_{\mathsf{H}} \cdot 2^{-rb}$. In conclusion, we have

$$|\varepsilon_4 - \varepsilon_5| \leqslant Q_{\mathsf{H}} \cdot 2^{-rb}.$$

Observe that Hyb_5 is identical to the ideal game. Therefore, we have

$$\varepsilon_5 = \Pr[\mathsf{Game}^{\mathsf{sim\text{-}ext\text{-}ideal}}_{\Pi,\mathcal{A},\mathsf{ZKSim}}(1^\lambda) = 1].$$

Collecting the above bounds yields the claim. □

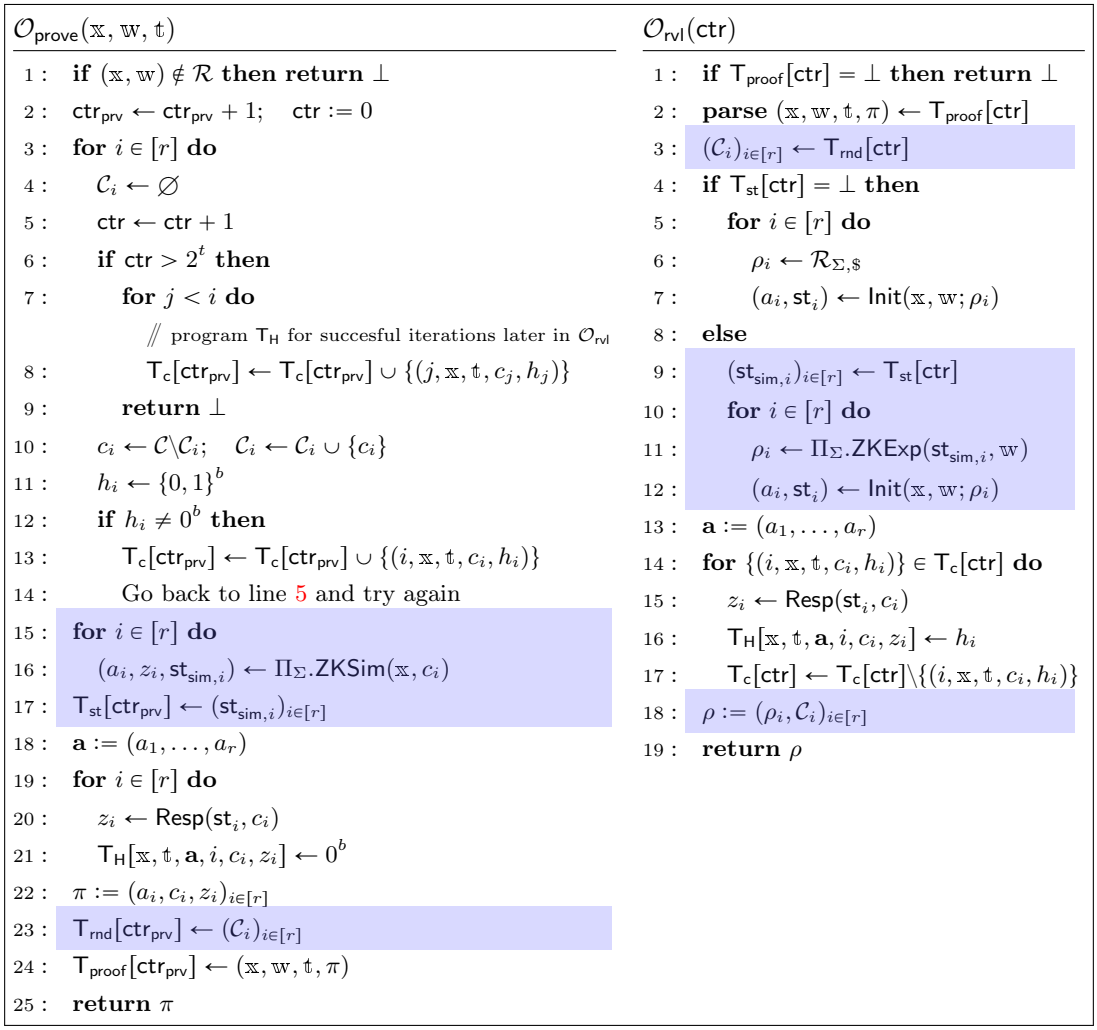

Figure 28: The fourth hybrid game Hyb_4. Differences from the previous hybrid are highlighted in blue.

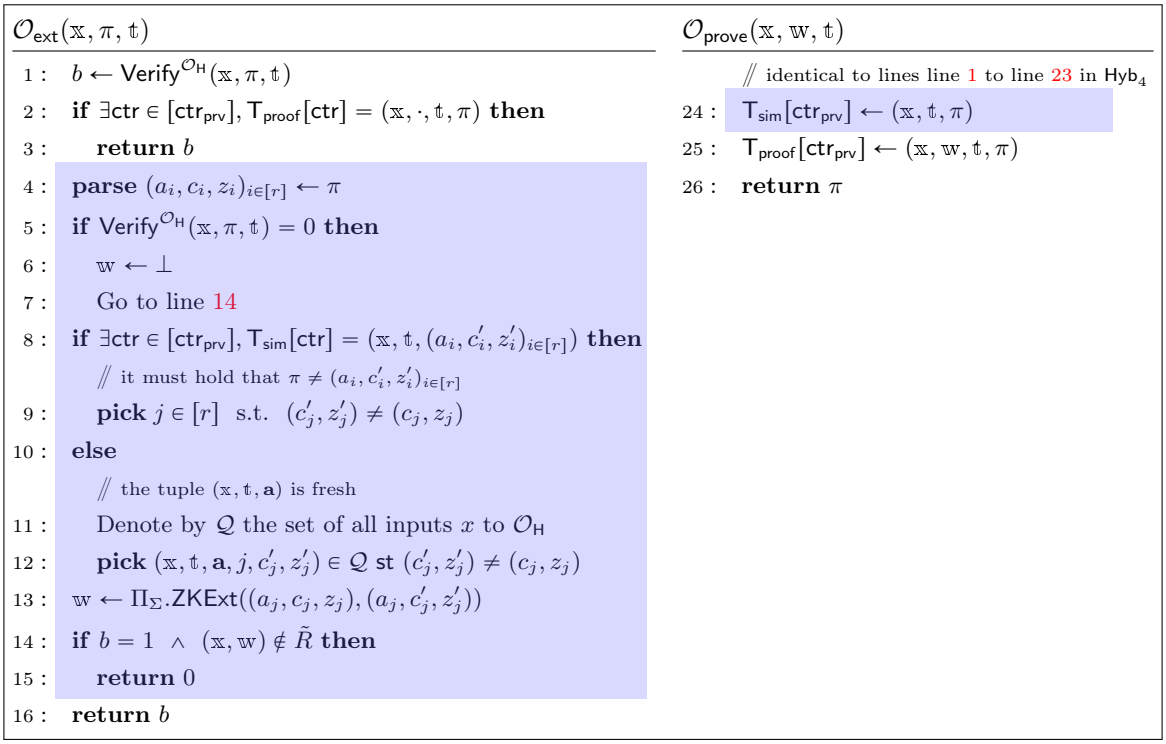

Figure 29: The fifth hybrid game Hyb_5. Differences from the previous hybrid are highlighted in blue. Above, the statement $(\mathbb{x}, \mathbb{t}, \mathbf{a})$ is fresh means that this tuple did not appear in any $\mathcal{O}_{\mathsf{prove}}$ query so far, *i.e.*, no proof π was simulated for these tuples. Therefore, the random oracle H has not been programmed yet on such inputs.

C Proving Knowledge for BCJ Commitments

In this section, we describe an NIZK Π_{aux} that allows to prove knowledge of an opening for BCJ commitments. For this purpose, we construct an appropriate Σ-protocol Π_Σ. Let us first recall the relations $\mathcal{R}_{\mathsf{aux}}$ for Π_Σ and its knowledge relation $\tilde{\mathcal{R}}_{\mathsf{aux}}$ (cf. Eqs. (1) and (2)). Let

$$\mathcal{R}_{\mathsf{aux}} := \{(\mathbb{x}, \mathbb{w}) \mid \mathbf{C} = (G^s H^t, Y^s Z^t M) \ \wedge \ M = G^r\},$$

where $\mathbb{x} = (G, Y, H, Z, \mathbf{C}) \in \mathbb{G}^6$ and $\mathbb{w} = (r, s, t) \in \mathbb{Z}_p^3$, and

$$\tilde{\mathcal{R}}_{\mathsf{aux}} := \{(\mathbb{x}, \mathbb{w}) \mid ((G, Y, Z), \mathbb{w}) \in \mathcal{R}_{\mathsf{dl}} \ \vee \ (\mathbb{x}, \mathbb{w}) \in \mathcal{R}_{\mathsf{aux}}\},$$

where $\mathbb{x} = (G, Y, H, Z, \mathbf{C})$ is as above and $\mathcal{R}_{\mathsf{dl}}$ is defined as

$$\mathcal{R}_{\mathsf{dl}} := \{((G_i)_{i\in[3]}, (x_i)_{i\in[3]}) \mid 1 = \textstyle\prod_{i\in[3]} G_i^{x_i} \ \wedge \ \exists i \in [3], x_i \neq 0\}. \tag{7}$$

The Σ-protocol Π_Σ with challenge space $\mathcal{C} := \mathbb{Z}_p$ is given in Fig. 30. The NIZK $\Pi_{\mathsf{aux}} := \mathsf{RF}[\Pi_\Sigma]$ is obtained by compiling Π_Σ via the randomized Fischlin transform. Below, we show that Π_Σ satisfies all required properties for Theorems 5.5 and 5.6. Therefore, Π_{aux} is correct and simulation-extractable.

$\mathsf{Init}(\mathbb{x}, \mathbb{w})$

1 : **parse** $(G, Y, H, Z, \mathbf{C}) \leftarrow \mathbb{x}$
2 : **parse** $(r, s, t) \leftarrow \mathbb{w}$
3 : $\boldsymbol{\rho} \leftarrow \mathbb{Z}_p^3$
4 : $\mathbf{A} := (G^{\rho_2} H^{\rho_3}, Y^{\rho_2} Z^{\rho_3} G^{\rho_1})$
5 : $\mathsf{st} := ((r, s, t), \boldsymbol{\rho})$
6 : **return** $(\mathbf{A}, \mathsf{st})$

$\mathsf{Resp}(\mathsf{st}, c)$

1 : **parse** $(\mathbf{w}, \boldsymbol{\rho}) \leftarrow \mathsf{st}$
2 : $\mathbf{z} := c \cdot \mathbf{w} + \boldsymbol{\rho}$
3 : **return** $\mathbf{z}$

$\mathsf{Verify}(\mathbb{x}, \mathbf{A}, c, \mathbf{z})$

1 : **require** $c \in \mathbb{Z}_p$
2 : **if** $A_1 \cdot C_1^c \neq G^{z_2} H^{z_3}$ **then return** 0
3 : **if** $A_2 \cdot C_2^c \neq Y^{z_2} Z^{z_3} G^{z_1}$ **then return** 0
4 : **return** 1

Figure 30: The Σ-protocol Π_Σ for relation $\mathcal{R}_{\mathsf{aux}}$.

Instantiation of Π_{aux}. For our concrete instantiation, we set $b = 8$ and $r = \lceil \lambda/b \rceil$. Note that $k = \log p$. Let $\lambda = 128$. For groups $\mathbb{G}$ of size 2λ bits, this leads to proofs of size 3.07 KB.

Security Analysis

Let us show that Π_Σ is correct, explainable special HVZK, strong 2-special sound for $\tilde{R}$ and has high min-entropy.

Correctness. Correctness is immediate.

$$\begin{aligned} A_1 \cdot C_1^c &= G^{\rho_2} H^{\rho_3} \cdot (G^s H^t)^c \\ &= G^{\rho_2 + c\cdot s} \cdot H^{\rho_3 + c\cdot t} \\ &= G^{z_2} \cdot H^{z_3} \end{aligned}$$

$$\begin{aligned} A_2 \cdot C_2^c &= Y^{\rho_2} Z^{\rho_3} G^{\rho_1} \cdot (Y^s Z^t G^r)^c \\ &= Y^{\rho_2 + c\cdot s} \cdot Z^{\rho_3 + c\cdot t} \cdot G^{\rho_1 + c\cdot r} \\ &= Y^{z_2} \cdot Z^{z_3} \cdot G^{z_1} \end{aligned}$$

Explainable Special HVZK. Observe that the randomness $\boldsymbol{\rho}$ is recoverable given $\mathbb{w}$. Therefore, the folklore simulator ZKSim suffices and ZKExp recomputes $\boldsymbol{\rho}$ given $\mathbb{w}$ as follows.

- $\mathsf{ZKSim}(\mathbb{x}, c)$: On input $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$ and $c \in \mathbb{Z}_p$, parses $\mathbb{x} = (G, Y, H, Z, \mathbf{C})$, samples $\mathbf{z} \leftarrow \mathbb{Z}_p^3$ and sets

$$A_1 := G^{z_2} \cdot H^{z_3} \cdot C_1^{-c}; \quad A_2 := Y^{z_2} \cdot Z^{z_3} \cdot G^{z_1} \cdot C_2^{-c}.$$

  Sets $\mathbf{A} := (A_1, A_2)$ and $\mathsf{st}_{\mathsf{sim}} := (c, \mathbf{z})$. Outputs $(\mathbf{A}, \mathbf{z}, \mathsf{st}_{\mathsf{sim}})$.

- $\mathsf{ZKExp}(\mathsf{st}_{\mathsf{sim}}, \mathbb{w})$: On input $\mathbb{w} = (r, s, t)$ and $\mathsf{st}_{\mathsf{sim}} = (c, \mathbf{z})$, sets $\mathbf{w} = (r, s, t)$ and outputs $\boldsymbol{\rho} := (\mathbf{z} - c \cdot \mathbf{w})$.

It is clear that the real and simulated distributions of $(\mathbf{A}, c, \mathbf{z}, \boldsymbol{\rho})$ are identical: the value c is identical in both distirbutions and $(\mathbf{A}, \mathbf{z})$ is uniform conditioned on the verification equation for fixed c. Also, $\boldsymbol{\rho}$ is unique given $\mathbb{w}$ and $(c, \mathbf{z})$.

Strong 2-special Soundness. Given two accepting transcripts $(\mathbf{A}, c, \mathbf{z})$ and $(\mathbf{A}, c', \mathbf{z}')$ with $(c, \mathbf{z}) \neq (c', \mathbf{z}')$ for statement $\mathbb{x} = (G, Y, H, Z, \mathbf{C})$, we have

$$\begin{aligned} A_1 &= G^{z_2} \cdot H^{z_3} \cdot C_1^{-c}, \\ A_2 &= Y^{z_2} \cdot Z^{z_3} \cdot G^{z_1} \cdot C_2^{-c}. \end{aligned} \qquad \text{and} \qquad \begin{aligned} A_1 &= G^{z_2'} \cdot H^{z_3'} \cdot C_1^{-c'}, \\ A_2 &= Y^{z_2'} \cdot Z^{z_3'} \cdot G^{z_1'} \cdot C_2^{-c'}. \end{aligned}$$

Let $\Delta\mathbf{z} := \mathbf{z} - \mathbf{z}'$ and $\Delta c := c - c'$. We obtain

$$\begin{aligned} C_1^{\Delta c} &= G^{\Delta z_2} \cdot H^{\Delta z_3}, \\ C_2^{\Delta c} &= Y^{\Delta z_2} \cdot Z^{\Delta z_3} \cdot G^{\Delta z_1}. \end{aligned}$$

If $\Delta c \neq 0$, then $\mathbb{w} := \Delta\mathbf{z}/\Delta c$ is a valid witness for $\mathbb{x}$. Otherwise if $\Delta c = 0$, we have $\mathbf{z} \neq 0$. Since

$$Y^{\Delta z_2} \cdot Z^{\Delta z_3} \cdot G^{\Delta z_1} = 1$$

we have $((G, Y, Z), \mathbf{z}) \in \mathcal{R}_{\mathsf{dl}}$. It is straightforward to describe a suitable extractor ZKExt for $\mathcal{R}_{\mathsf{aux}}$ given the above considerations.

High min-entropy. Since G generates $\mathbb{G}$, the min-entropy of $\mathbf{A}$ is at least $\log p$.